National University of Computer and Emerging Sciences
Lahore Campus

MLOps, Fall 2026

VisionGuard

AI vs Real Image Classification and Deployment

Assignment 2: `23L-0764_A2_Ops`

Name	Anas Naveed Butt
Roll Number	23L-0764
Section	7-D
Instructor	Usman Anwer
Submission date	6 November 2026

Project Information

Links and graded version

GitHub repository	https://github.com/Anasnaveed69/23L-0764_A2_Ops
DagsHub project (code, data, experiments, registry)	https://dagshub.com/Anasnaveed69/23L-0764_A2_Ops
MLflow tracking server	https://dagshub.com/Anasnaveed69/23L-0764_A2_Ops.mlflow
Live model API (cloud)	https://20-219-10-43.sslip.io (`GET /health`, `POST /v1/predict`)
Web application	Flask on `http://localhost:5000` calling the API above (Task 8 design)
Commit hash of graded version	`9fbde8c`

System at a glance

VisionGuard is a web service that estimates whether an uploaded image was made by a generative model or taken with a camera. The data comes from a Kaggle dataset, which I audited and cleaned and then versioned with DVC on DagsHub. A `tf.data` pipeline feeds two Keras models, a CNN trained from scratch and a fine-tuned MobileNetV2, and every run is tracked in MLflow on DagsHub. The best model is registered with the alias `champion` and served by a model API container on an Azure VM behind HTTPS. A Flask web app, which runs in Docker Compose with a PostgreSQL audit trail, calls that API.

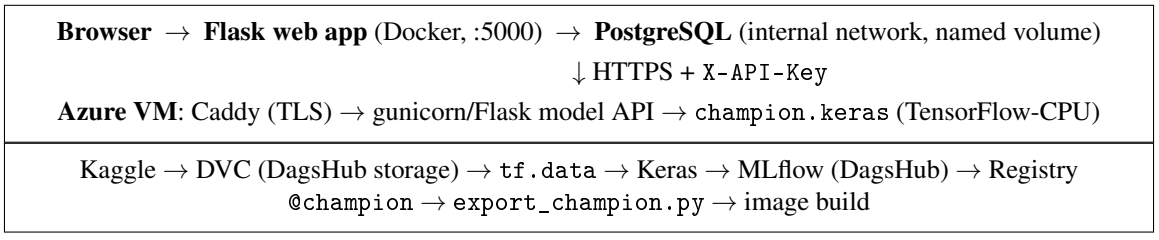


Figure 1: Serving path (top) and training path (bottom).

Disclosure of AI assistance and tutorials

I used an AI assistant, Claude (Anthropic), a lot in this assignment. Claude wrote most of the code in the repository (data scripts, training and tracking code, the Flask app, the API, Dockerfiles, Compose files, the CI workflow and the Colab notebooks), and I reviewed it. Through the Claude desktop app it also operated my browser to run the Colab notebooks, create the GitHub and DagsHub repositories and set up the Azure VM from Cloud Shell, while I signed in to each service, created the access tokens and entered every secret myself. Claude drafted the explanations in this report, and I checked them against my results and edited them. All numbers, tables and plots come from the actual runs: the project scripts write them from MLflow and the evaluation code, so none of them were typed in by hand.

Documentation I used: the TensorFlow/Keras guides (`tf.data`, transfer learning) and the MLflow, DagsHub, DVC, Docker/Compose, GitHub Actions and Azure documentation.

How this report is organised

Each chapter is one task of the assignment and each numbered section is one sub-task. Commands are explained where they are first used. Tables built by the project scripts live in `report/generated/`, so their numbers always match the runs.

Contents

Project Information **i**

1 Data Understanding and Versioning **1**

1.1 Environment 1

1.2 Dataset audit 2

1.3 Visual EDA 5

1.4 Data version control with DVC 6

2 Model Development with TensorFlow/Keras **12**

2.1 Input pipeline 12

2.2 Baseline CNN from scratch 13

2.3 Transfer learning (MobileNetV2) 15

2.4 Training discipline 17

2.5 Evaluation 18

2.6 Error analysis 19

2.7 Tool lab: Keras Tuner 20

3 Experiment Tracking with MLflow and DagsHub **25**

3.1 Tool lab: MLflow locally 25

3.2 DagsHub setup 27

3.3 Instrumenting VisionGuard training 28

3.4 Artifacts 31

3.5 Experiment comparison 32

3.6 Model registry 34

4 Flask Web Application **37**

4.1 Inference module 37

4.2 Routes and validation 38

4.3 HTML and CSS 39

4.4 Configuration and production server 42

5 Database Trail **43**

6 Containerization with Docker and Docker Compose **46**

6.1 Docker warm-up 46

6.2 Dockerfile for the web app 49

6.3 Compose stack 51

6.4 Image analysis 54

6.5 Operations and troubleshooting 56

6.6 Tool lab: GitHub Actions (CI) 59

7 API Deployment **63**

8 Integrating Flask and the Cloud API **71**

9 Conclusion and Limitations **75**

1. Data Understanding and Versioning

1.1 Environment

I kept every part of the project in its own isolated environment so that library versions do not clash and the same setup can be rebuilt later.

For training and data work I used Google Colab. Each session is a fresh Linux VM with an NVIDIA T4 GPU that is thrown away afterwards. Colab already ships TensorFlow and Keras, so the notebooks only install pinned versions of the tracking tools (MLflow 2.22.5, Keras Tuner 1.4.7, DVC 3.67.1) and my own package `visionguard` in editable mode with `pip install -e .`. For serving I used Docker images built from the pinned `requirements-serve.txt` and `requirements-tf.txt` (Chapter 6). On my own machine, `py -m venv .venv` gives a private interpreter with an empty `site-packages`, and `pip install -r requirements.txt` installs the pinned versions (Python 3.11 or 3.12).

`scripts/00_env_check.py` prints the date, user and host, the Python version, the version of every library, whether a virtualenv is active and which devices TensorFlow can see (`tf.config.list_physical_devices("GPU")`). On Colab it shows TensorFlow 2.21.0 built with CUDA and one visible GPU. It also prints "in virtualenv: False", which is expected: on Colab the isolation comes from the disposable VM itself.

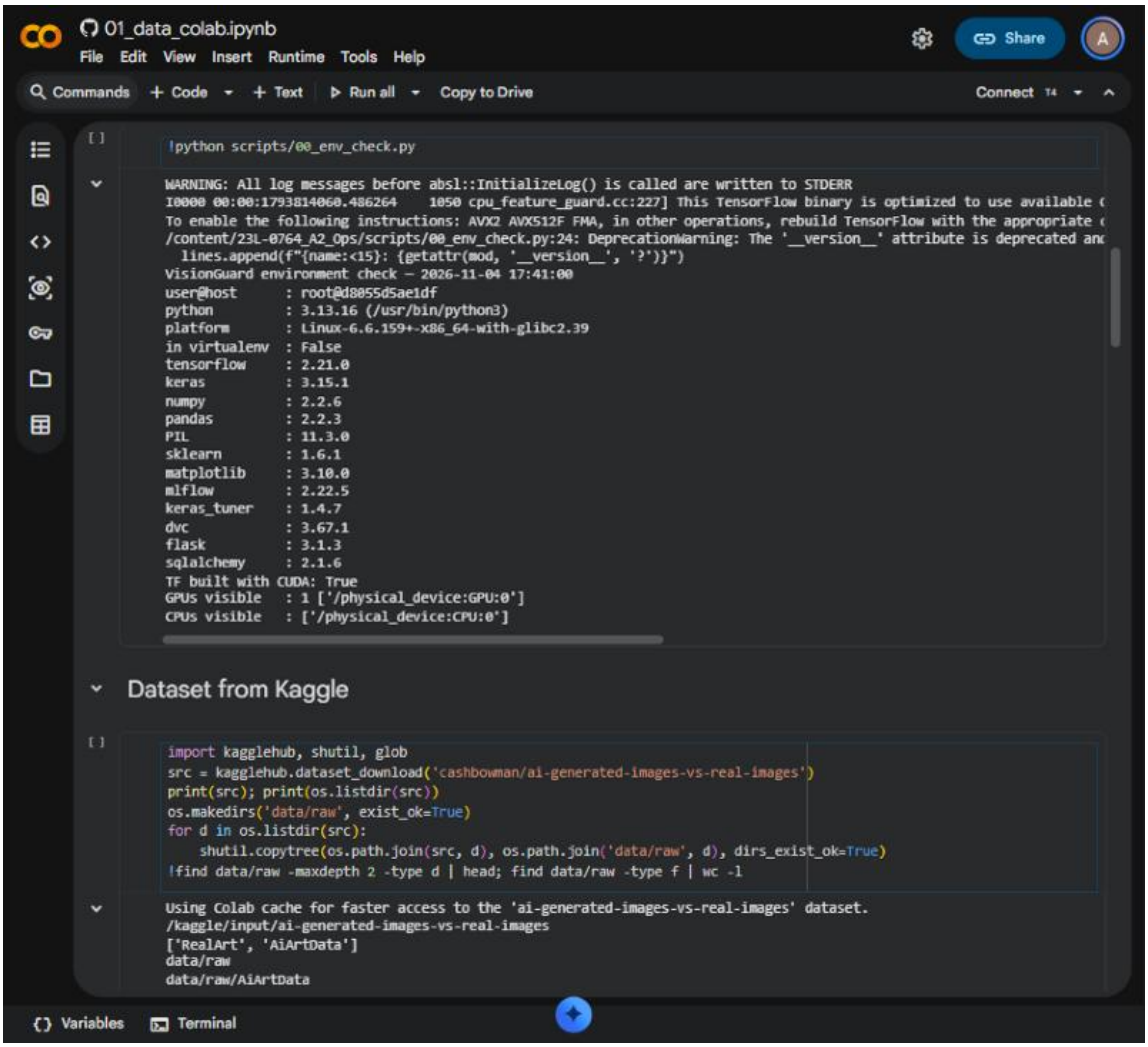


Figure 1.1: Library versions and GPU visibility (`scripts/00_env_check.py` on Colab).

I pinned MLflow to the 2.x line on purpose. MLflow 3 logs models through a new "logged models" REST API, and the tracking server hosted by DagsHub implements the MLflow 2 API, so with a 3.x client `log_model` and the Model Registry would not work against DagsHub.

1.2 Dataset audit

`scripts/01_audit.py` goes through every file under `data/raw` and records the class (taken from the top-level folder name), file size, width, height, colour mode, format, a SHA-256 hash of the bytes, whether the file decodes completely, and the error message if it does not. The full table goes to `reports/audit/audit.csv` and the summaries are written directly into this report.

The checks work like this:

- Readability: `PIL.Image.verify()` catches broken headers, and `load()` forces a full decode of the pixels, which catches truncated files that open fine and only fail later. The audit uses the same decode function as training and serving (`src/visionguard/imaging.py`), so an image that passes the audit is one the model can actually read.
- Duplicates: two files with the same SHA-256 hash have identical bytes, and I keep only the first copy. This matters because a copy in the training set and another in the test set would leak the answer and make the test accuracy look better than it is.
- Label conflicts: if the same hash shows up under both classes, it is label noise and I drop every copy.
- Images smaller than 32 pixels on either side are marked unusable.

Class	Files	Readable	Corrupt	Duplicates	Too small
AI-generated	538	538	0	1	0
Real	435	435	0	0	0
Total	973	973	0	1	0

Table 1.1: Files per class: total, readable, corrupt (cannot be decoded), exact duplicates and too-small images.

Class	Quantity	Min	Median	Max
AI-generated	Width (px)	225	1028	6000
AI-generated	Height (px)	135	800	4001
AI-generated	File size (KB)	11	195	5494
Real	Width (px)	183	940	8495
Real	Height (px)	158	720	7091
Real	File size (KB)	11	142	9956

Table 1.2: Minimum, median and maximum of width, height and file size per class (readable files only).

Class	Format	Colour mode	Files
AI-generated	JPEG	L	1
AI-generated	JPEG	RGB	427
AI-generated	PNG	P	14
AI-generated	PNG	RGB	61
AI-generated	PNG	RGBA	35
Real	JPEG	L	6
Real	JPEG	RGB	396
Real	PNG	P	9
Real	PNG	RGB	10
Real	PNG	RGBA	14

Table 1.3: Image formats and colour modes per class.

The dataset has 973 files, 538 AI-generated and 435 real. 0 files could not be decoded and 1 are exact duplicates (1 duplicate groups, 0 hashes found in both classes). After cleaning, 972 images are left. I split them 70/15/15 with a fixed seed and stratification, so every split has the same AI/real ratio:

Split	Images	AI-generated	Real	% AI
train	680	376	304	55.3
val	146	81	65	55.5
test	146	80	66	54.8

Table 1.4: Stratified train / validation / test split after removing corrupt files, duplicates and label conflicts.

File	Error
none	-

Table 1.5: Files that could not be decoded (first 15) and the decoder's error.

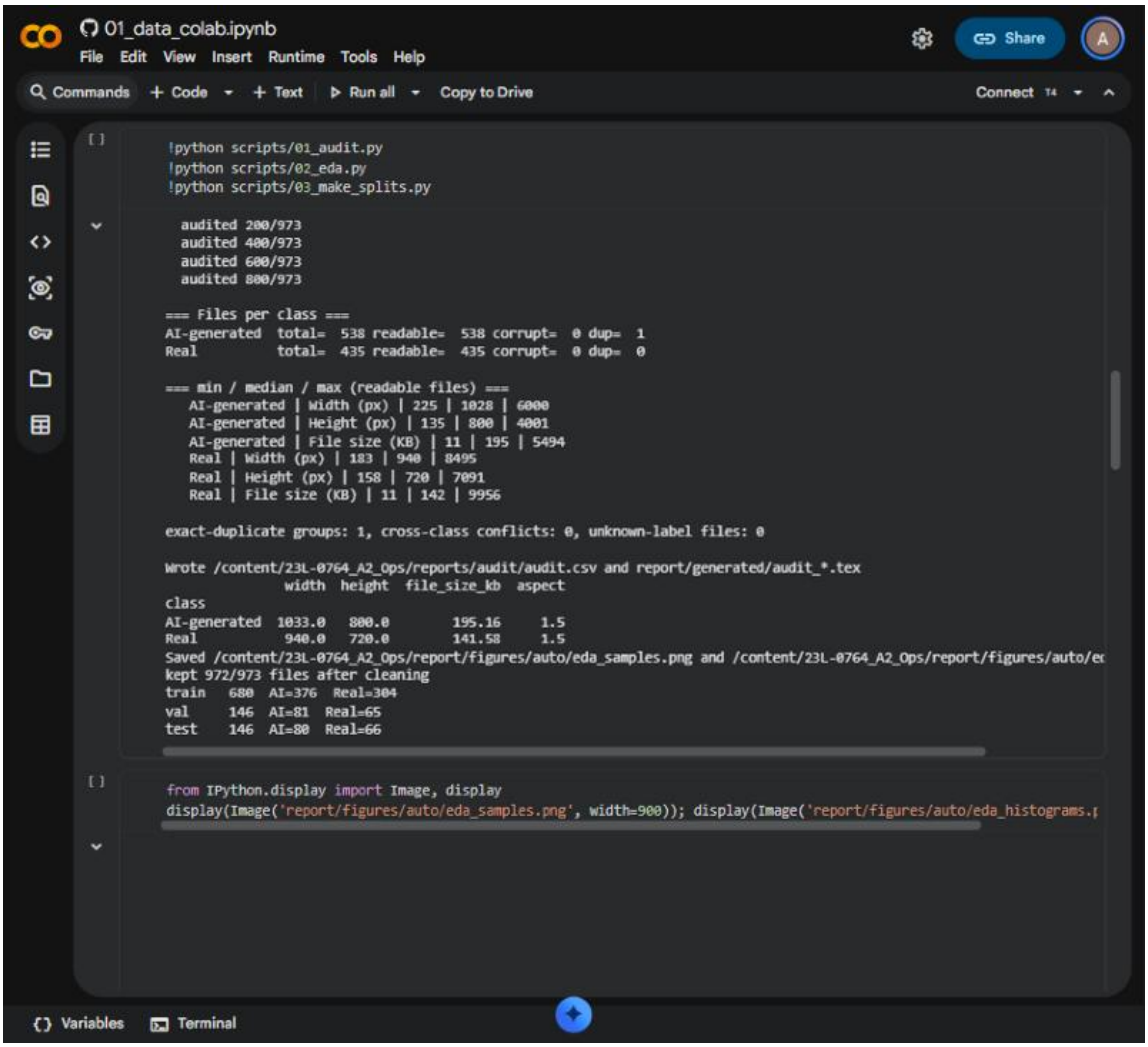


Figure 1.2: Terminal output of the audit script.

One problem shaped the whole data pipeline. My first choice of decoder was `tf.io.decode_image`, but it cannot read WEBP files, CMYK JPEGs or palette PNGs with transparency, and scraped data has all of these. It would have marked good images as corrupt and I would have thrown them away. So the audit, training and serving all call one Pillow function that converts every image to RGB and composites transparent pixels onto white (otherwise they come out black).

1.3 Visual EDA

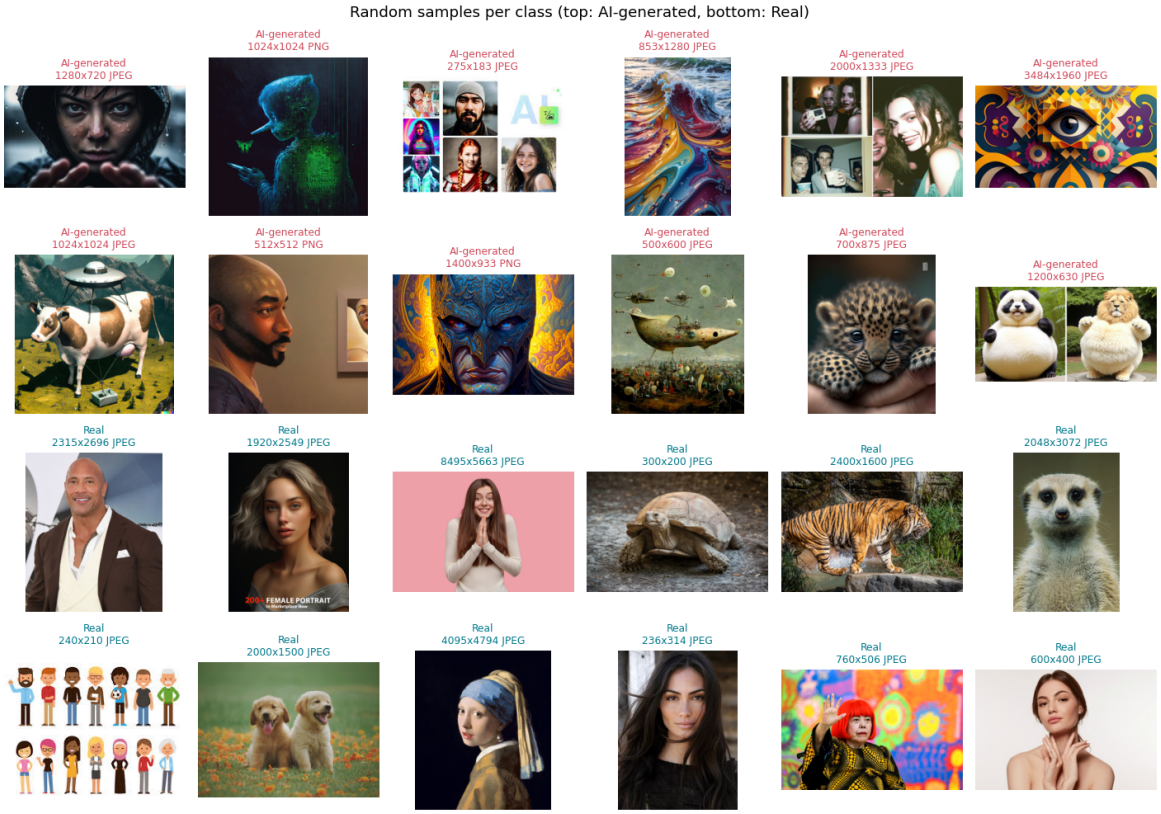


Figure 1.3: Random samples per class (top two rows AI-generated, bottom two rows real), with original size and format.

Most of the AI-generated images are fantasy or "digital art": saturated colours, glossy lighting, psychedelic patterns, impossible scenes (a cow under a UFO) and a few collages made of many small pictures. The real class turned out to be broader than I expected. It is mostly professional stock photos and studio portraits, but it also has classical paintings (a Vermeer portrait) and flat cartoon clip-art. In this dataset "real" really means "made without a generator". Both classes contain people, animals and landscapes, so the model cannot get away with recognising the topic and has to pick up on style and texture.

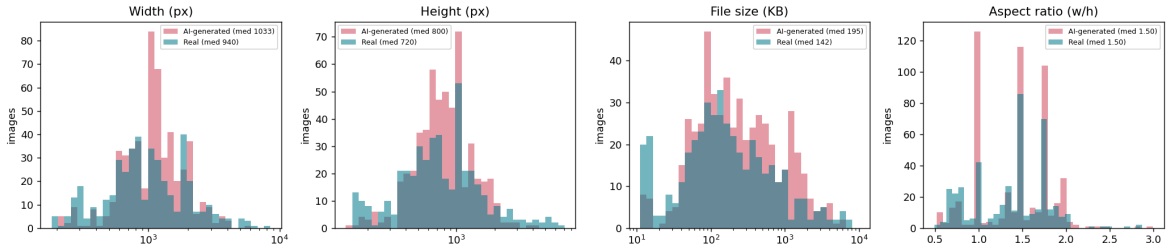


Figure 1.4: Width, height, file size (log scale) and aspect ratio per class.

The median AI image is 1033×800 pixels and 195 KB; the median real image is 940×720 pixels and 141 KB. 23% of the AI images are exactly square, compared with 9% of the real ones, because generators output fixed square resolutions. Differences like these are a shortcut risk: a network could learn "square and small file means AI" and ignore the actual generation artefacts. Resizing everything to the same square input removes the size signal from the pixels, but compression strength still differs between the classes. I come back to this in the error analysis (Section 2.6).

Looking at the histograms, the size gap goes the way the medians suggest (AI images are larger), and the peak of square images is visible but small (23 % against 9 %). Size on its own could not separate the two classes.

1.4 Data version control with DVC

Toy exercise

To learn DVC before using it on the real data, I made a 3-row `scores.csv` in an empty folder, versioned it, changed it, went back to the old version, and pushed it to and pulled it from remote storage on DagsHub (repository `23L-0764_A2_Ops-dvc-toy`). The commands, in the order I ran them (`labs/dvc_toy.sh`):

1. `git init` and `dvc init`. Git tracks code and small text files, and DVC adds a `.dvc/` folder (config and local cache) on top.
2. `dvc add scores.csv` moves the content into the cache at `.dvc/cache/<md5>`, writes the pointer file `scores.csv.dvc` and adds `scores.csv` to `.gitignore`, so Git never stores the data. I committed the pointer.
3. I appended two rows, ran `dvc add` again and committed. The pointer now has a different md5.
4. `git checkout HEAD~1 - scores.csv.dvc` brings back the old pointer, and `dvc checkout` replaces the file in the workspace with the cached content for that md5, so the 3-row version is back. Git decides which version I get and DVC provides its bytes.
5. `dvc remote add -d origin s3://dvc` plus `endpointurl` points DVC at DagsHub's S3-compatible storage. I set the token with `-local`, which writes it to the git-ignored `.dvc/config.local` and keeps it out of the committed `.dvc/config`.
6. `dvc push -all-commits` uploads the data of every committed version. To test it I deleted `scores.csv` and the local cache, and `dvc pull` downloaded the file again.

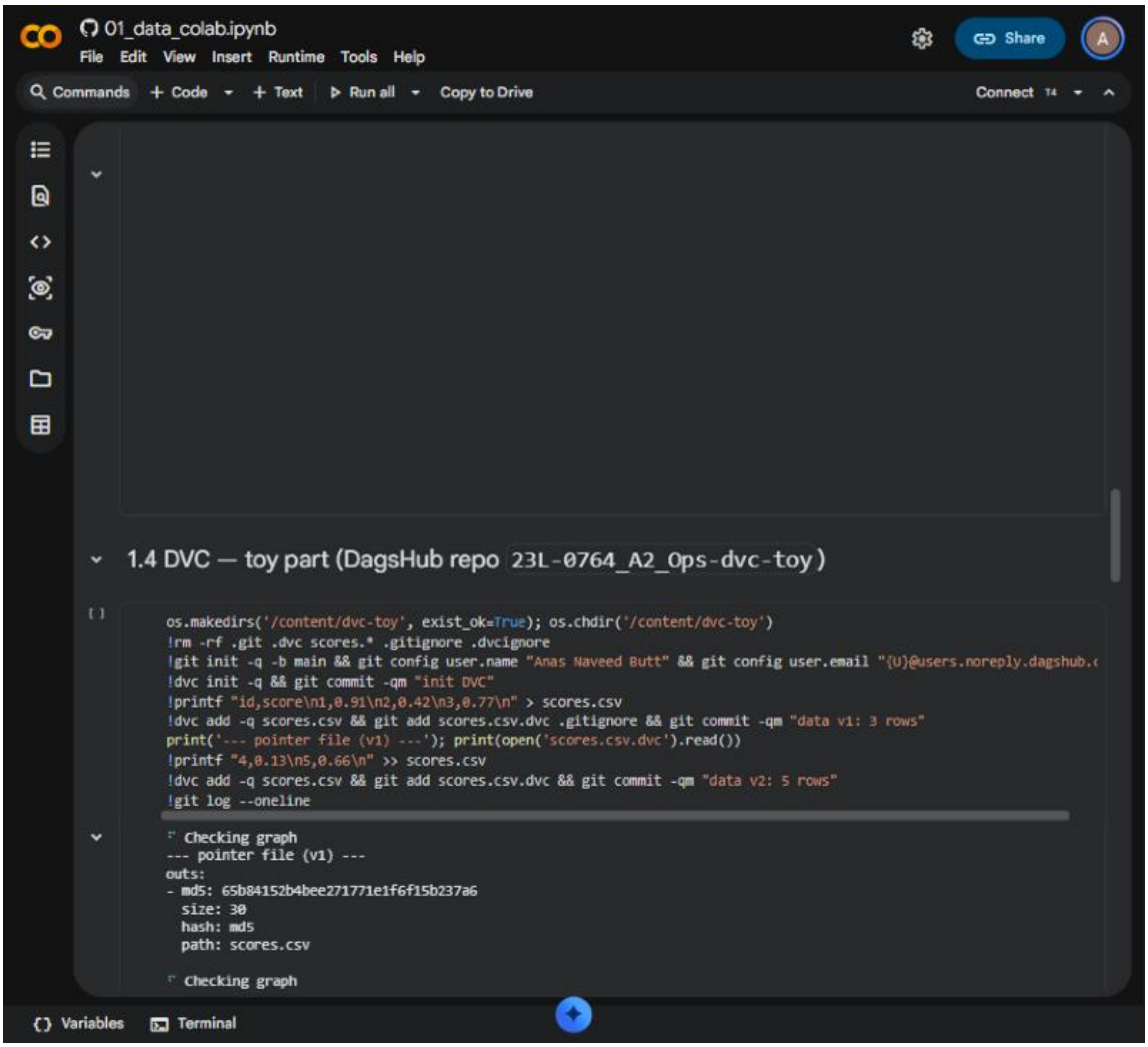


Figure 1.5: The pointer file and the two commits (v1, v2).

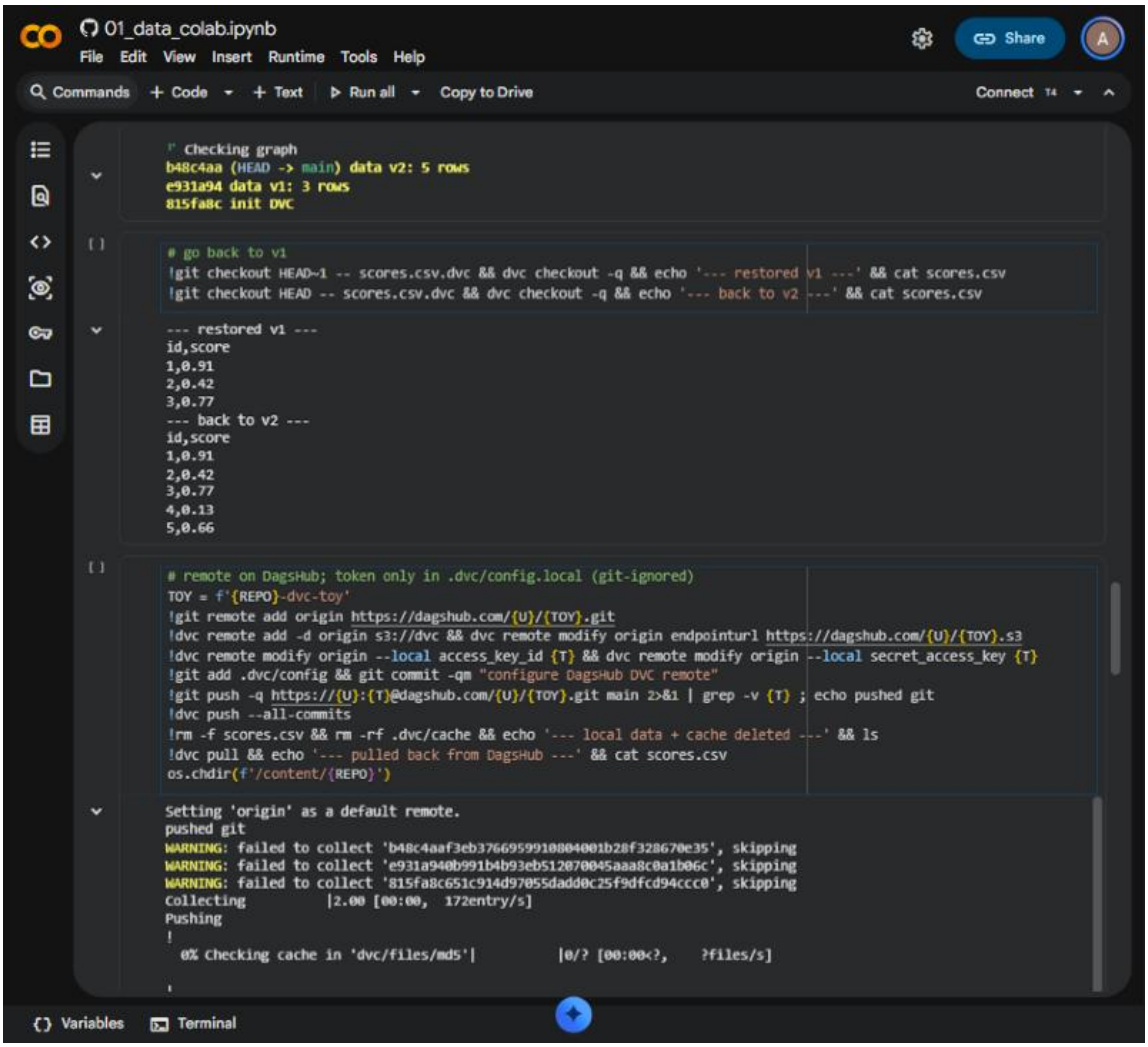


Figure 1.6: Returning to the older version with `git checkout` + `dvc checkout`.

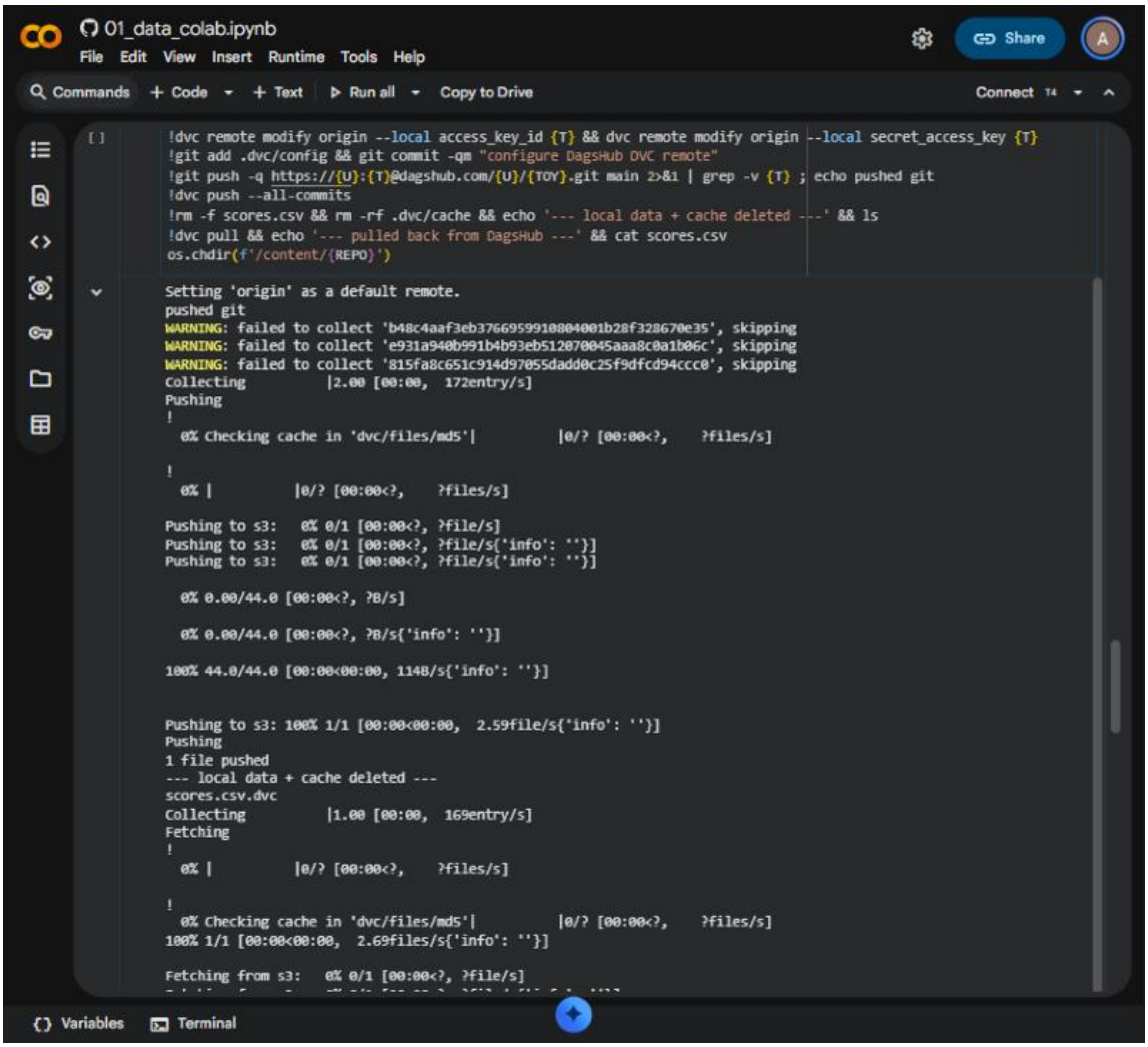


Figure 1.7: Push to DagsHub, delete local data and cache, pull it back.

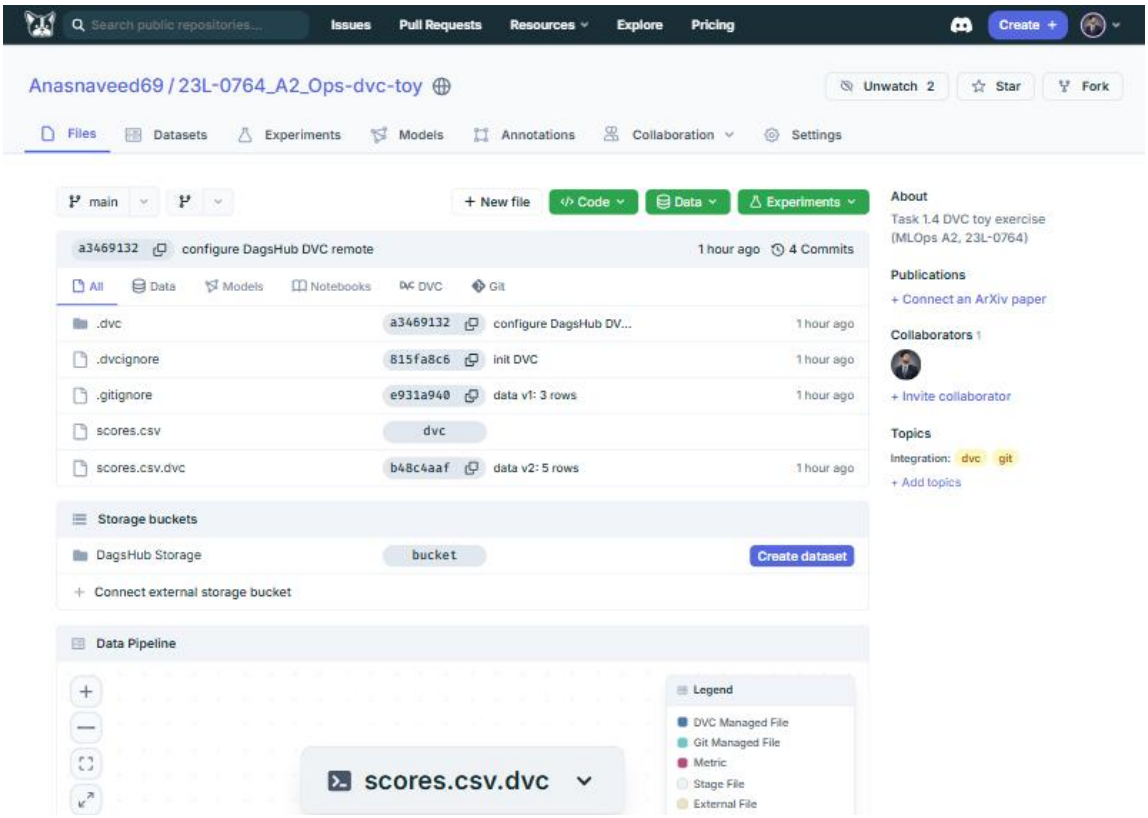


Figure 1.8: The toy repository on DagsHub with the DVC-tracked file.

The pointer file, and why the data stays out of Git

The `.dvc` file is a few lines of YAML: the `md5` of the content (for a folder, the md5 of a manifest that lists every file and its hash), the total `size`, the number of files and the `path`. It only identifies the data. Git is designed for small text files that change line by line, and it keeps every version of every file in `.git` forever. If I committed hundreds of megabytes of images, every clone would have to download all of them, diffs of binary files would be useless, and GitHub refuses files over 100 MB anyway. With DVC the Git history stays small and the data version is just the md5 in the pointer. I also log that md5 to MLflow as the tag `dvc_data_version`, so every experiment records which data it was trained on.

Applying it to VisionGuard

`dvc add data/raw` tracks the whole dataset folder as one unit (`data/raw.dvc`), and the DagsHub storage of the `23L-0764_A2_Ops` repository is the default remote. The split lists (`data/splits/*.csv`, a few KB) are committed to Git directly, since they are small text files and should change together with the code that creates them.

```
Task 1.4: notebook 01 output (dvc add / push) + pointer commit
$ dvc remote add -d -f origin s3://dvc
$ dvc remote modify origin endpointurl https://dagshub.com/Anasnaveed69/23L-0764_A2_Ops.s3
$ dvc remote modify origin --local access_key_id ****   # token only in .dvc/config.local
$ dvc add data/raw
$ cat data/raw.dvc
outs:
- md5: b2cec67839f742df4653f66d4bcbbafb.dir
  size: 503783378
  nfiles: 975
  hash: md5
  path: raw
$ dvc push
Pushing
975 files pushed
$ git log --oneline -1 -- data/raw.dvc
4c3de23 Track dataset with DVC (DagsHub storage); audit, EDA and stratified spli
$ cat data/.gitignore
/raw
```

Figure 1.9: `dvc add data/raw`, the pointer file and `dvc push` of the dataset (notebook 01 output).

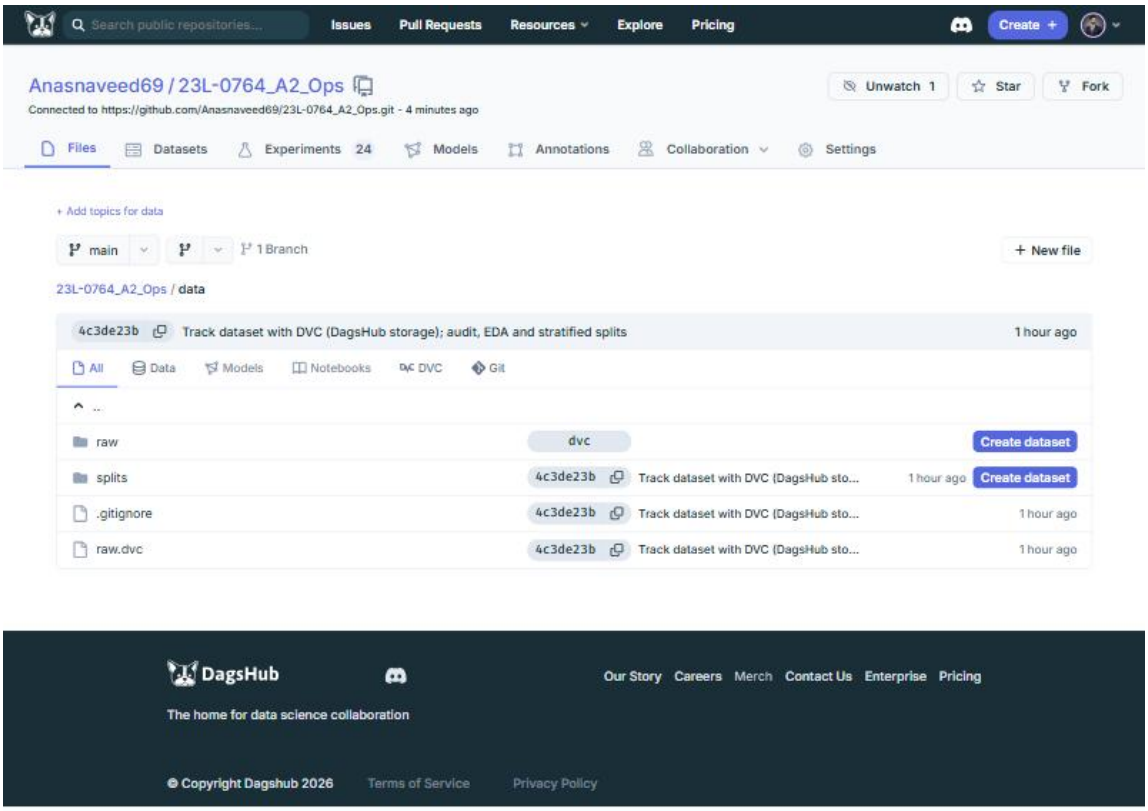


Figure 1.10: DagsHub repository page showing `data/raw` tracked by DVC.

Two things almost caught me out here. A plain `dvc push` only uploads the data of the current commit, so once the cache is deleted an older version cannot be restored from the remote. `dvc push -all-commits` uploads every version that Git history refers to. The other one was the credentials: running `dvc remote modify` without `-local` writes the token into `.dvc/config`, which is committed, so the token would end up public.

2. Model Development with TensorFlow/Keras

Where training ran

All the model code is in `src/visionguard/` and `scripts/train.py` runs it. Every setting is read from `params.yaml` and can be overridden for a single run, for example `-set train.lr_head=0.0001`. Training ran on a Colab T4 GPU in two notebooks: `notebooks/01_data_colab.ipynb` downloads the data from Kaggle and does the audit, EDA, splits and DVC push, and `notebooks/02_train_colab.ipynb` does training, tuning and tracking. The second notebook clones the repository from GitHub and gets the exact data version with `dvc pull`, so each run can be traced back through the `git_commit` and `dvc_data_version` tags it logs. Colab already comes with matching TensorFlow and Keras versions, so I only pinned the tracking libraries there.

2.1 Input pipeline

The pipeline is built in `src/visionguard/data.py` (`make_dataset`) and has these steps:

1. List the images with `from_tensor_slices((paths, labels))` from the split CSV. I read the split lists and not the folders, so the same image always ends up in the same split.
2. Decode and resize in parallel: `tf.numpy_function` calls the shared Pillow function `decode_and_resize` with `num_parallel_calls=AUTOTUNE`. Each image comes out as a $224 \times 224 \times 3$ tensor of raw 0–255 values.
3. Cache the decoded images after the first epoch as `uint8`, which is four times smaller than `float32`. JPEG decoding then happens only once.
4. Shuffle the training set (only the training set) with a buffer as large as the set and a fixed seed. It is reshuffled every epoch.
5. Batch (32 images) and, for training only, augment whole batches: horizontal flip, rotation $\pm 5\%$, zoom $\pm 10\%$, translation $\pm 5\%$ and contrast $\pm 10\%$.
6. Prefetch, so the CPU prepares the next batch while the GPU is busy with the current one.

Validation and test data go through the same steps without shuffling or augmentation, so they are evaluated exactly as they are.

I used 224 px because MobileNetV2 was pre-trained at that resolution, so its filters see objects at the scale they learned. It also keeps a batch cheap. One experiment uses 160 px to see how much the resolution matters (Section 3.5).

By default the resize squashes every image into a square, so a 1000×500 photo is squeezed to half its width and loses its aspect ratio. The other option, `resize_mode: pad`, keeps the ratio and fills the rest with black bars, but then the size of the bars tells the model the original shape, which is one of the shortcuts I found in the EDA. I kept squashing for both classes so that the shape gives no signal either way.

The augmentations change the geometry and the overall contrast and leave the fine texture alone. I left out blur, noise and JPEG re-compression on purpose. Those remove the high-frequency artefacts that give generated images away, and they could make an AI image look statistically like a camera photo while it still carries the "AI" label.

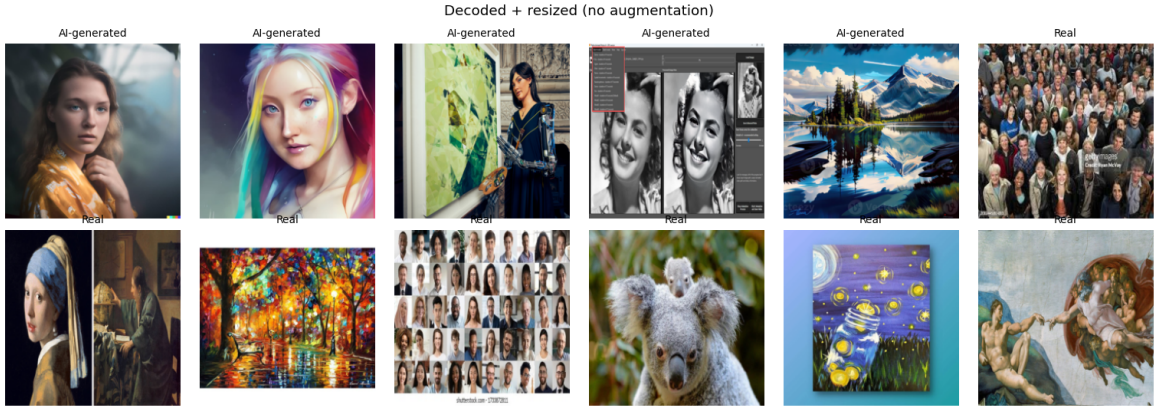


Figure 2.1: A training batch after decode + resize, before augmentation.

Figure 2.2: The same batch after the training-time augmentation.

2.2 Baseline CNN from scratch

The baseline is `build_baseline` in `models.py`. Going through it layer by layer:

- `Rescaling(1/255)` maps the pixels to $[0, 1]$ inside the model. The serving code only has to send raw pixels and cannot forget the scaling.
- Four blocks of Conv2D(3×3), BatchNorm, ReLU and MaxPool(2), with 32, 64, 128 and 256 filters. The convolutions learn local patterns, first edges, then textures, then parts. Doubling the filters while halving the resolution keeps the compute per block roughly even. BatchNorm keeps the activations normalised so training stays stable (the convolutions use `use_bias=False` because BatchNorm's shift does the job of the bias). ReLU adds the non-linearity, and pooling gives some tolerance to shifts and grows the receptive field.
- GlobalAveragePooling averages each feature map to one number, turning $14 \times 14 \times 256$ into 256 values. That needs far fewer parameters than `Flatten` followed by a Dense layer, which matters with only a few hundred training images.
- A small classifier head: Dropout(0.4), Dense(64, ReLU), Dropout(0.2). Dropout randomly zeroes units during training, which reduces over-fitting.
- Dense(1, sigmoid) gives a single output, $p = P(\text{AI} \mid \text{image})$.

For the loss I used binary cross-entropy. It is the negative log-likelihood of a Bernoulli output, so it fits a single sigmoid unit, and it pushes the model towards calibrated probabilities, which I need because the

decision threshold is picked afterwards. The optimiser is Adam, which adapts the step size per parameter; the baseline uses a learning rate of $3 \cdot 10^{-4}$ (the reason is in Chapter 3). I track accuracy because it is easy to read, ROC-AUC because it measures ranking quality without a threshold, and precision and recall of the AI class because they are the two kinds of error discussed in Section 2.5. Class weights set to the inverse class frequency stop the model from ignoring the smaller class.

Layer	Type	Output shape	Params
image	InputLayer	(None, 224, 224, 3)	0
scale_0_1	Rescaling	(None, 224, 224, 3)	0
conv1	Conv2D	(None, 224, 224, 32)	864
bn1	BatchNormalization	(None, 224, 224, 32)	128
relu1	Activation	(None, 224, 224, 32)	0
pool1	MaxPooling2D	(None, 112, 112, 32)	0
conv2	Conv2D	(None, 112, 112, 64)	18,432
bn2	BatchNormalization	(None, 112, 112, 64)	256
relu2	Activation	(None, 112, 112, 64)	0
pool2	MaxPooling2D	(None, 56, 56, 64)	0
conv3	Conv2D	(None, 56, 56, 128)	73,728
bn3	BatchNormalization	(None, 56, 56, 128)	512
relu3	Activation	(None, 56, 56, 128)	0
pool3	MaxPooling2D	(None, 28, 28, 128)	0
conv4	Conv2D	(None, 28, 28, 256)	294,912
bn4	BatchNormalization	(None, 28, 28, 256)	1,024
relu4	Activation	(None, 28, 28, 256)	0
pool4	MaxPooling2D	(None, 14, 14, 256)	0
gap	GlobalAveragePooling2D	(None, 256)	0
dropout1	Dropout	(None, 256)	0
dense	Dense	(None, 64)	16,448
dropout2	Dropout	(None, 64)	0
p_ai	Dense	(None, 1)	65

Table 2.1: Baseline CNN: output shape and parameter count per layer (406,369 parameters).

baseline CNN — model.summary()

```
$ python -c "from visionguard.models import build_baseline; build_baseline(224).summary()"
Model: "baseline_cnn"
```

Layer (type)	Output Shape	Param #
image (InputLayer)	(None, 224, 224, 3)	0
scale_0_1 (Rescaling)	(None, 224, 224, 3)	0
conv1 (Conv2D)	(None, 224, 224, 32)	864
bn1 (BatchNormalization)	(None, 224, 224, 32)	128
relu1 (Activation)	(None, 224, 224, 32)	0
pool1 (MaxPooling2D)	(None, 112, 112, 32)	0
conv2 (Conv2D)	(None, 112, 112, 64)	18,432
bn2 (BatchNormalization)	(None, 112, 112, 64)	256
relu2 (Activation)	(None, 112, 112, 64)	0
pool2 (MaxPooling2D)	(None, 56, 56, 64)	0
conv3 (Conv2D)	(None, 56, 56, 128)	73,728
bn3 (BatchNormalization)	(None, 56, 56, 128)	512
relu3 (Activation)	(None, 56, 56, 128)	0
pool3 (MaxPooling2D)	(None, 28, 28, 128)	0
conv4 (Conv2D)	(None, 28, 28, 256)	294,912
bn4 (BatchNormalization)	(None, 28, 28, 256)	1,024
relu4 (Activation)	(None, 28, 28, 256)	0
pool4 (MaxPooling2D)	(None, 14, 14, 256)	0
gap (GlobalAveragePooling2D)	(None, 256)	0
dropout1 (Dropout)	(None, 256)	0
dense (Dense)	(None, 64)	16,448
dropout2 (Dropout)	(None, 64)	0
p_ai (Dense)	(None, 1)	65

```
Total params: 406,369 (1.55 MB)
Trainable params: 405,409 (1.55 MB)
Non-trainable params: 960 (3.75 KB)
```

Figure 2.3: `model.summary()` of the baseline CNN.

2.3 Transfer learning (MobileNetV2)

I picked MobileNetV2 because its depthwise-separable convolutions give good ImageNet features with only about 2.3 M backbone parameters (14 MB on disk). It fine-tunes quickly on a small dataset and is fast on the CPU-only server. I load it with `keras.applications.MobileNetV2(include_top=False, weights="imagenet")`, which drops the 1000-class classifier and keeps the convolutional feature extractor, ending in a $7 \times 7 \times 1280$ map.

MobileNetV2 was trained on pixels scaled to $[-1, 1]$ (`mobilenet_v2.preprocess_input` computes $x/127.5 - 1$). If I fed it $[0, 1]$ or $[0, 255]$ instead, every activation would be shifted away from what the pre-trained filters expect and the weights would lose most of their value. I put the scaling inside the model as `Rescaling(1/127.5, offset=-1)`, so the saved model applies it the same way in training and in serving. EfficientNetB0, which I tried in one experiment, has its own normalisation and expects raw 0–255 input, so it gets an identity layer here.

The head is GlobalAveragePooling, Dropout(0.3) and Dense(1, sigmoid).

Training has two stages. In stage 1 the backbone is frozen (`base.trainable = False`) and only the head's 1 281 weights learn, at a learning rate of 10^{-3}. A freshly initialised head produces large gradients, and if the backbone were trainable at this point those gradients would damage the pre-trained filters.

In stage 2 I unfreeze only MobileNetV2's last inverted-residual block (`block_16`) and the final `Conv_1` layer: `block_16_expand`, `block_16_expand_relu`, `block_16_depthwise`, `block_16_depthwise_relu`, `block_16_project`, `Conv_1`, `out_relu` . The model has to be recompiled for this to take effect, and I recompile it with a learning rate 100 times smaller (10^{-5}), so the high-level features can adapt to generator artefacts without forgetting ImageNet. The BatchNorm layers stay frozen

and the backbone is called with `training=False`; otherwise BatchNorm would overwrite the ImageNet statistics with statistics from batches of 32 images.

Stage	Trainable weight tensors	Backbone layers unfrozen
1 (head only)	2	0
2 (fine-tune)	6	7

Table 2.2: Trainable weights in each stage.

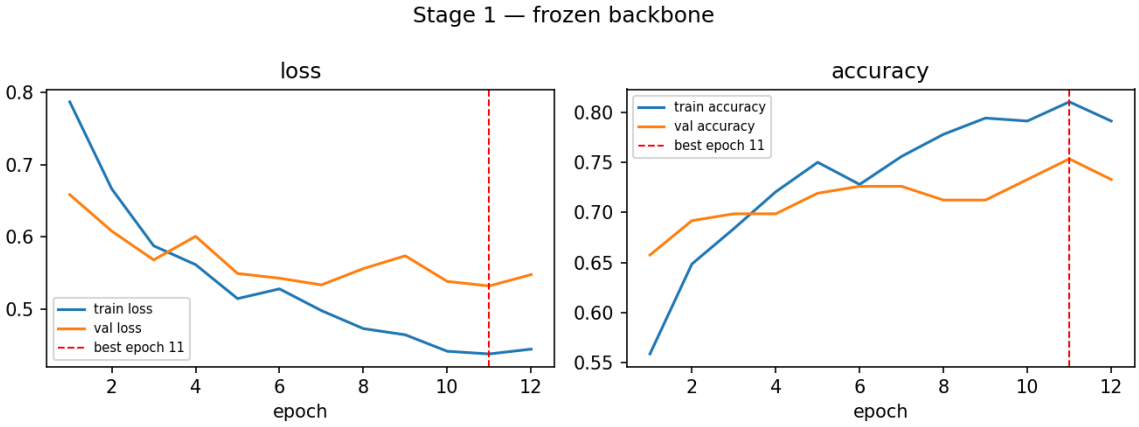


Figure 2.4: Stage 1 (frozen backbone): loss and accuracy.

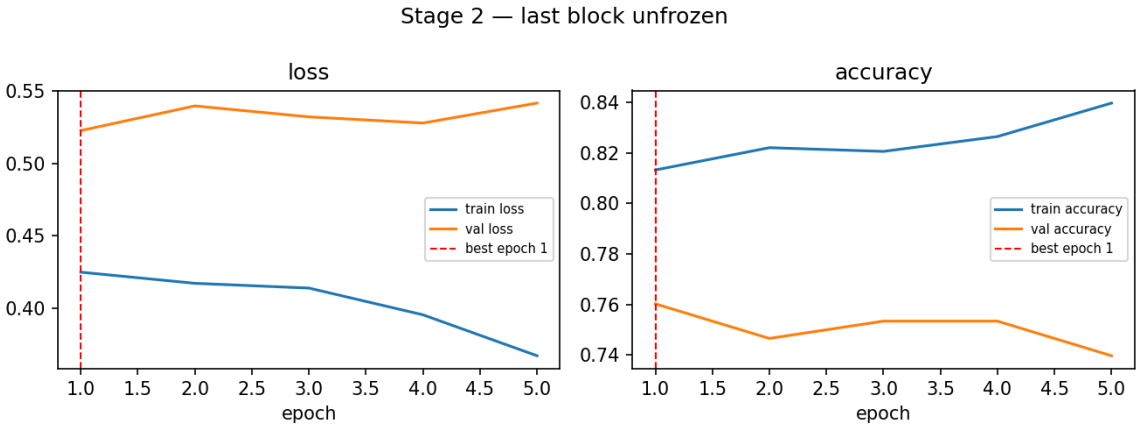


Figure 2.5: Stage 2 (last block unfrozen, learning rate 10^{-5}): loss and accuracy.

Layer	Type	Output shape	Params
image	InputLayer	(None, 224, 224, 3)	0
mobilenetv2_preprocess	Rescaling	(None, 224, 224, 3)	0
backbone	Functional	(None, 7, 7, 1280)	2,257,984
gap	GlobalAveragePooling2D	(None, 1280)	0
head_dropout	Dropout	(None, 1280)	0
p_ai	Dense	(None, 1)	1,281

Table 2.3: Transfer model, top-level layers (2,259,265 parameters in total, 880,321 trainable in stage 2).

One bug cost me some time here. In Keras 3, renaming the backbone with `base._name = "backbone"` does nothing and raises no error, so `model.get_layer("backbone")` failed when I tried to unfreeze it. Passing `name="backbone"` to the constructor fixed it.

2.4 Training discipline

- Seeds: `keras.utils.set_random_seed(42)` seeds Python, NumPy and TensorFlow in one call, `PYTHONHASHSEED` is fixed, and the split, the shuffle and every augmentation layer get their own explicit seed. GPU kernels are not bit-exact, so a rerun on a GPU can still differ in the third decimal. The seed is logged as a tag on every run.
- EarlyStopping on `val_loss` with patience 4 and `restore_best_weights=True`. The baseline uses patience 8 and cannot stop before epoch 6 (explained below).
- ModelCheckpoint on `val_loss` with `save_best_only`. Both transfer stages share one checkpoint object, so it keeps the best epoch across the two stages. After training I reload that file, so the model I evaluate is the best one and not simply the last one.
- ReduceLROnPlateau multiplies the learning rate by 0.3 when `val_loss` has not improved for 2 epochs.

The lowest validation loss was at epoch 38 for the baseline CNN and at epoch 13 for the transfer model (counting across both stages).

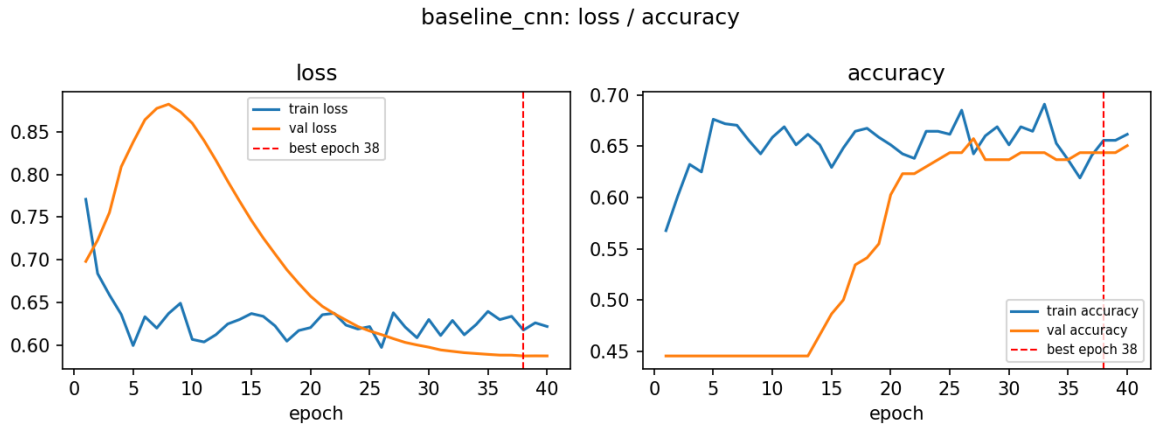


Figure 2.6: Baseline CNN: loss and accuracy, best epoch marked in red.

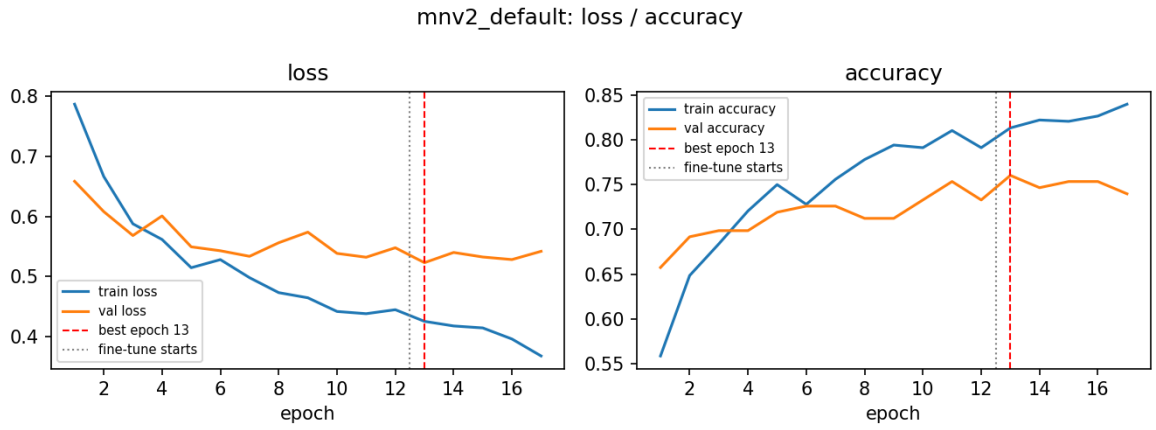


Figure 2.7: Transfer model, both stages: best epoch in red, start of fine-tuning dotted.

For the baseline, the training loss drops quickly to about 0.62 and then hardly moves, so the small CNN under-fits. For the first 13 epochs its validation accuracy is stuck at 0.45, which is the share of real images: it predicts "real" for everything. The validation loss even goes up until epoch 8, because the BatchNorm running statistics used at validation time are still far from the statistics of the training batches. Once they settle, the validation loss falls steadily until its minimum at epoch 38. My first baseline run stopped at epoch 1 for exactly this reason, which is why the baseline now has a lower learning rate and a warm-up before early stopping can trigger.

The transfer model gets to a validation loss of about 0.53 within 5 epochs of the frozen stage and then bounces around, while the training loss keeps going down to 0.37. That widening gap is over-fitting on roughly 680 training images. Fine-tuning (the dotted line) gave one more small improvement, at epoch 13, which is the first fine-tuning epoch, and nothing after that, so early stopping ended the run 4 epochs later and restored the best weights.

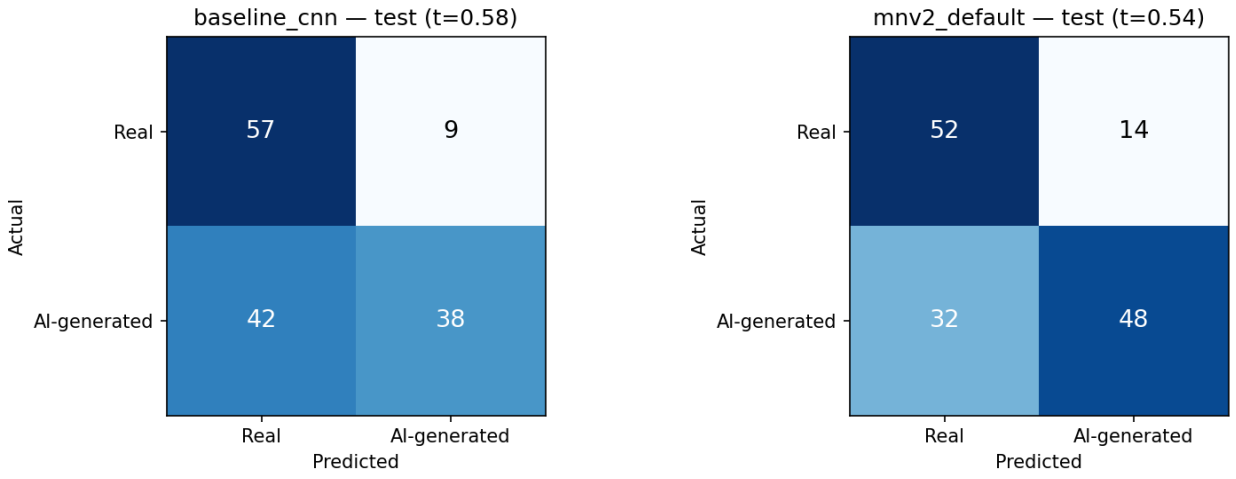
2.5 Evaluation

For this product, a false positive is the worse error: a real photo labelled AI-generated. VisionGuard is meant to help decide whether someone's photo is genuine, and a false accusation hurts a real person (a journalist, a seller, a student) and makes people stop trusting the tool. A false negative, an AI image that passes as real, is bad too, but that is what already happens when no detector is used. So I care most about precision on the AI class, in other words a low false-positive rate.

The test set is used only once, at the very end, so the threshold is chosen on the validation predictions. I scan $t \in [0.05, 0.95]$ and take the value with the highest F_β for $\beta = 0.5$, which weights precision twice as much as recall and so matches the reasoning above. This gave a threshold of 0.58 for the baseline and 0.54 for the transfer model. The threshold is saved next to the model in `model_meta.json` and the server reads it from there.

Model	Thr.	Accuracy	Precision	Recall	F1	ROC-AUC	FPR
Baseline CNN	0.58	0.651	0.809	0.475	0.598	0.743	0.136
MobileNetV2 transfer	0.54	0.685	0.774	0.600	0.676	0.754	0.212

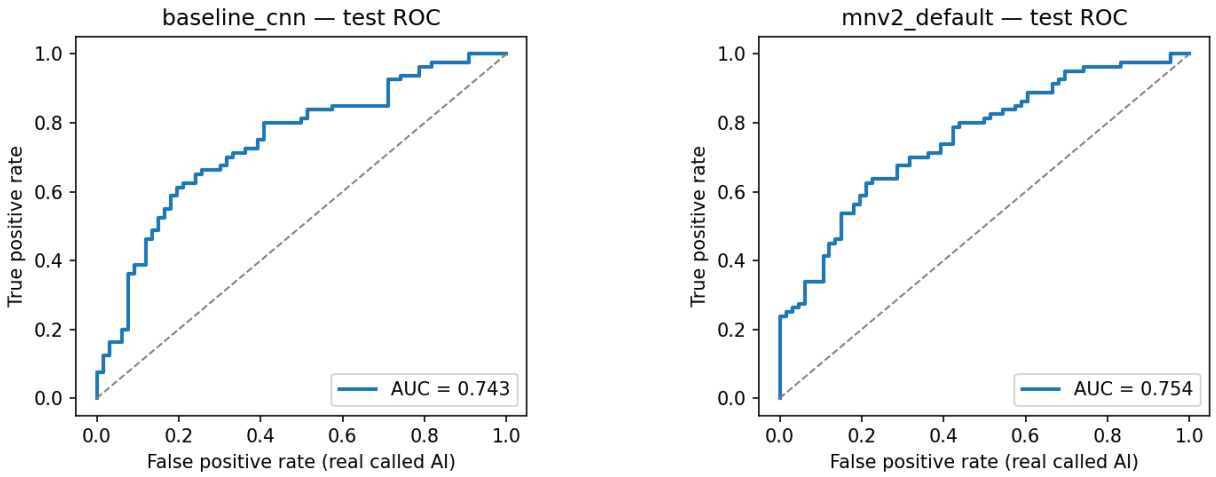
Table 2.4: Test-set metrics at the validation-chosen threshold (FPR = real photos called AI).



(a) Baseline CNN

(b) MobileNetV2 transfer

Figure 2.8: Test confusion matrices.



(a) Baseline CNN

(b) MobileNetV2 transfer

Figure 2.9: Test ROC curves.

On the test set the transfer model gets accuracy 0.685, F1 0.676 and ROC-AUC 0.754, with a false-positive rate of 0.212. The baseline gets 0.651, 0.598 and 0.743, with an FPR of 0.136.

With the reference settings the transfer model is only a little better than the CNN trained from scratch (AUC 0.754 against 0.743, F1 0.676 against 0.598). Most of the difference is recall at the chosen threshold (0.60 against 0.48); the baseline is more conservative and has the lower FPR (0.14 against 0.21). The pre-trained features start to pay off once the transfer settings are tuned. The final champion, `mnv2_tuned`, has the same architecture with the tuner's learning rate, dropout and 60 unfrozen layers, and reaches a test AUC of 0.844, accuracy 0.733 and F1 0.731 (Section 3.5).

2.6 Error analysis

`scripts/error_analysis.py` takes the champion's test predictions and keeps the ten mistakes the model was most confident about, since those say the most about what it learned. For each one it measures size, sharpness (variance of an edge filter), colour saturation, whether it is grey-scale, aspect ratio and format, so every error gets a hypothesis I can check.

Figure 2.10: The ten most confident test errors (true label, prediction, $p(\text{AI})$, size).

\{}#	True	p(AI)	Size	Sharpness	Hypothesis (from measured cues)
1	AI	0.06	1200x800	2699	very sharp, heavy detail or noise looks 'camera-like'
2	Real	0.94	681x960	1517	very sharp, heavy detail or noise looks 'camera-like'; highly saturated, painted / illustration style
3	Real	0.94	1920x1080	2115	very sharp, heavy detail or noise looks 'camera-like'; extreme aspect ratio squashed by resizing
4	AI	0.07	1024x512	609	extreme aspect ratio squashed by resizing
5	AI	0.07	1000x1012	1447	very sharp, heavy detail or noise looks 'camera-like'
6	Real	0.91	2880x1620	2290	very sharp, heavy detail or noise looks 'camera-like'; extreme aspect ratio squashed by resizing
7	Real	0.88	1800x1544	770	highly saturated, painted / illustration style
8	AI	0.16	1600x1067	1543	very sharp, heavy detail or noise looks 'camera-like'
9	AI	0.17	1200x600	2290	very sharp, heavy detail or noise looks 'camera-like'; extreme aspect ratio squashed by resizing
10	AI	0.20	512x768	2886	very sharp, heavy detail or noise looks 'camera-like'

Table 2.5: Measured cues and hypothesis per error.

The measured cues only explain part of it. Looking at the images themselves showed a clearer pattern. The false positives (#2, #6 and #7, real called AI) are all painted or illustrated "real" art: a richly coloured painted portrait, a 17th-century oil portrait and a flat vector illustration. The model has more or less learned that painterly, saturated images are AI, which is usually true in the training data. #3 is a glossy, over-lit stock photo of smiling people that honestly looks like generator output. The false negatives (AI called real) are photorealistic: #1 shows cows next to a robot in a barn that looks real, #5 is a photographic face with a cartoon pasted in, and #4 is a side-by-side composite of a painting and a press photo. #8 has large text over the picture, and #9 and #10 are landscapes that look a lot like real landscape paintings. Two of the errors (#4 and #9) have extreme aspect ratios that were squashed into a square. My conclusion is that the biggest weakness comes from the labels: in this dataset, style (painting or photo) is mixed up with origin (generator or human).

2.7 Tool lab: Keras Tuner

Toy part

`labs/tuner_toy.py` tunes a network with one hidden layer on Fashion-MNIST using Hyperband. It searches the number of units (32 to 512) and the learning rate (10^{-2}, 10^{-3} or 10^{-4}). Hyperband trains many configurations for a few epochs, keeps the best third, trains those for longer and repeats, so most of the budget goes to the promising settings. `tuner.get_best_hyperparameters()` returns the winner, which was 448 units with a learning rate of 10^{-3} (test accuracy 0.88).

Trial	Units	LR	Epochs	val accuracy
0005	448	0.001	8	0.8862
0006	224	0.001	8	0.8832
0004	448	0.001	3	0.8792
0001	224	0.001	3	0.8776
0003	512	0.001	3	0.8761
0009	256	0.0001	8	0.8748
0000	128	0.001	3	0.8637
0008	320	0.01	8	0.8467

Table 2.6: Toy tuner: best trials on Fashion-MNIST (validation accuracy).

```
!bash scripts/colab_regen.sh

== 2.7 toy tuner
Trial 0001 summary
Hyperparameters:
units: 224
lr: 0.001
tuner/epochs: 3
tuner/initial_epoch: 0
tuner/bracket: 1
tuner/round: 0
Score: 0.8775833249092102
Trial 0003 summary
Hyperparameters:
units: 512
lr: 0.001
tuner/epochs: 3
tuner/initial_epoch: 0
tuner/bracket: 1
tuner/round: 0
Score: 0.8760833144187927
BEST: units=448 lr=0.001
1500/1500 - 6s - 4ms/step - accuracy: 0.8210 - loss: 0.4968 - val_accuracy: 0.8490 - val_loss: 0.425
1500/1500 - 4s - 3ms/step - accuracy: 0.8646 - loss: 0.3706 - val_accuracy: 0.8680 - val_loss: 0.364
1500/1500 - 5s - 3ms/step - accuracy: 0.8788 - loss: 0.3284 - val_accuracy: 0.8771 - val_loss: 0.342
1500/1500 - 4s - 3ms/step - accuracy: 0.8879 - loss: 0.3022 - val_accuracy: 0.8798 - val_loss: 0.334
1500/1500 - 4s - 3ms/step - accuracy: 0.8954 - loss: 0.2808 - val_accuracy: 0.8834 - val_loss: 0.321
test accuracy with best settings: 0.8804
```

Figure 2.11: Toy tuner results summary.

Applied to VisionGuard

`scripts/tune.py` defines a `HyperModel` whose `fit()` runs the same two-stage training inside each trial. It searches the head learning rate (log-uniform between 10^{-4} and $3 \cdot 10^{-3}$), the dropout (0.1 to 0.5) and how many backbone layers to unfreeze (0, 11 for the last block, 30 or 60) with RandomSearch, 10 trials and validation loss as the objective. Each trial is also logged to MLflow as a nested run.

Rank	lr\{}_head	Dropout	Unfrozen	val loss	val AUC
1	2.24e-03	0.1	60	0.4773	0.864
2	8.80e-04	0.1	11	0.5440	0.796
3	6.45e-04	0.1	11	0.5908	0.768
4	1.02e-03	0.5	11	0.5911	0.777
5	4.54e-04	0.3	11	0.6397	0.695
6	2.29e-04	0.5	30	0.6478	0.675
7	2.52e-04	0.5	11	0.6551	0.680
8	2.85e-04	0.3	60	0.6605	0.688
9	1.27e-04	0.1	30	0.6806	0.649
10	1.16e-04	0.3	0	0.8280	0.413

Table 2.7: VisionGuard tuner trials, best first.

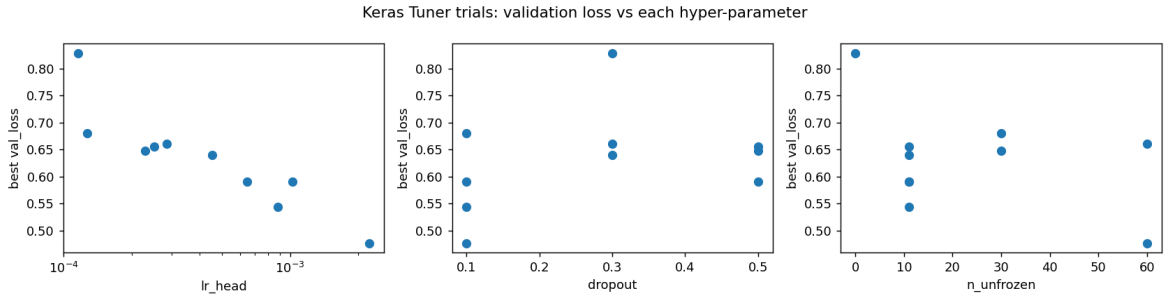


Figure 2.12: Validation loss of each trial against each hyper-parameter.

Hyper-parameter	Range of mean val loss across levels
unfrozen layers	0.2590
learning rate	0.1521
dropout	0.1362

Table 2.8: Effect size: spread of mean validation loss across the levels of each hyper-parameter.

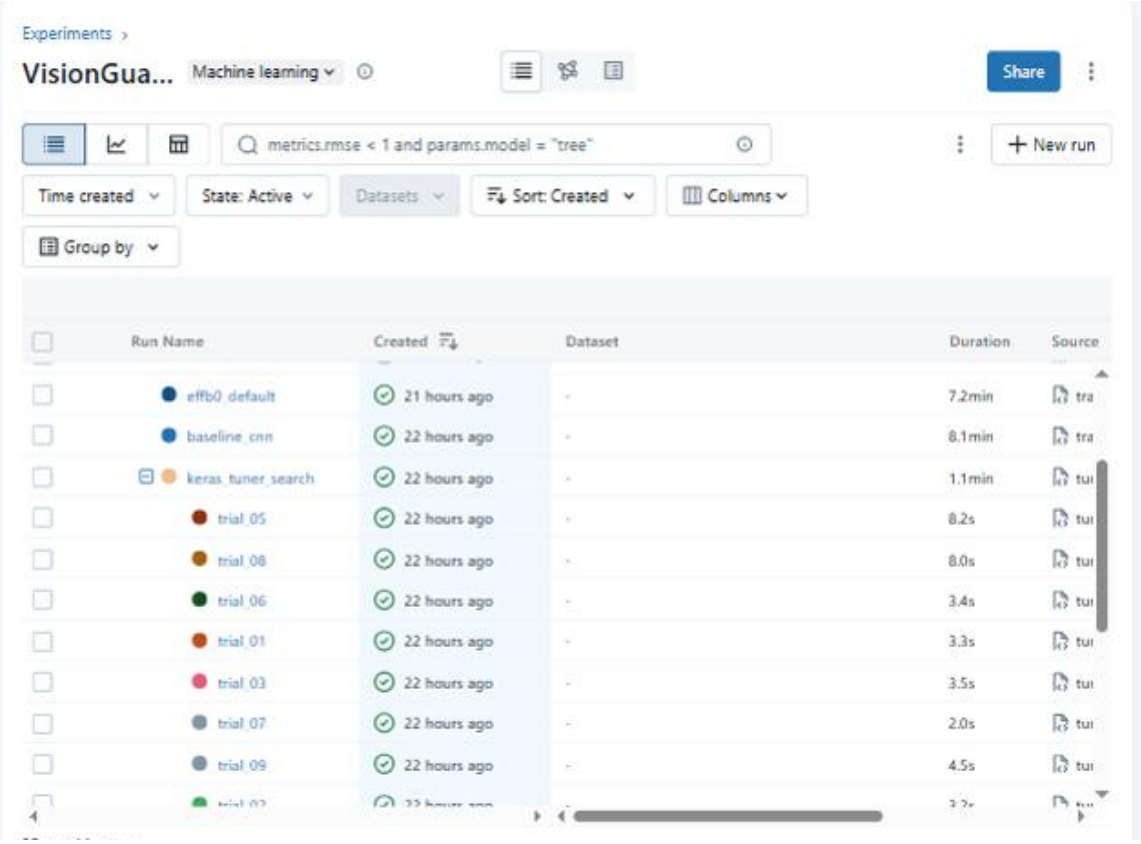


Figure 2.13: The tuner search in MLflow: parent run with one nested run per trial.

The best trial used a learning rate of 2.24e-03, dropout 0.1 and 60 unfrozen layers (validation loss 0.4773). The setting with the largest effect was unfrozen layers. That makes sense to me: the number of unfrozen layers decides how far the ImageNet features can adapt to generator artefacts, which ImageNet never had to detect. With 0 layers the network can only re-weight generic object features, while 60 layers with a small learning rate let the mid-level texture filters change. It also has the biggest influence on overfitting, which is why its levels differ the most in validation loss.

The best validation score from a search like this is too optimistic. The tuner looks at the validation loss of every trial and keeps the lowest one, and with many trials some configurations score well partly by luck on this small validation set (the “winner’s curse”). The selection itself fits noise in the validation data. That is why my final numbers come from the test set, which I used only once, and why a simpler setting that scores within noise of the best one would also be a reasonable pick.

Bonus: Grad-CAM

Figure 2.14: Grad-CAM heat maps: top row correct predictions, bottom row errors. Red = regions that pushed $p(\text{AI})$ up.

Grad-CAM takes the gradient of the output with respect to the last convolutional feature map, averages it per channel to get a weight for each channel, and shows the weighted sum of the feature maps as a heat map.

For the AI images that were classified correctly, the heat map covers the dense, swirling decorative texture (the ornamental "G" and the psychedelic face), the kind of high-frequency pattern generators produce. For real images it looks at a small region (the tortoise's shell, the area beside the Mona Lisa's face) and $p(\text{AI})$ stays below 0.5. In the errors it looks at the colourful headscarf and flowers (painted portrait called AI), the bright faces in the stock photo, and only the left, painted half of the composite (AI called real). This agrees with the error analysis: colour and texture style drive the decision more than real generation artefacts do.

3. Experiment Tracking with MLflow and DagsHub

3.1 Tool lab: MLflow locally

`labs/mlflow_toy.py` trains a network with one hidden layer on 10 000 Fashion-MNIST images. It logs the parameters (`lr`, `epochs`, `units`, `optimizer`), the per-epoch metrics through a small Keras callback (`mlflow.log_metrics(..., step=epoch)`), a loss plot (`mlflow.log_figure`) and the model (`mlflow.tensorflow.log_model`). I ran it twice, with `lr=0.001` and `lr=0.01`, against a local store (`-local` writes to `./mlruns_toy`), and then opened `mlflow ui -backend-store-uri ./mlruns_toy`. The comparison view overlays the two `val_accuracy` curves and highlights the parameter that differs. These two runs ran on CPU in a separate Linux workspace (TensorFlow 2.20, MLflow 2.22.5) with the UI on `127.0.0.1`.

After 5 epochs the higher learning rate fits the training data faster (training accuracy 0.862 against 0.849, loss 0.373 against 0.437) but does no better on validation (0.842 against 0.846), so I would keep `lr=0.001`.

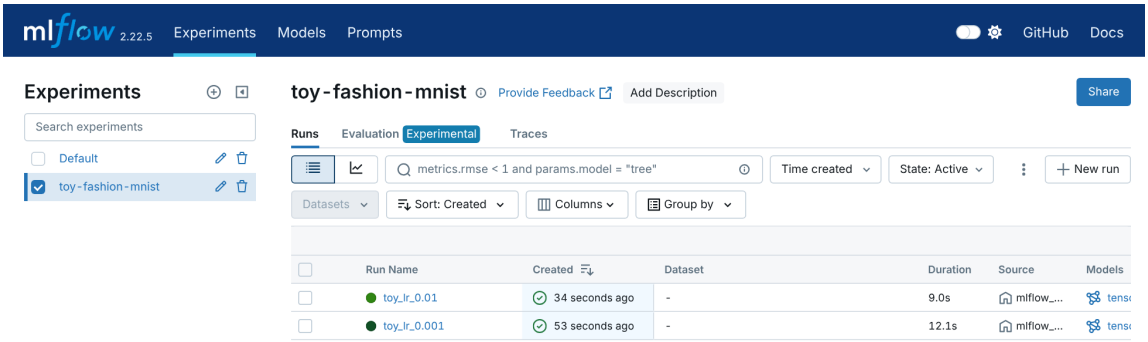


Figure 3.1: Local MLflow UI: the toy experiment with both runs.

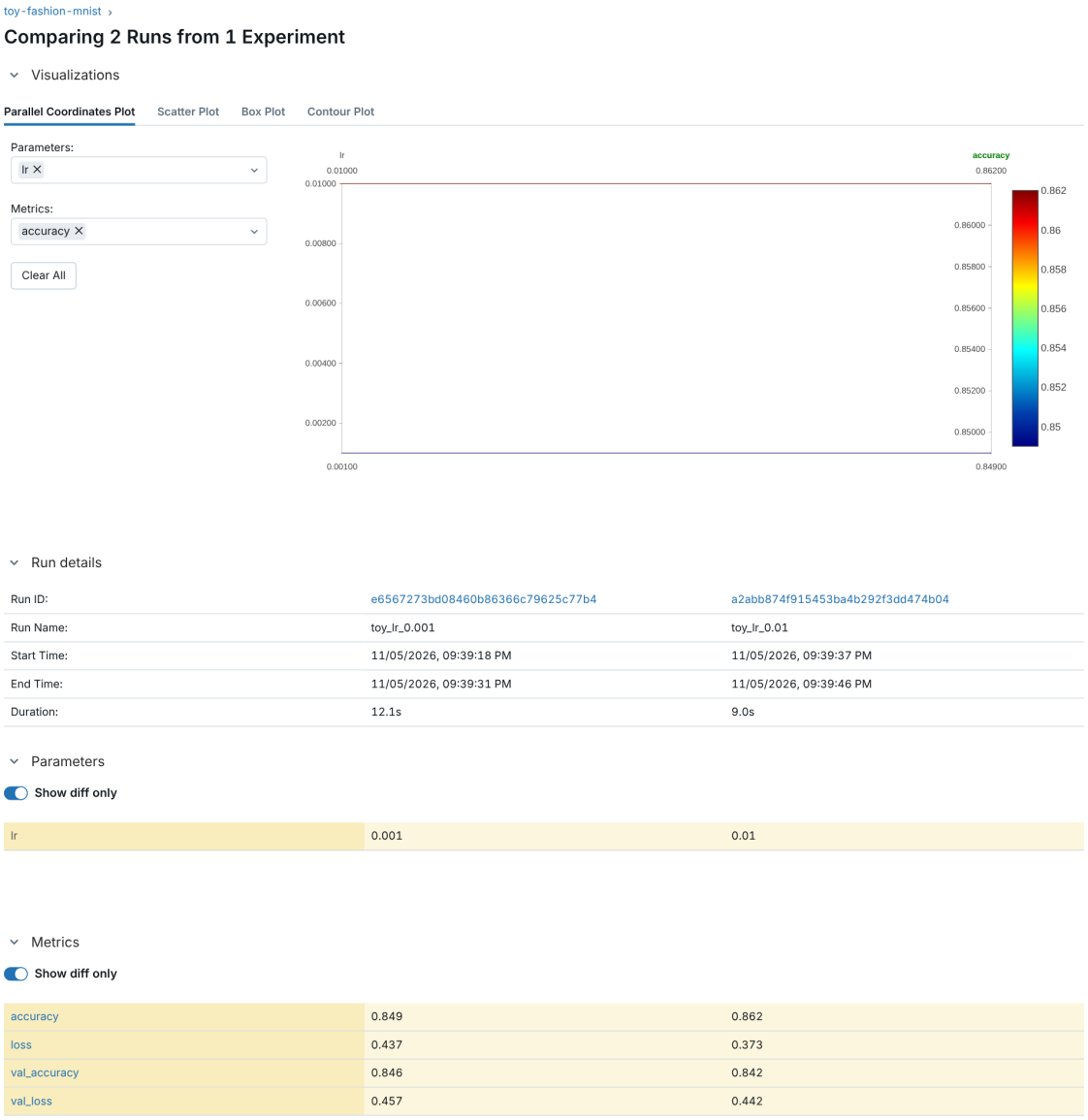


Figure 3.2: Comparison of the two learning rates.

In my own words, the MLflow terms are:

Experiment a named group of runs working on the same problem, so they can be compared side by side. Here I have `VisionGuard` and `toy-fashion-mnist`.

Run one execution of the training code. It has a unique id, a start and end time and a status, and everything logged while it runs belongs to it.

Parameter an input setting that is fixed for the run and written once, such as the learning rate. Parameters cannot be changed: logging a different value for the same key raises an error.

Metric a numeric result that can be logged many times with a step, for example `val_loss` after every epoch. This is how MLflow draws the curves.

Artifact any output file stored with the run, such as plots, CSVs, the requirements list or the model itself.

Tag free-form key/value information that describes the run without configuring it, such as the git commit, the data version or who ran it. Tags can be edited later and are useful for searching.

3.2 DagsHub setup

I created the DagsHub project `23L-0764_A2_Ops` with "Connect a repository", so it mirrors my GitHub repository and adds DVC storage and a hosted MLflow server. The tracking address is `https://dagshub.com/Anasnaveed69/23L-0764_A2_Ops.mlflow`, but it does not appear anywhere in the code. `tracking.py` calls `load_dotenv()` and reads `MLFLOW_TRACKING_URI`, and MLflow reads `MLFLOW_TRACKING_USERNAME` and `MLFLOW_TRACKING_PASSWORD` (the DagsHub token) from the environment. These values are in `.env`, which is listed in `.gitignore` and `.dockerignore`, and `.env.example` shows the variable names with dummy values. If `MLFLOW_TRACKING_URI` is not set, the code falls back to a local `mlruns/` folder, so it still works offline.

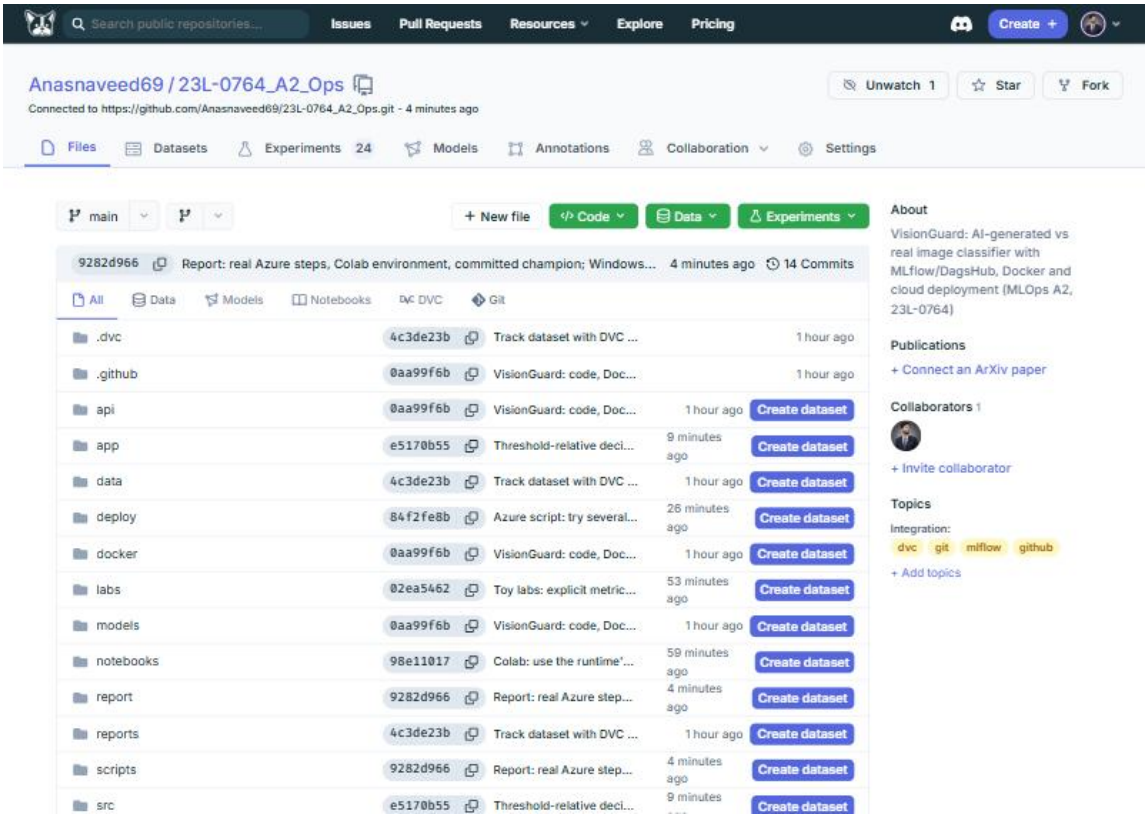


Figure 3.3: The DagsHub project connected to GitHub.

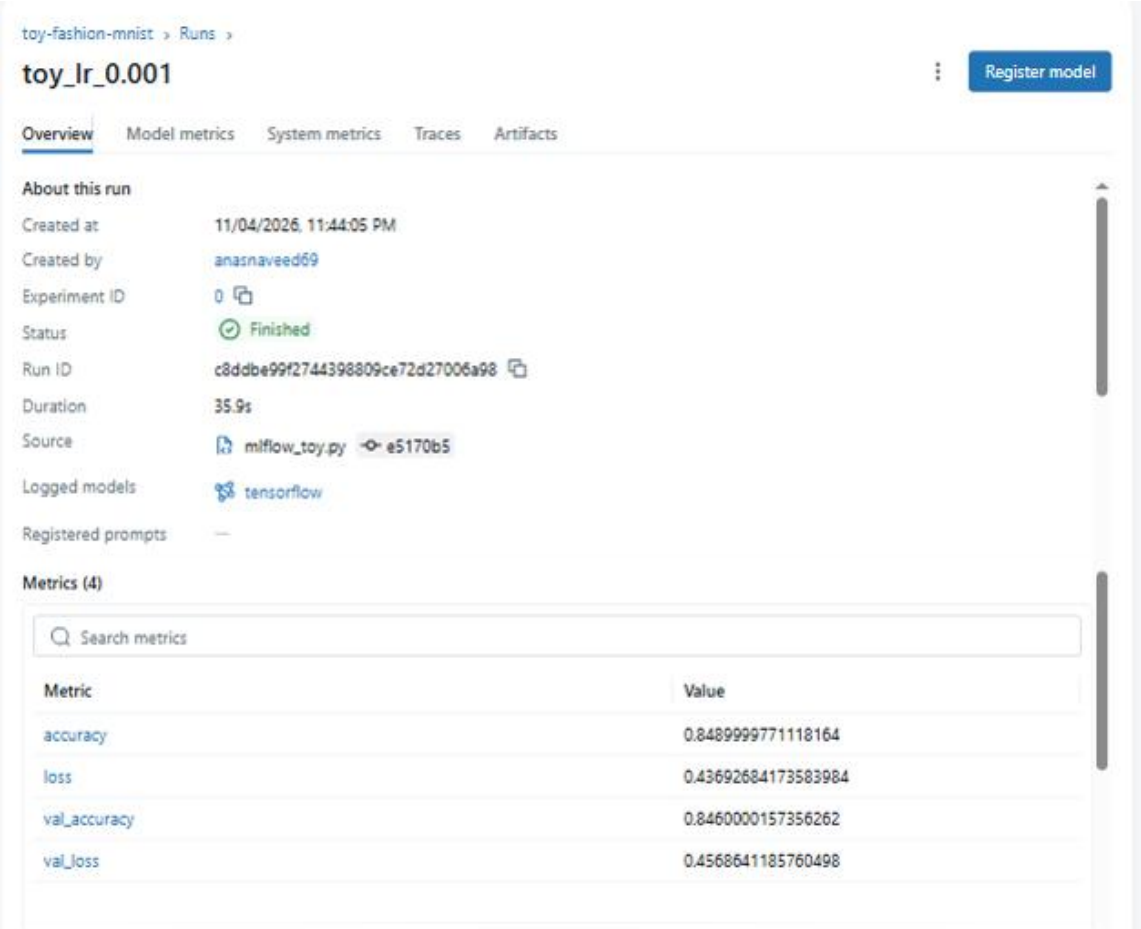


Figure 3.4: The toy run logged to DagsHub's MLflow server.

```
23L-0764_A2_Ops — .env is never committed
$ git check-ignore -v .env
.gitignore:2:.env	.env
$ git status --short --ignored | grep -E "\.env|mlruns|data/raw"
!! .env
!! mlruns_toy/
$ git ls-files | grep -c "^\.env$"
0
$ grep -n "env" .gitignore
2:.env
3:.env.*
4:!.env.example
26:.venv/
27:venv/
```

Figure 3.5: `.env` is git-ignored: `git check-ignore -v .env` names the rule and `git status` does not list it.

3.3 Instrumenting VisionGuard training

Each run of `scripts/train.py` logs:

- every value from `params.yaml` that was actually used, flattened into names like `train.lr_head`, plus the split sizes and class weights, as parameters;
- loss, accuracy, AUC, precision and recall for training and validation after every epoch, and the learn-

ing rate, so ReduceLROnPlateau shows up in the curves;

- final metrics: `best_epoch`, the chosen `val_threshold`, all validation and test metrics at that threshold (`test_accuracy`, `test_f1`, `test_roc_auc`, `test_false_positive_rate`, the confusion-matrix cells), and test accuracy and F1 at the plain 0.5 threshold for comparison;
- tags: `git_commit`, `git_dirty`, `dvc_data_version` (the md5 from `data/raw.dvc`), `seed`, the Python, TensorFlow and Keras versions, the platform, model type, backbone and tracking mode.

With manual tracking (`-tracking manual`, for example the run `mnv2_default`) my callback `MLflowEpochLogger` passes Keras' `logs` to `mlflow.log_metrics` at the end of every epoch with a global step, so the two training stages form one continuous curve. With autologging (`-tracking autolog`, run `mnv2_autolog`), `mlflow.tensorflow.autolog()` patches `model.fit()` and records things by itself.

	Manual callback	Autologging
Captures	exactly what I choose: per-epoch metrics and LR, my parameters and tags	every `fit()` argument (epochs, batch size, class weight…), all optimizer settings (`opt_*`), per-epoch metrics, model summary and EarlyStopping info, with no extra code
Misses	anything I forget to log	test metrics, the chosen threshold, git/DVC tags, the confusion matrix and the splits, because it only sees `fit()`
Two-stage training	one continuous run	the second `fit()` tries to log `epochs` again with a new value, which is not allowed, so I put stage 2 in a nested child run
Clutter	short	many parameters that have nothing to do with the experiment

Table 3.1: Manual logging versus autologging.

In practice I would use both together: autolog for what happens inside training, and explicit logging for provenance and evaluation. Both kinds of run here came from the same configuration.

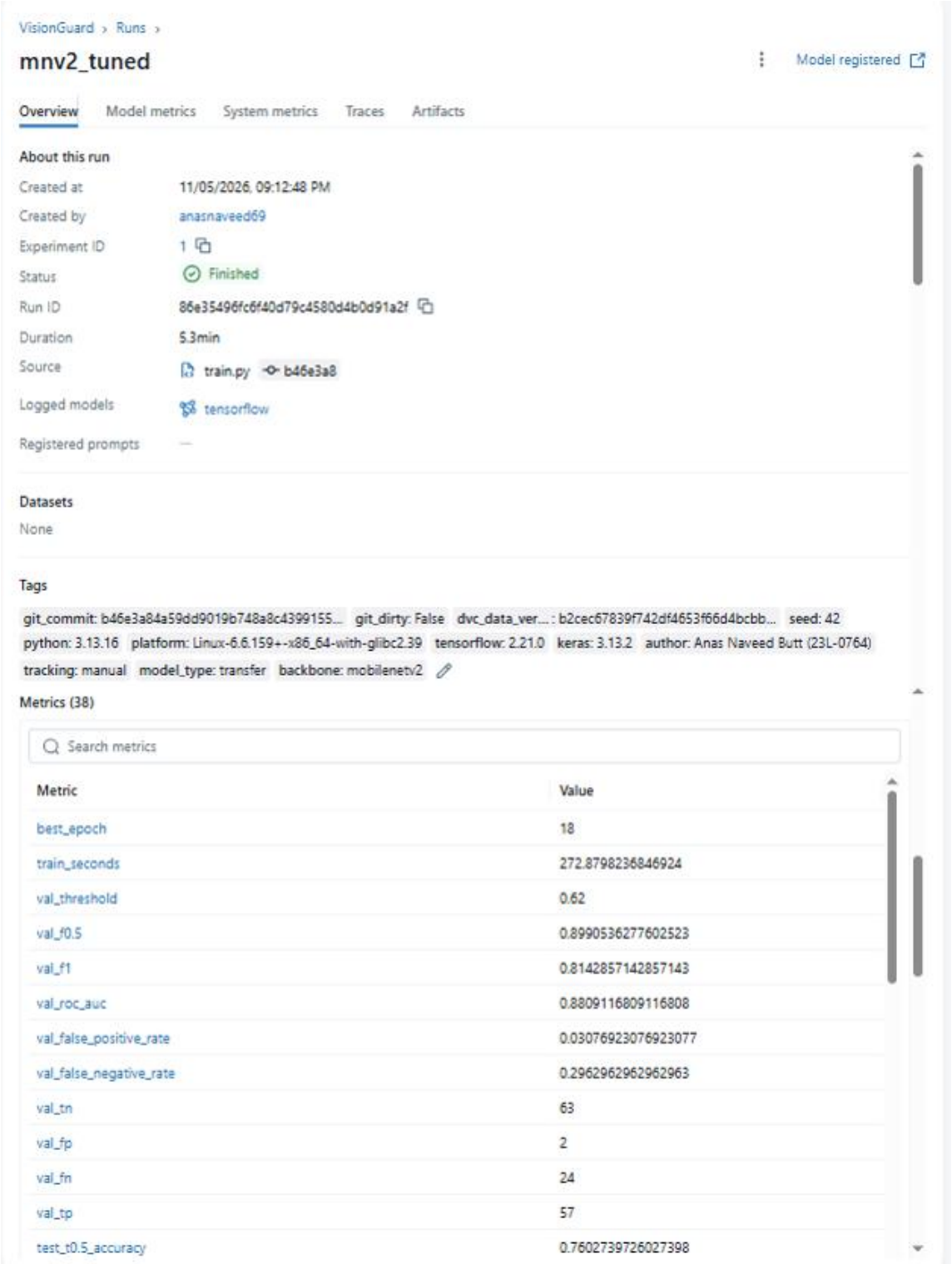


Figure 3.6: Manual run: parameters, metric curves and provenance tags.

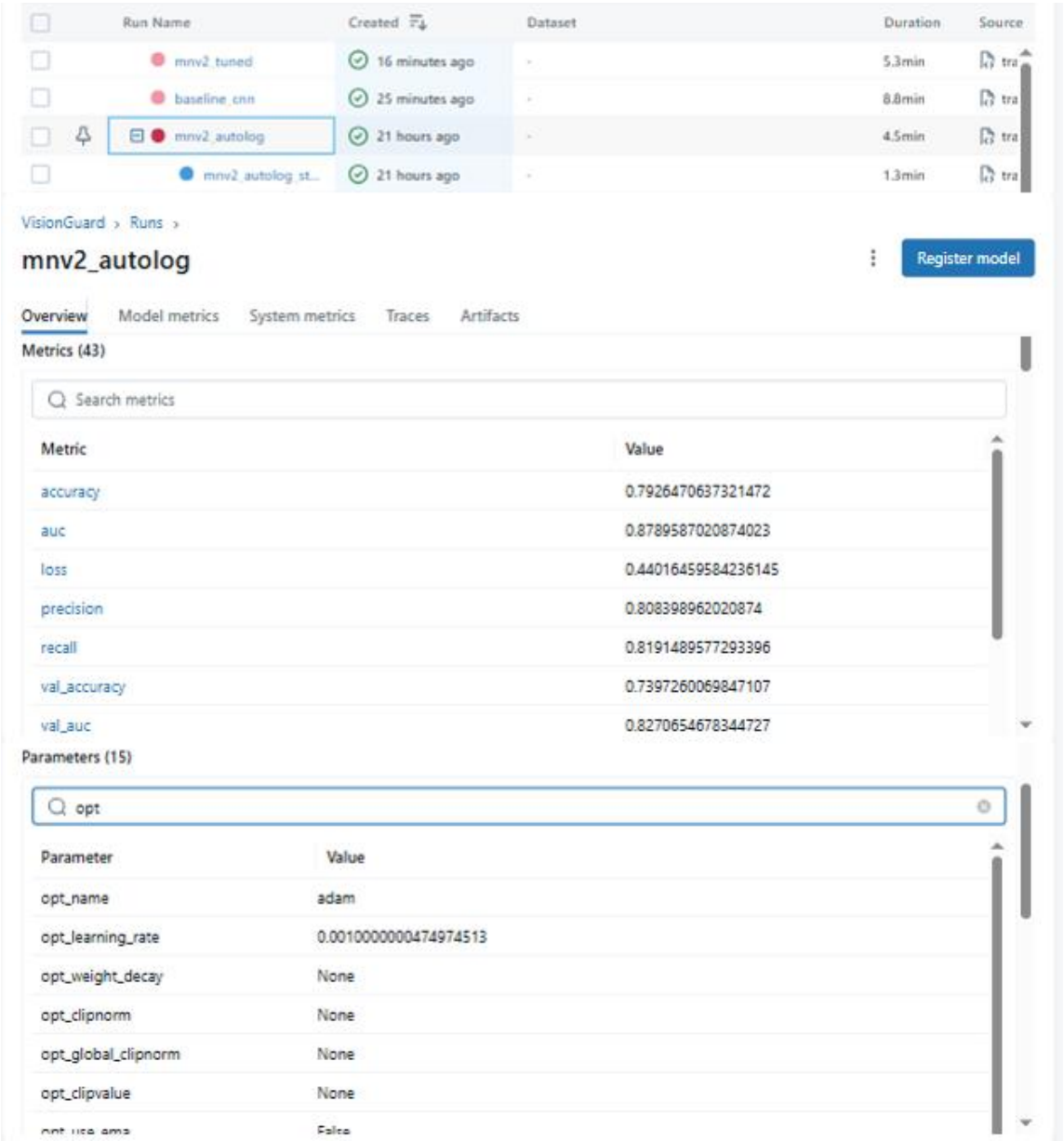


Figure 3.7: Autologged run: the automatically captured `fit()` and optimizer parameters.

I hit two errors while setting this up. MLflow rejected my first metric name, `test@0.5_accuracy`, because metric names may only contain letters, digits, `_ - . / :` and spaces, so I renamed it to `test_t0.5_accuracy`. Then the first autolog run had no epoch metrics at all. Autolog tried to log `class_weight`, which I had already logged under the same key in a different string format, and because parameters cannot change, the autolog hook gave up. I renamed my parameter to `class_weights` and moved the second `fit()` into a nested run.

3.4 Artifacts

Each run stores `plots/` (training curves, confusion matrix, ROC curve), `splits/` (the exact train, validation and test CSVs), `environment/` (`requirements-freeze.txt` from `pip freeze`, and `params.yaml`), `evaluation/test_predictions.csv`, `model_info/` (model summary and the trainable layers per stage), `keras_model/` (the `model.keras` file and `model_meta.json` with image size, resize mode and threshold) and `model/`, the MLflow TensorFlow-flavour model with an input/output signature that the registry uses.

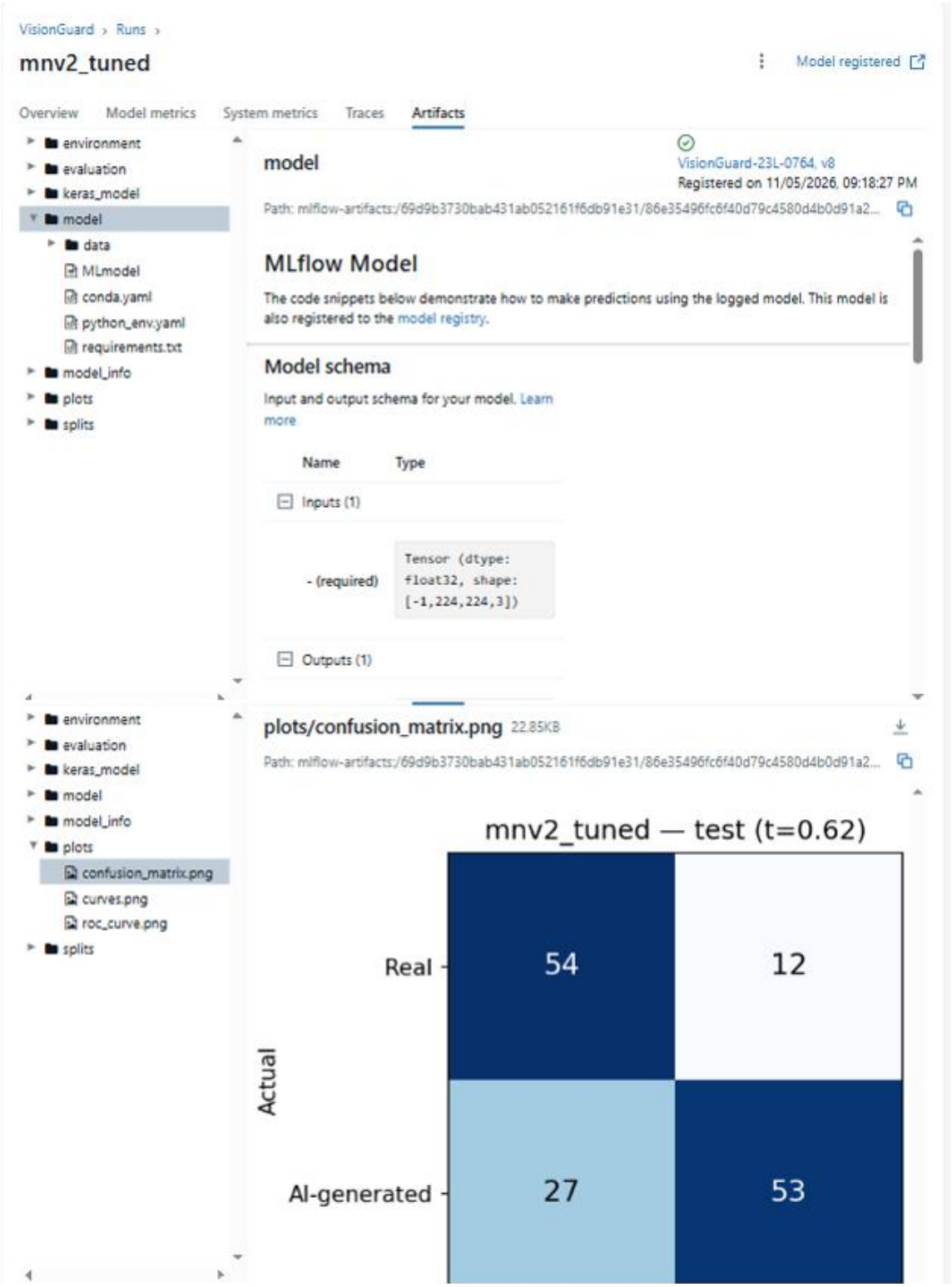


Figure 3.8: Artifacts tab of the champion run.

3.5 Experiment comparison

`scripts/run_experiments.py` launches the reference run and a set of variants, each changing one thing compared with the reference, so any difference can be put down to that change. `scripts/compare_runs.py` pulls all finished runs from MLflow and builds the table and the chart.

Run	Change vs reference	val AUC	val FPR	test acc	test F1	test AUC	best ep.
mnv2_tuned	tuner's best: lr 2.2e-3, dropout 0.1, 60 layers unfrozen	0.881	0.031	0.733	0.731	0.844	18
mnv2_no_aug	augmentation off	0.859	0.031	0.678	0.618	0.814	18
mnv2_lr_1e-4	head lr 1e-3 -> 1e-4	0.835	0.046	0.651	0.598	0.760	22
mnv2_default	reference (MobileNetV2, lr 1e-3, aug, 224, last block)	0.830	0.092	0.685	0.676	0.754	13
mnv2_autolog	same as reference, autolog tracking	0.830	0.123	0.678	0.671	0.764	13
mnv2_frozen	no fine-tuning (all frozen)	0.822	0.062	0.678	0.647	0.733	7
effb0_default	backbone -> EfficientNetB0	0.805	0.062	0.685	0.629	0.852	22
mnv2_img160	image size 224 -> 160	0.764	0.292	0.747	0.758	0.793	18
baseline_cnn	model: CNN from scratch	0.757	0.123	0.651	0.598	0.743	38

Table 3.2: All VisionGuard runs (sorted by validation ROC-AUC).

Figure 3.9: Validation ROC-AUC and test F1 per run.

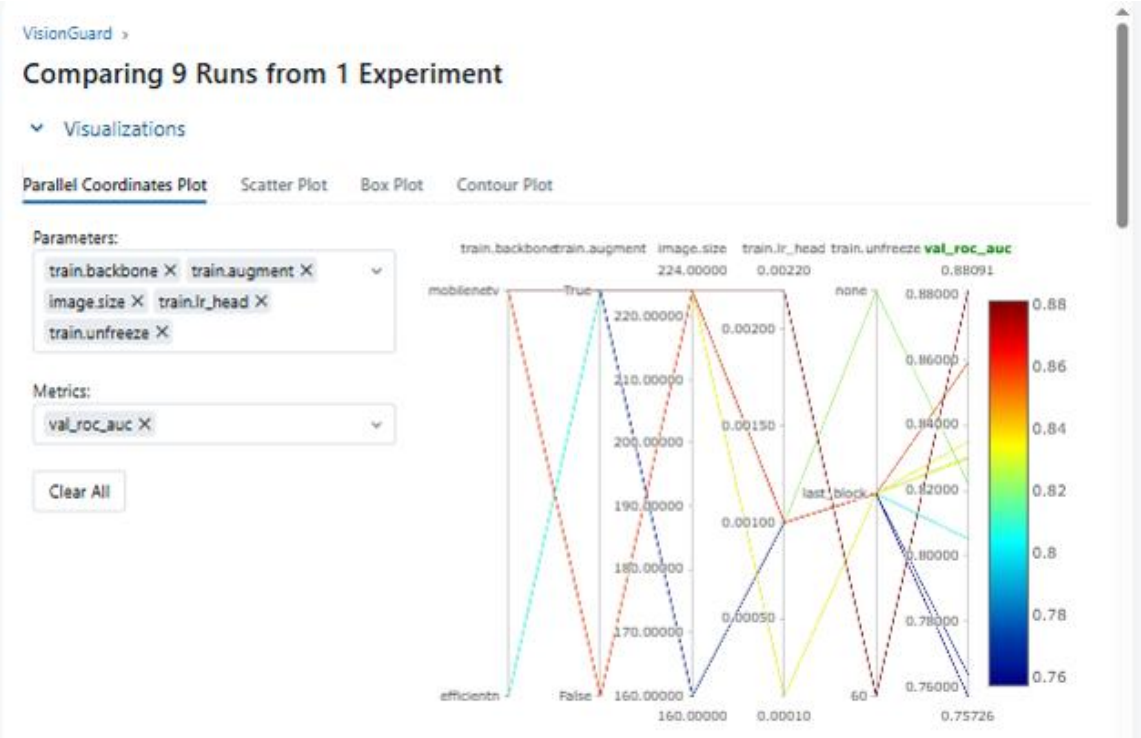


Figure 3.10: MLflow/DagsHub run comparison view.

I pick the model by validation ROC-AUC, which does not depend on a threshold and measures how well the model ranks AI images above real ones. Ties are broken by the validation false-positive rate, since false accusations are the expensive error. The test metrics are in the table but play no part in the choice; otherwise the test set would turn into a second validation set. The chosen run is `mnv2_tuned`, with validation AUC 0.881, test AUC 0.844, test accuracy 0.733 and test F1 0.731.

Starting the transfer stage from the tuner's settings helped the most (validation AUC from 0.830 to 0.881, test F1 from 0.676 to 0.731). Turning augmentation off also improved the validation AUC, to 0.859. On a dataset this small the random flips, rotations and zoom mostly slowed learning down, and zoom and rotation also change the fine generator textures the model depends on. The 160 px input did worst on the selection metric (validation AUC 0.764, validation FPR 0.29), since shrinking the image wipes out fine texture. It had a higher test accuracy, but on 146 images that is noise, and the selection rule ignores it on purpose. EfficientNetB0 had the best test AUC (0.852) but a lower validation AUC, so I did not choose it. The autolog run matches the manual reference (validation AUC 0.830 for both; test AUC 0.764 against 0.754, because GPU training is not bit-exact).

My first version of the comparison ranked runs by `val_auc`. That is the Keras AUC from the last epoch the callback logged, which is not the best checkpoint that actually gets evaluated, and autologged runs did not have it with the same meaning. I switched to `val_roc_auc`, which my evaluation code computes on the reloaded best model for every run.

3.6 Model registry

`scripts/register_model.py` makes sure the registered model `VisionGuard-23L-0764` has at least two versions. The reference transfer run and the baseline CNN are registered during training with `-register`, and the selected run is registered if it is not there yet. The script gives the best version the alias `champion` and the runner-up the alias `challenger`, and adds a description and a `val_auc` tag. An alias works like a pointer that can be moved. To promote a new model I move the alias, and anything that loads `@champion` picks up the new version with no code change.

Version	Run	Alias	val AUC	test acc
1	mnv2_default		0.830	0.685
2	baseline_cnn		0.393	0.548
3	mnv2_no_aug		0.859	0.678
4	mnv2_lr_1e-4		0.835	0.651
5	baseline_cnn		0.751	0.664
6	mnv2_tuned		0.852	0.705
7	baseline_cnn		0.757	0.651
8	mnv2_tuned		0.881	0.733

Table 3.3: Registered versions and aliases.

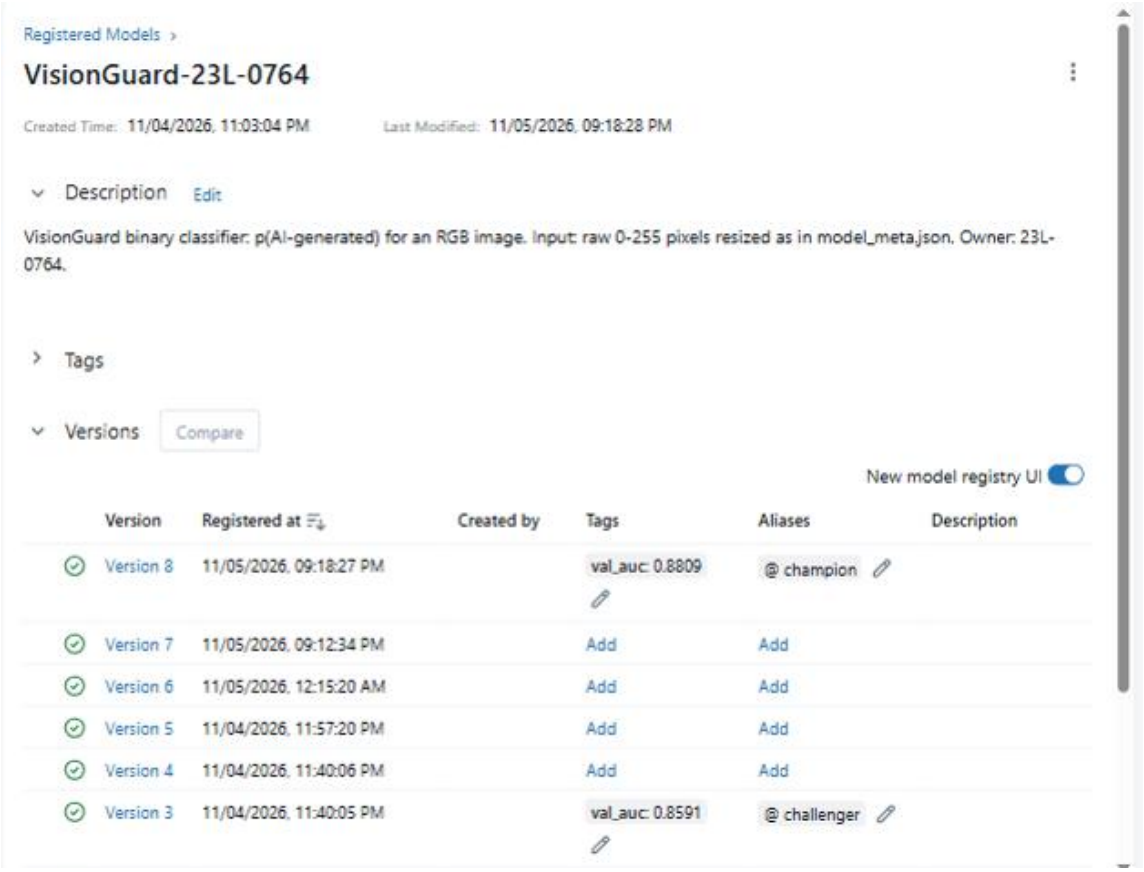


Figure 3.11: Model registry on DagsHub with the `champion` and `challenger` aliases.

`scripts/load_champion.py` is a separate script. It resolves the alias with `MlflowClient().get_model_version_by_alias()`, loads that version with `mlflow.tensorflow.load_model("models:/VisionGuard-23L-0764/<v>")`, reads the threshold and image size from the version's `model_meta.json` and classifies one image.

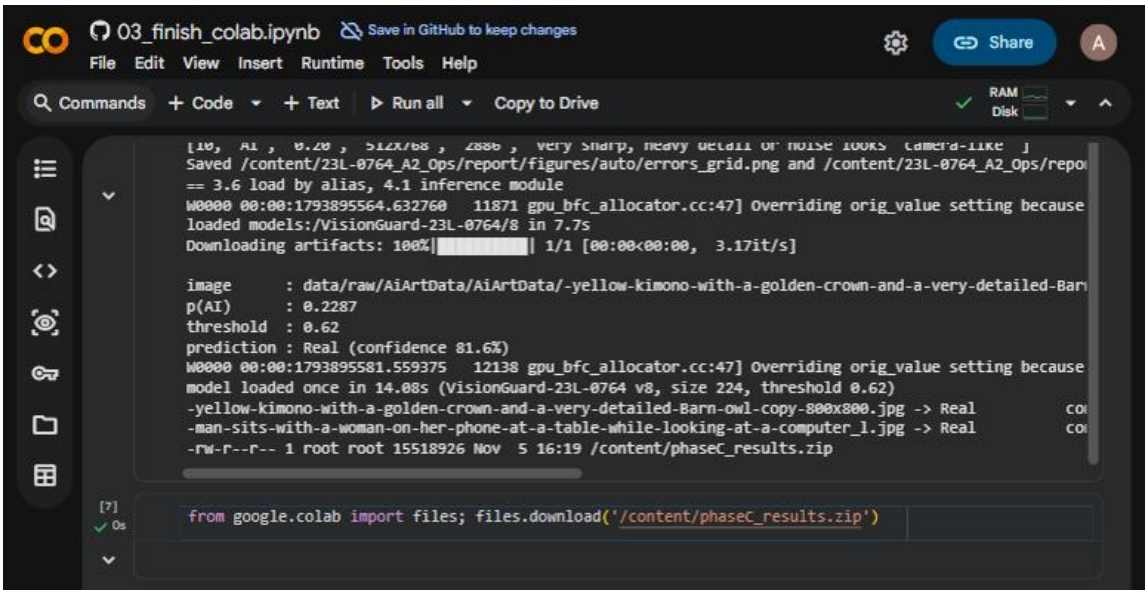


Figure 3.12: Loading the model by alias and classifying one image.

The serving processes should not contact DagsHub on every request, so `scripts/export_champion.py` resolves `@champion` once, downloads `model.keras` and writes `models/champion.keras` together with `champion.json` (version, threshold, image size). It saves the model again without the optimizer state, since serving never trains, which brings it down to 9.6 MB. I committed this exported copy to Git. It is small, only used for inference and can be regenerated from the registry, which remains the source of truth, and having it in the repository lets CI and the cloud VM build the API image without registry credentials. Locally the `models/` folder is mounted read-only into the web container; for the cloud API the file is copied into the image at build time (Chapters 6 and 7).

4. Flask Web Application

4.1 Inference module

`src/visionguard/inference.py` defines the `Predictor` class. Its constructor runs once per process: it loads `champion.keras` with `keras.models.load_model`, reads `champion.json` (image size, resize mode, threshold, version) and runs one dummy prediction to warm up the TensorFlow graph. `predict_bytes(data)` decodes and resizes the bytes, predicts $p(\text{AI})$, applies the threshold and returns the label, a confidence value, $p(\text{AI})$, the threshold, the model name and version, and the latency.

The confidence is measured from the threshold. It is 50% when $p(\text{AI})$ sits exactly on the threshold and 100% when $p(\text{AI})$ is 0 or 1. My first version reported the probability of the predicted label. With a threshold below 0.5 that gave odd results: an image with $p(\text{AI}) = 0.49$ was labelled AI-generated with a "confidence" of 49%, which looks contradictory to anyone reading it. Measuring from the threshold fixes that, and the result page also shows the threshold on a scale so the user can see where the image falls.

The model is loaded once because loading means deserialising millions of weights and building the graph, which takes seconds and hundreds of MB of memory. If I did it per request, every user would wait for it, memory would spike when several requests came in together, and the first prediction of a freshly loaded model is always the slowest. Kept in memory, a prediction takes tens of milliseconds.

Preprocessing in serving has to match training exactly. The model only learned from the kind of input it saw during training. If serving used a different interpolation, kept the alpha channel, scaled to another range or kept the aspect ratio while training squashed it, the inputs would come from a different distribution and the predictions would get worse without any error being raised. This is called training/serving skew. In my code it cannot happen: training, the audit and serving all call the same `decode_and_resize` function, the image size and resize mode are stored with the model in `champion.json`, and the backbone-specific scaling is a layer inside the saved model.

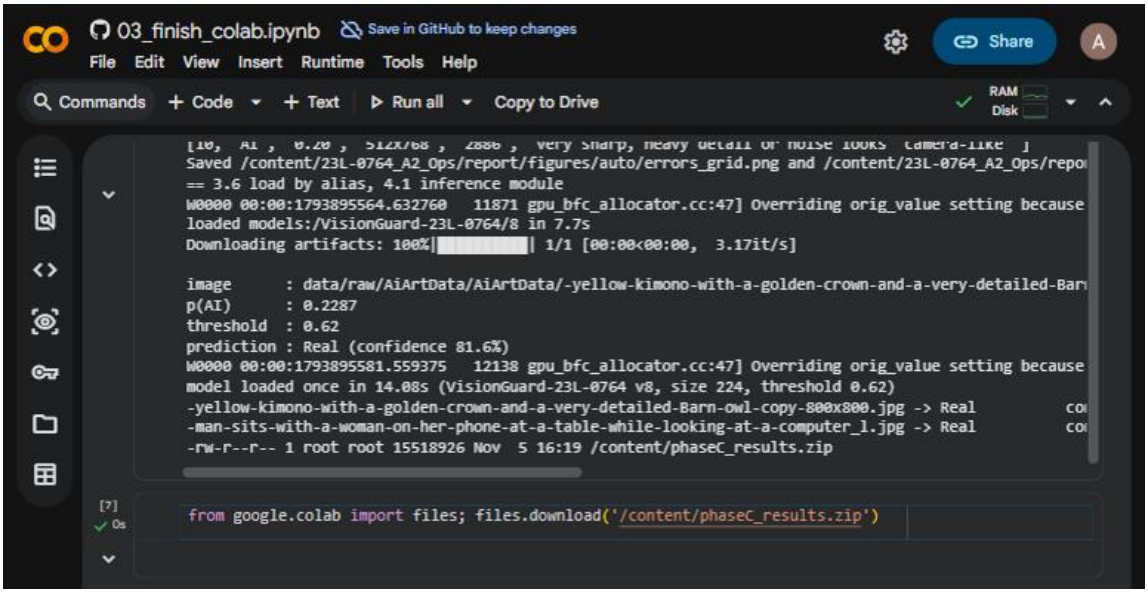


Figure 4.1: `scripts/infer_cli.py`: model loaded once, then two known test images (one AI, one real).

4.2 Routes and validation

Route	Method	Purpose
/	GET	upload page (drag and drop, preview, last 5 checks)
/predict	POST	validate, predict, write the DB row, redirect to the result (Post/Redirect/Get)
/result/<id>	GET	verdict, confidence, $p(\text{AI})$ against the threshold, caveat, audit record
/history	GET	paginated database trail, filter by verdict, show rejected uploads
/api/history	GET	the same trail as JSON
/health	GET	JSON health check: DB reachable, model backend reachable, model version

Validation is in `src/visionguard/validation.py` and runs the cheapest checks first:

1. No file gives 400.
2. An extension other than jpg, jpeg, png, webp, bmp or gif gives 415 (unsupported file type).
3. Flask's `MAX_CONTENT_LENGTH` rejects oversized requests before my code even runs, and the stream is read with a hard limit of 5 MB plus one byte. Too large gives 413.
4. A full decode with Pillow (verify, then load) catches renamed non-images and truncated files and gives 422 (not a readable image). Pillow's decompression-bomb limit is set as well.

File names go through `werkzeug.utils.secure_filename` (so `../../etc/passwd.png` becomes `etc_passwd.png`) and are only ever stored as text. The upload is read into memory and never written to disk; I keep only its SHA-256 hash, size and dimensions. Each error shows a simple page with the status code, a short explanation and a link back. Unexpected exceptions are logged with a traceback, and the user only sees a generic 500 page.

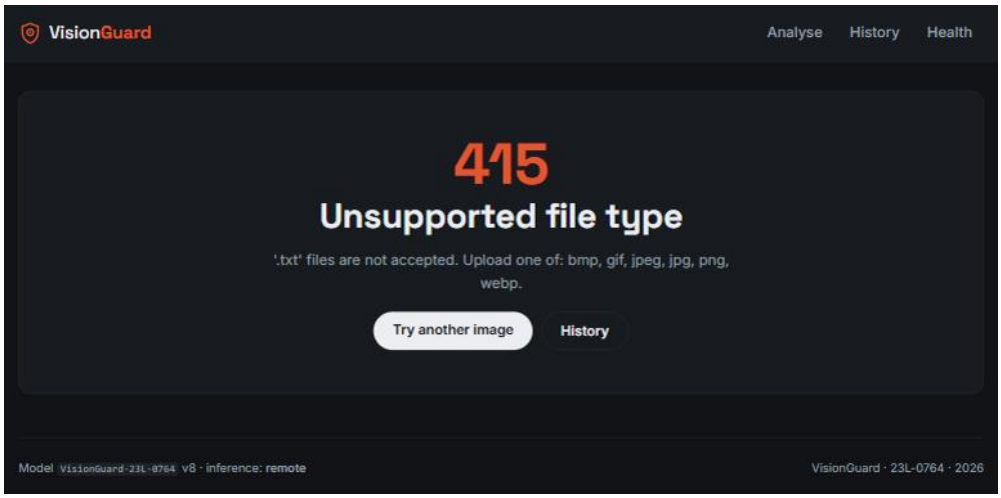


Figure 4.2: Failure case 1: wrong file type (415).

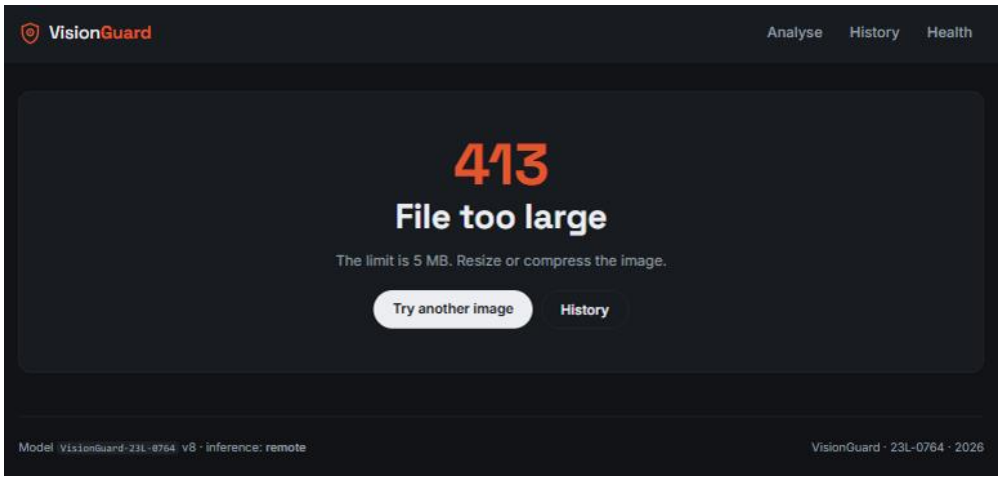


Figure 4.3: Failure case 2: file over the size limit (413).

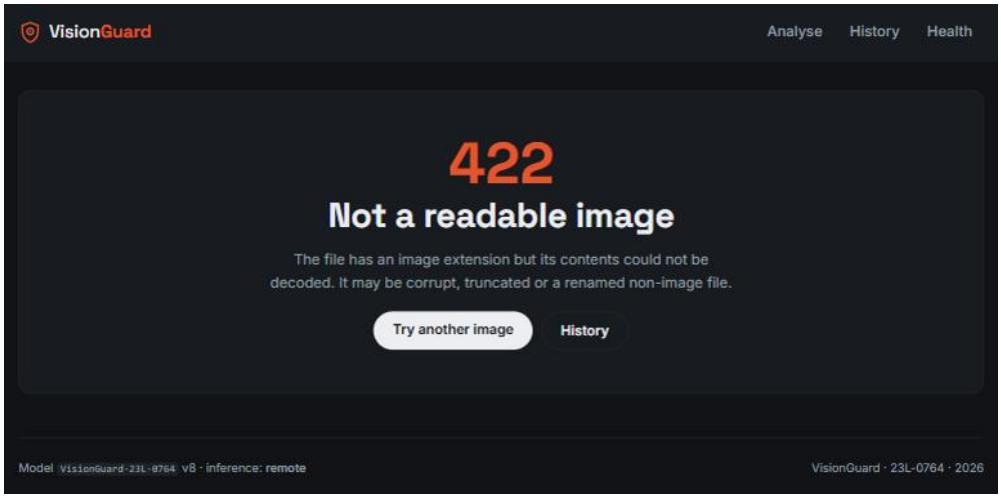


Figure 4.4: Failure case 3: random bytes saved as `.jpg` (422).

4.3 HTML and CSS

The templates use Jinja inheritance. `base.html` has the header, the navigation with the History menu, and a footer with the model version and inference mode. The result page shows the verdict in the colour of its class, a confidence bar, a scale with the decision threshold and the model's $p(\text{AI})$, and a "low certainty" badge when the confidence is under 65%. Below that is a caveat box. It says the result is a statistical estimate, that it is weakest on screenshots, heavily compressed images and newer generators, that the threshold was tuned to avoid false accusations, and that the result should never be used on its own to accuse someone. The stylesheet is plain CSS with a dark mode and a mobile layout.

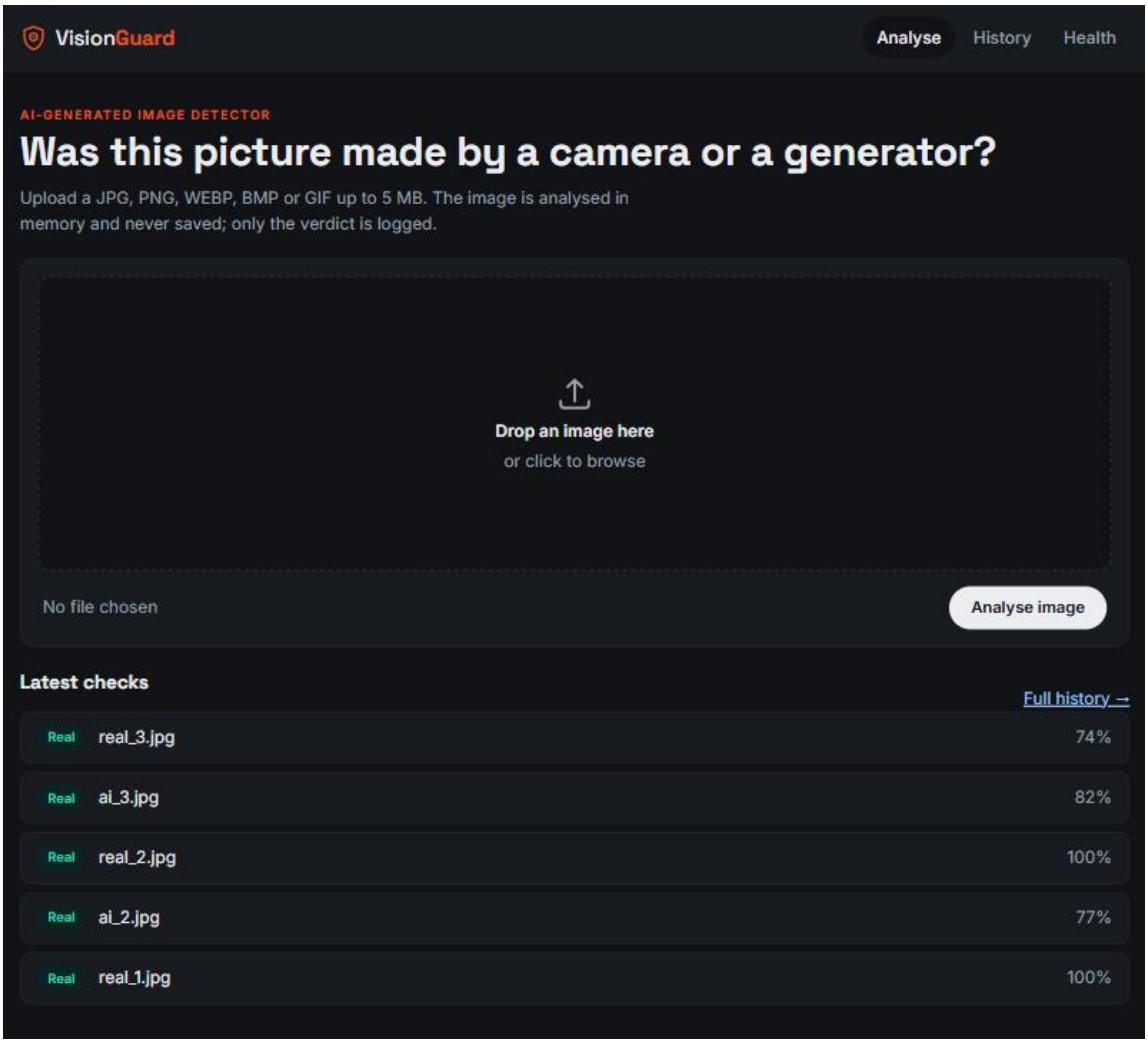


Figure 4.5: Upload page with the latest checks.

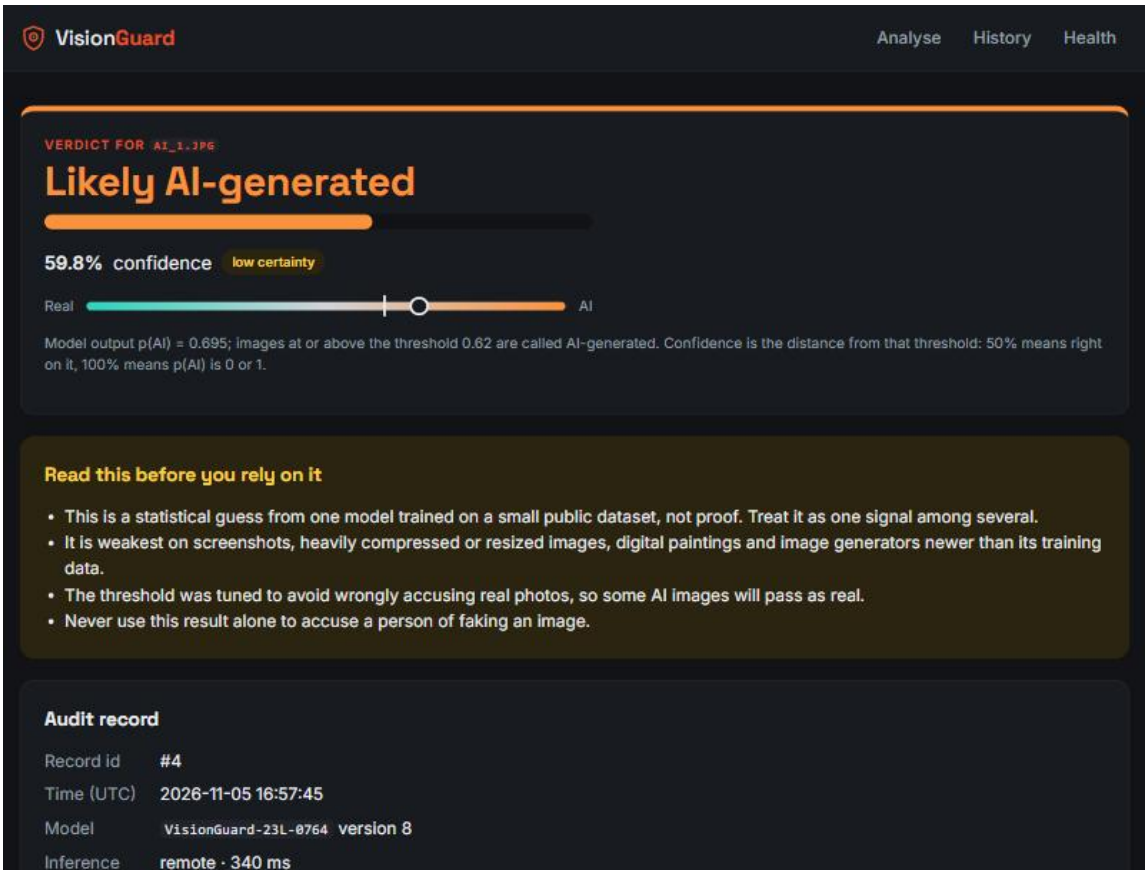


Figure 4.6: Result page with verdict, confidence, threshold scale and caveat.

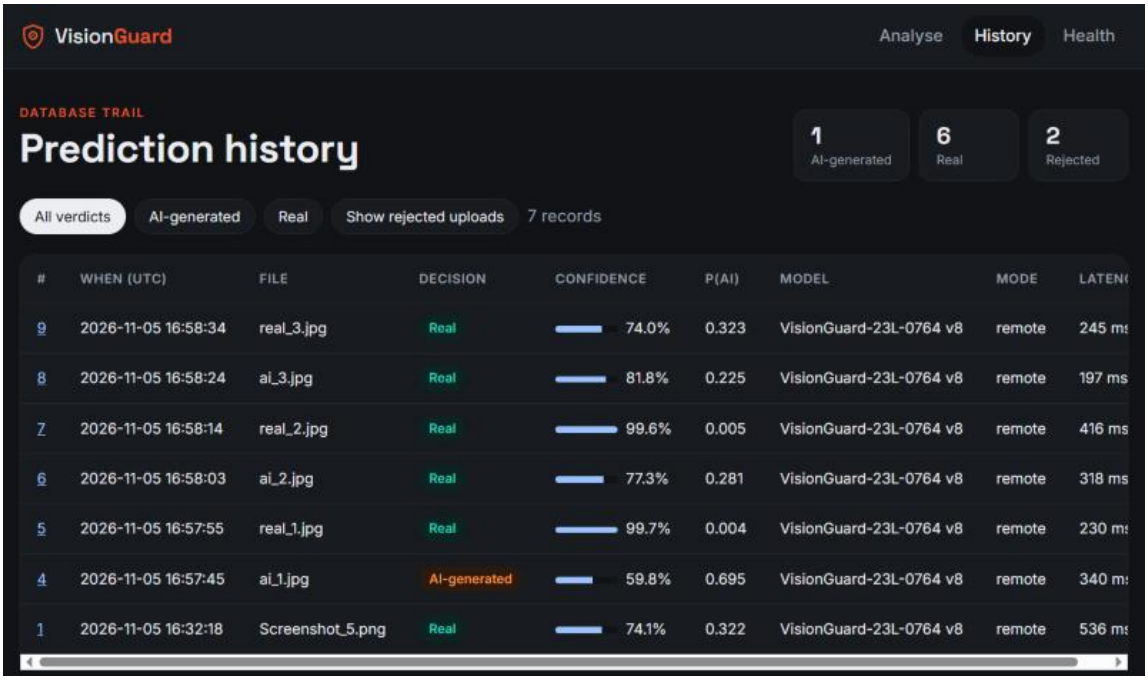


Figure 4.7: History menu: the database trail.

4.4 Configuration and production server

All settings are environment variables, read in `app/config.py`: `MODEL_PATH`, `THRESHOLD` (optional override), `DATABASE_URL`, `INFERENCE_MODE` (`local` or `remote`), `MODEL_API_URL`, `MODEL_API_KEY`, `MAX_UPLOAD_MB`, `SECRET_KEY` and `FLASK_DEBUG`. So the same code runs on my laptop, in Compose and against the cloud API without changes, and there are no secrets in the code or in the image.

In the container the app runs under gunicorn (1 worker, 4 threads) instead of `flask run`. Flask's built-in server is a single process meant for development and was never hardened for production. Debug mode is off (`FLASK_DEBUG=0`, shown as `"debug": false` in `/health`). With debug on, the Werkzeug debugger lets anyone who triggers an error run arbitrary Python from the browser, and the debug pages show source code and configuration. One worker keeps a single copy of the model in RAM, and the threads cover the time spent waiting on the database and the network.

Browser — http://127.0.0.1:5000/health

GET http://127.0.0.1:5000/health (Flask app on my laptop)


```
{
  "database": "ok",
  "debug": false,
  "inference_mode": "remote",
  "model": {
    "api": "https://20-219-10-43.sslip.io",
    "model_name": "VisionGuard-23L-0764",
    "model_version": "8",
    "threshold": 0.62
  },
  "model_backend": "ok",
  "status": "ok",
  "time": "2026-11-05T16:59:49+00:00"
}
```


Figure 4.8: `/health` response: database ok, model backend ok, debug false.

5. Database Trail

Every request leaves a permanent row that records what was decided, when, by which model and how confidently. In the Compose stack this is PostgreSQL, which handles concurrent writes and keeps its data in a named volume. For local runs and unit tests I use SQLite. The code is the same in both cases because it goes through SQLAlchemy; only `DATABASE_URL` changes.

Column	Type	Meaning
`id`	integer PK	record number, shown on the result page
`created_at`	timestamptz (indexed)	UTC time of the request
`status`	varchar	`ok`, `rejected` (failed validation) or `error` (model unreachable)
`filename`	varchar	cleaned-up upload name
`image_sha256`	char(64)	identifies the image without storing it, so repeated uploads can be spotted
`width, height, size_kb`	int/float	basic properties of the upload
`label`	varchar (indexed)	the decision: AI-generated or Real
`confidence, prob_ai, threshold`	float	how confident the model was, and the rule that produced the label
`model_name, model_version`	varchar	which model (registry name and version)
`inference_mode`	varchar	`local` (in-process) or `remote` (cloud API)
`latency_ms`	float	prediction or round-trip time
`error`	varchar	reason for a rejected or failed request

Table 5.1: Table `predictions`.

`app/db.py` creates the engine once at start-up, with a retry loop in case the database is still starting, and creates the table if it does not exist. In `/predict`, once the model has answered, one row is inserted in its own transaction (`Session.begin()` commits, or rolls back on an error) and the browser is redirected to `/result/<id>`. That page reads the row back from the database, so if the result page shows up, the record exists. Rejected uploads and model failures get a row too, so the trail has every attempt, including the failed ones.

`/history` lists the newest records first, 20 per page, with totals per verdict, filters for AI-generated and Real, and a toggle that shows the rejected uploads. `/api/history` returns the same rows as JSON.

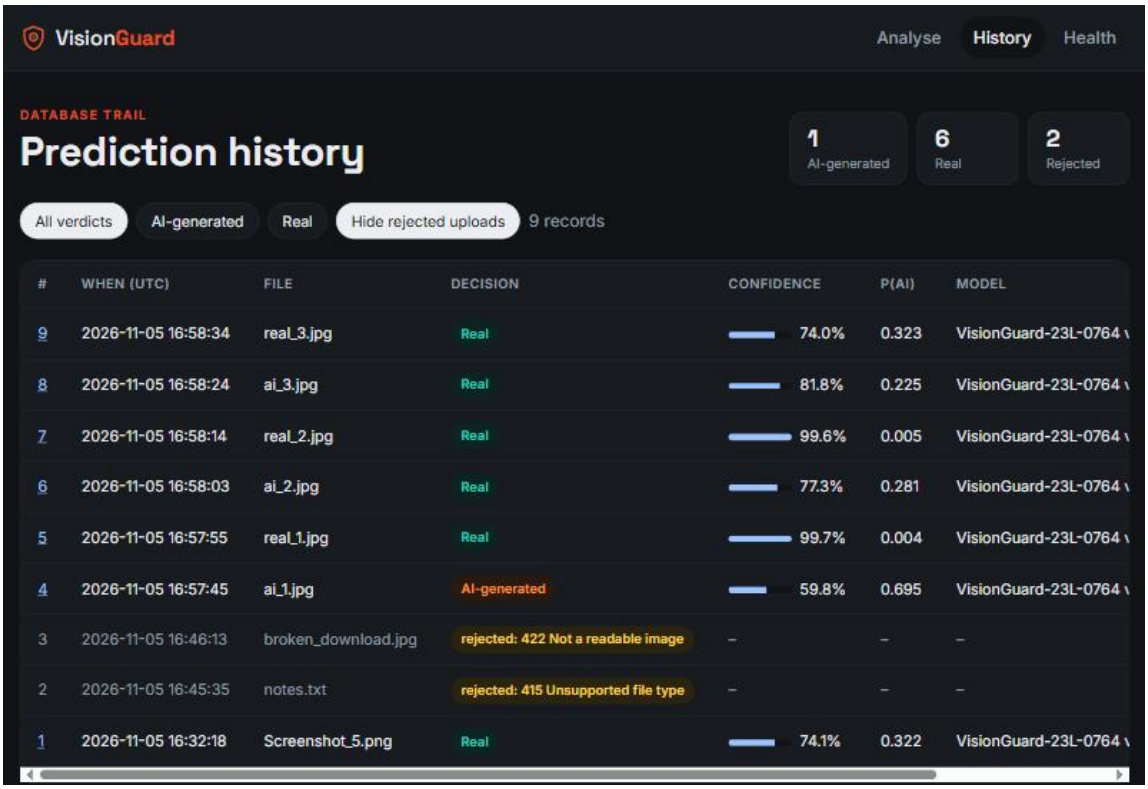


Figure 5.1: History page with the rejected uploads shown (laptop run, SQLite).

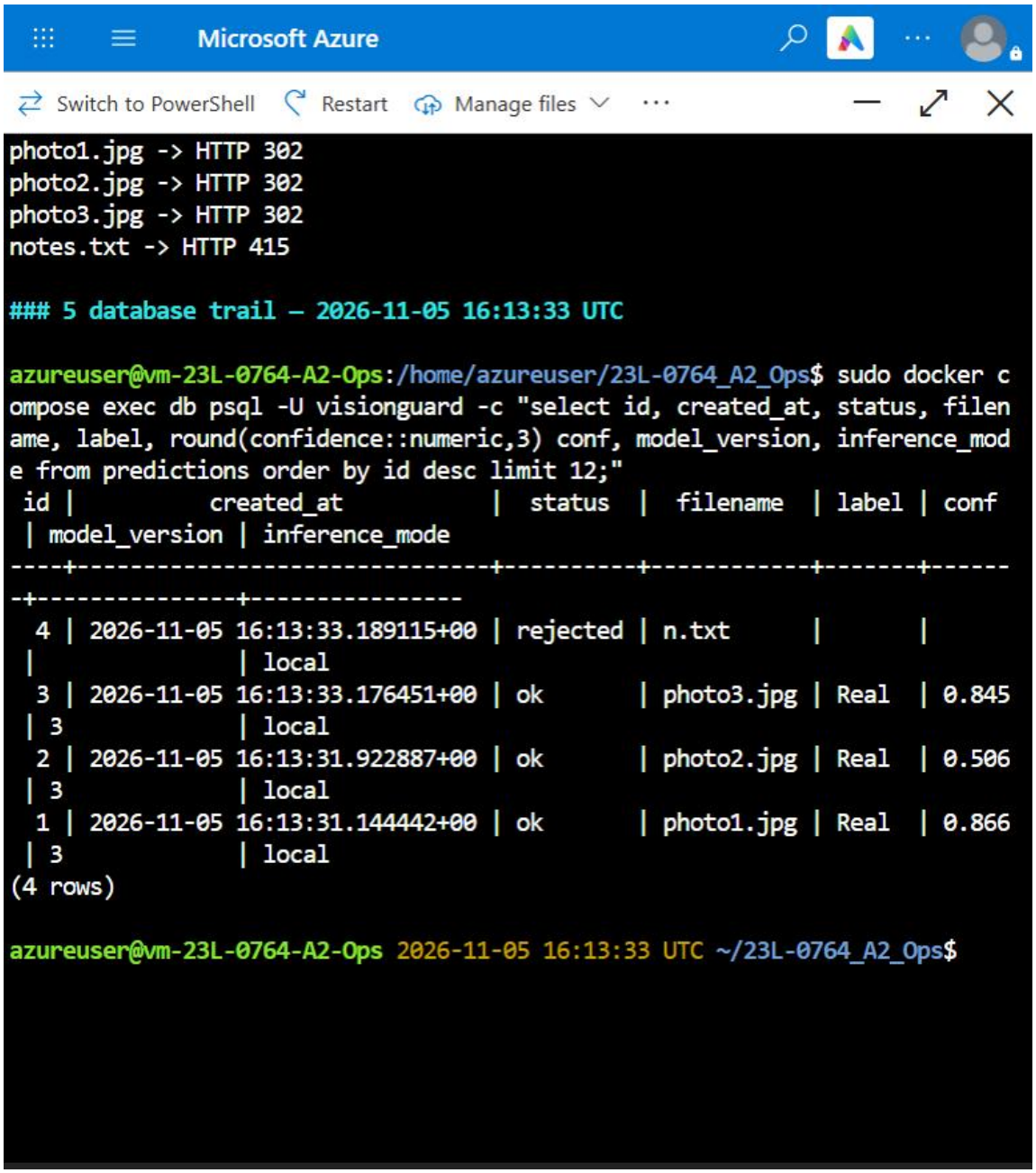


Figure 5.2: Records queried directly in PostgreSQL inside the `db` container of the Compose stack.

6. Containerization with Docker and Docker Compose

I did all the Docker work in this chapter on the project's Ubuntu 24.04 VM (`vm-23L-0764-A2-Ops`, Docker Engine 29.9, Compose v5.7), connected over SSH from Azure Cloud Shell. Every screenshot shows the `azureuser@vm-23L-0764-A2-Ops` prompt with the date and time. The same files also work with Docker Desktop on Windows without changes, and GitHub Actions builds and smoke-tests both images on every push (Section 6.6).

6.1 Docker warm-up

Before putting my own project in containers I practised with the ready-made `nginx:1.27-alpine` image (`labs/docker_warmup.md`). The command `docker run -d -name warmup -p 8081:80 -v .../labs/warmup-site:/usr/share/nginx/html:ro nginx:1.27-alpine` runs it in the background (`-d`), maps port 8081 on the host to port 80 in the container (`-p`) and mounts a folder with my own `index.html` read-only (`-v ...:ro`). The page then shows up at `http://localhost:8081`. `docker logs` shows an nginx access-log line for each refresh, `docker exec -it warmup sh` opens a shell inside the running container and `docker ps -a` lists the containers. `docker stop` and `docker rm` remove the container, and `docker images` and `docker rmi` list and remove the image.

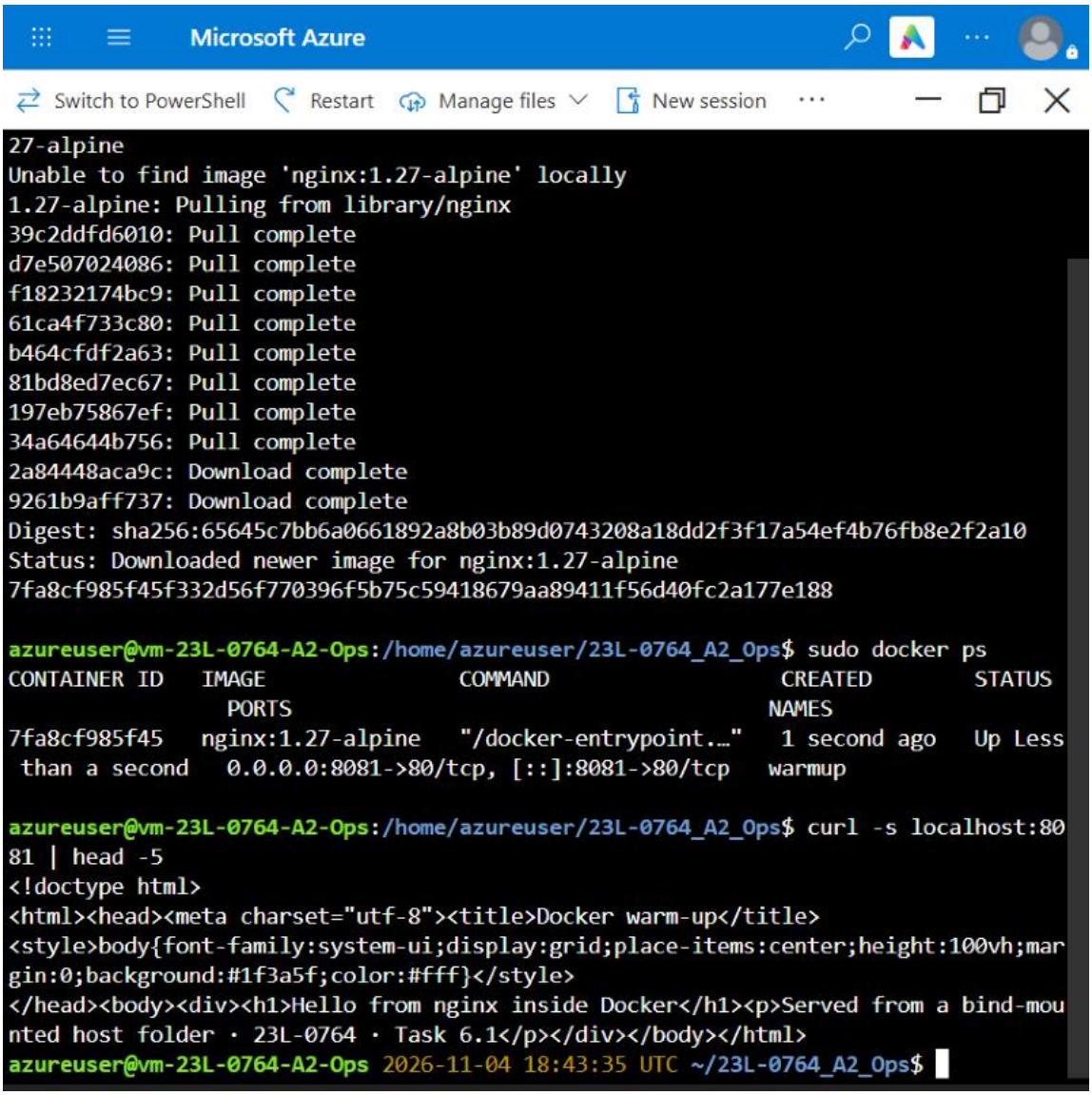


Figure 6.1: Run with port mapping and volume; page served in the browser.

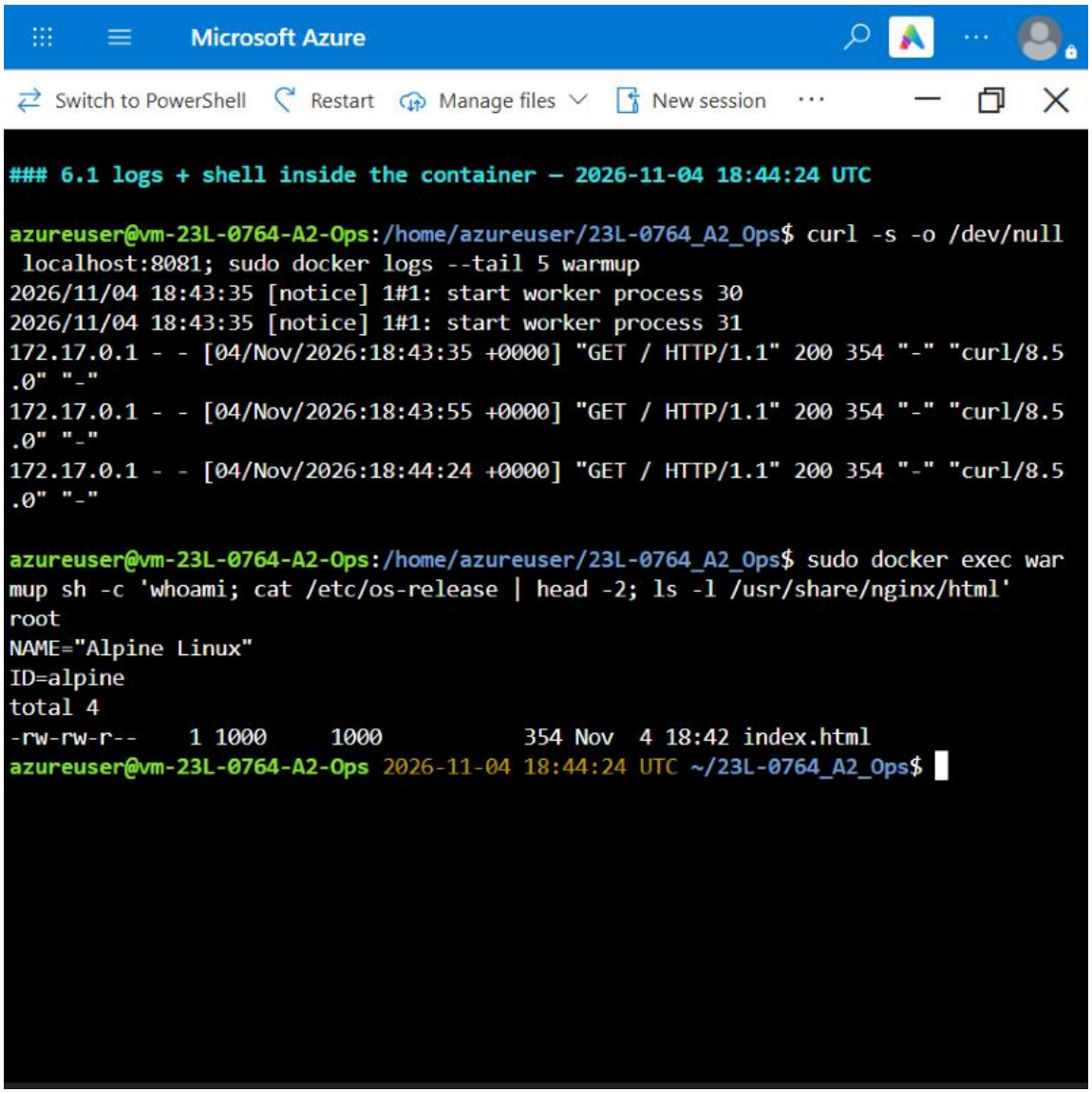


Figure 6.2: Logs and a shell inside the container.

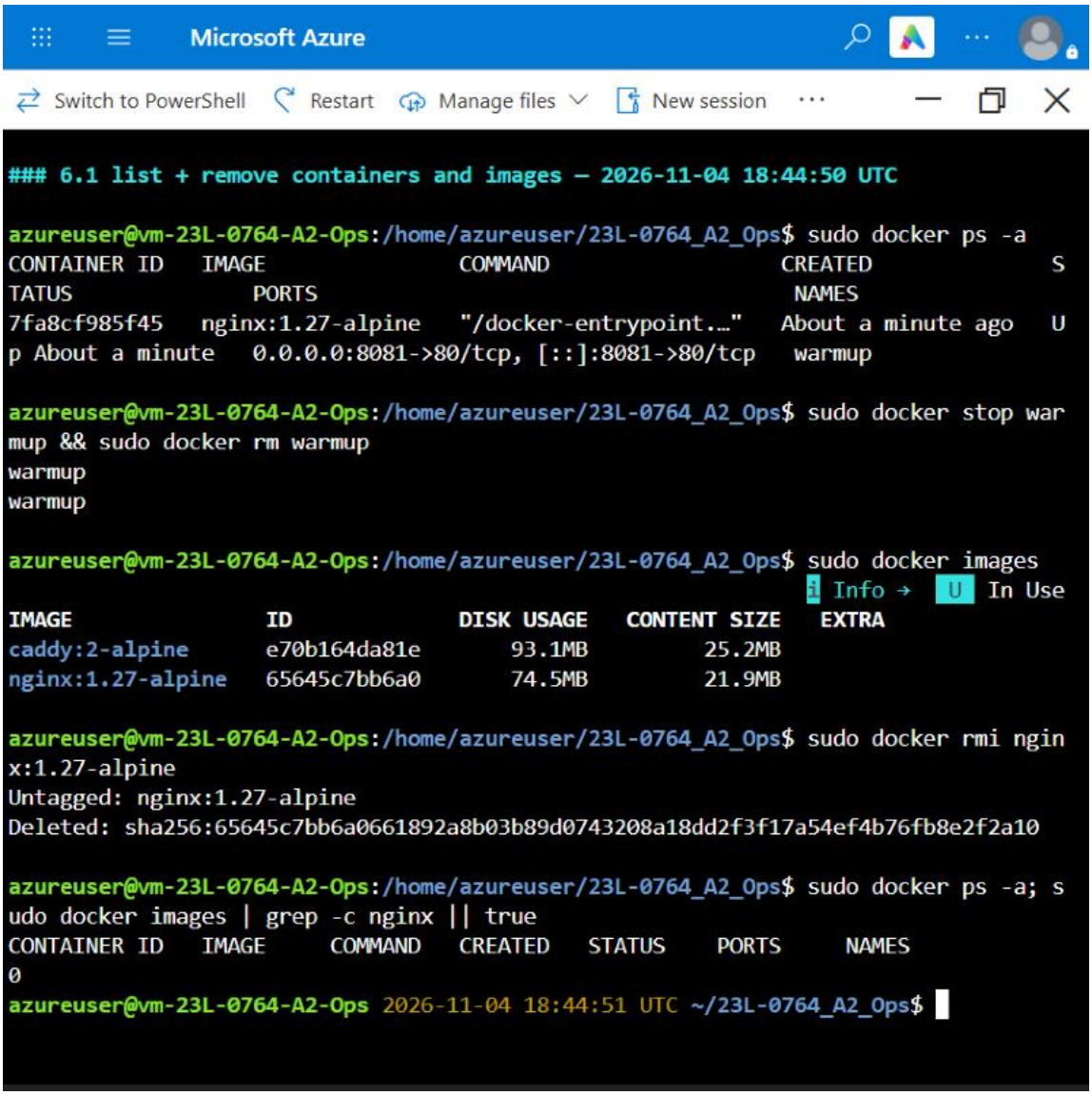


Figure 6.3: Listing and removing containers and images.

Image a read-only template built from a Dockerfile: a file system plus metadata (command, environment, exposed ports). An image does not run by itself.

Container a running or stopped instance of an image. It is an isolated process with its own view of the file system, its own network and process space, and a thin writable layer that is lost when the container is removed.

Layer each Dockerfile instruction that changes the file system creates one read-only layer, and an image is a stack of them. Layers are cached and shared between images, so an instruction that has not changed is not rebuilt.

Volume storage kept outside the container's writable layer, so it survives when the container is removed. A bind mount maps a folder of the host instead.

Port mapping forwards a port on the host to a port inside the container's private network (`host:container`). Without it the service cannot be reached from outside.

6.2 Dockerfile for the web app

The `Dockerfile` has its instructions in this order:

1. `FROM python:3.11-slim`, a slim Debian without compilers or docs (about 130 MB, compared with about 1 GB for `python:3.11`). All my dependencies come as wheels, so I do not need a compiler.
2. `ENV` settings: no `.pyc` files, unbuffered output so logs appear in `docker logs` straight away, and `PIP_NO_CACHE_DIR=1` so the pip cache does not end up inside a layer.
3. `COPY requirements-*.txt` and then `RUN pip install`, before any source code. Docker reuses a cached layer as long as the instruction and the files it copies have not changed. The requirements rarely change and the code changes all the time, so with this order editing a template or a route reuses the cached TensorFlow layer (about 1 GB) and the rebuild takes seconds. If the source were copied first, every edit would invalidate the cache and reinstall everything.
4. `useradd vg` (uid 10001) and later `USER vg`, so the app does not run as root. A compromised process then cannot change the image's files or install software.
5. `COPY src app` copies the code, owned by `vg`. The model is not copied here; it is mounted at run time (Section 6.3).
6. `HEALTHCHECK`: every 30 s a small Python call requests `/health` (the slim image has no `curl`), and Docker marks the container unhealthy after 3 failures.
7. `CMD gunicorn ...` starts the production WSGI server. The exec form makes gunicorn PID 1, so it receives `SIGTERM` and shuts down cleanly.

`.dockerignore` leaves `.git`, `.env`, `data/`, `mlruns/`, `outputs/`, the notebooks, the tests and the report out of the build context. The upload to the Docker daemon stays small, and secrets or the dataset cannot end up in a layer. Secrets are only passed in at run time, as environment variables from the untracked `.env`, and `docker history` of the image shows none of them.

I install `tensorflow-cpu==2.20.0`. The default `tensorflow` wheel on Linux comes with the CUDA and cuDNN libraries (several GB), which a CPU-only server never uses, and a single MobileNetV2 prediction only takes tens of milliseconds on a CPU anyway. A build argument, `WITH_TF=false`, builds the web image without TensorFlow at all for `INFERENCE_MODE=remote`, where the model runs in the cloud API.

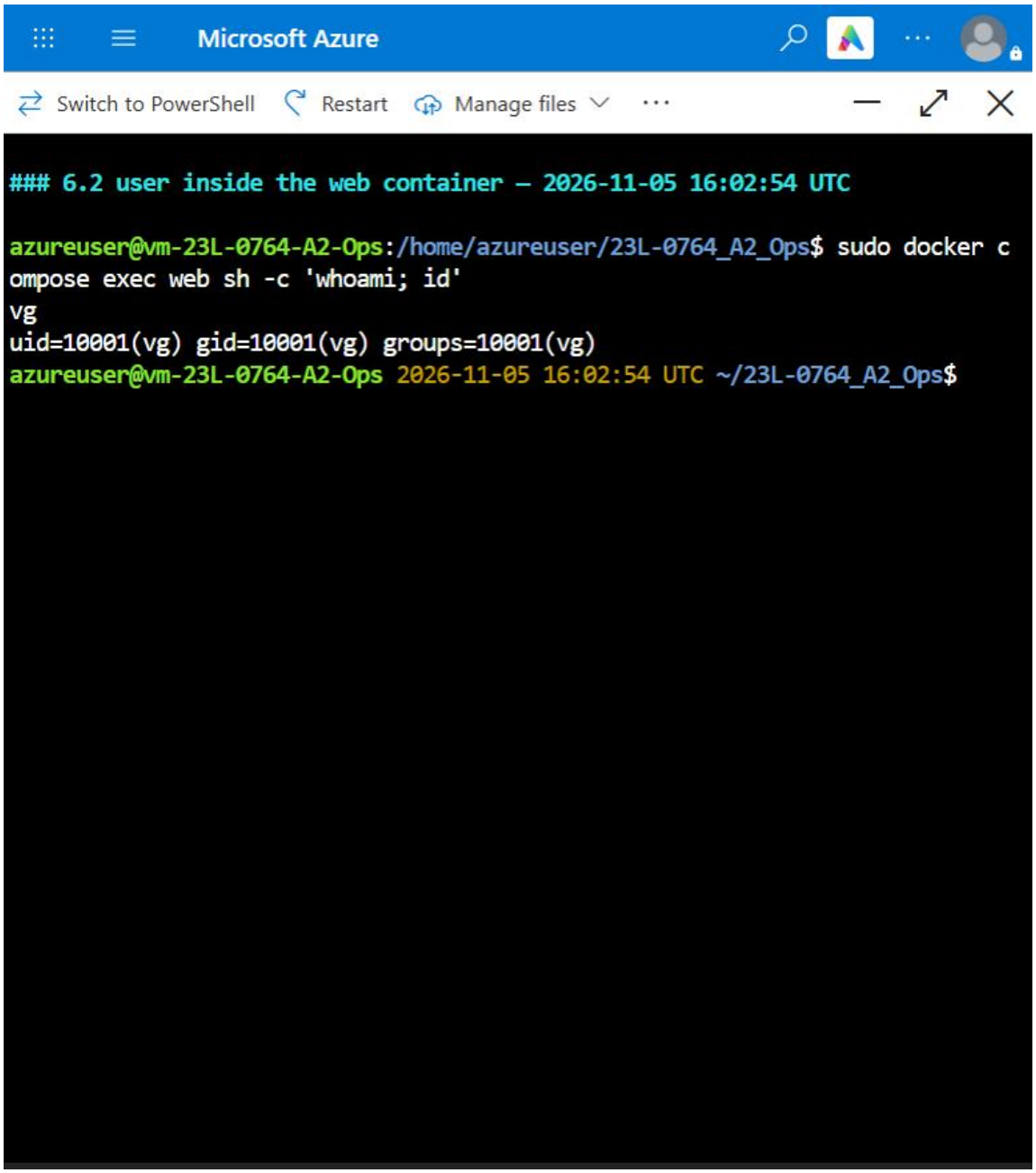


Figure 6.4: The process inside the container runs as the unprivileged user `vg` (uid 10001).

My first container started gunicorn with `-preload`, which loads the app in the master process before forking the workers so that start-up errors show up immediately. The health checks passed, but the first prediction hung until gunicorn's timeout killed the worker. TensorFlow starts its thread pools when it initialises, and `fork()` only copies the thread that calls it, so the worker inherited locks held by threads that no longer existed and deadlocked. Without `-preload` the worker loads the model itself after the fork, and it is still loaded only once per process.

6.3 Compose stack

`docker-compose.yml` works as follows:

- There are two services, `web` and `db` (`postgres:16-alpine`), on a user network called `backend`. Compose's internal DNS resolves service names, so the web app connects to

`postgresql+psycopg2://...@db:5432/visionguard`, where `db` is the service name and not an IP address.

- A named volume, `pgdata`, holds the database files, so the history survives `docker compose down` and image upgrades. Only `down -v` deletes it.
- `db` has a health check (`pg_isready`) and `web` declares `depends_on: db: condition: service_healthy`, so the app only starts once PostgreSQL accepts connections. A plain `depends_on` only waits for the container to start, which is too early. The app also retries its first connection.
- The database port is not published. `db` has no `ports:` entry, so only containers on `backend` can reach it, and it is invisible from my machine and the network.
- Passwords come from the untracked `.env`. `${POSTGRES_PASSWORD:?}` makes Compose refuse to start if the variable is missing, and `.env.example` is committed with placeholders.

The model reaches the web container through a mount: `./models` is mounted read-only at `/app/models`. Promoting a new champion then only takes `python scripts/export_champion.py` and `docker compose restart web`, with no rebuild, and the 9.6 MB model is not copied into every version of the image. Because of `:ro` the app cannot change it. For the cloud API I did the opposite and built the model into the image (Chapter 7), because that image is shipped to another machine and should be self-contained and never change after it is built.

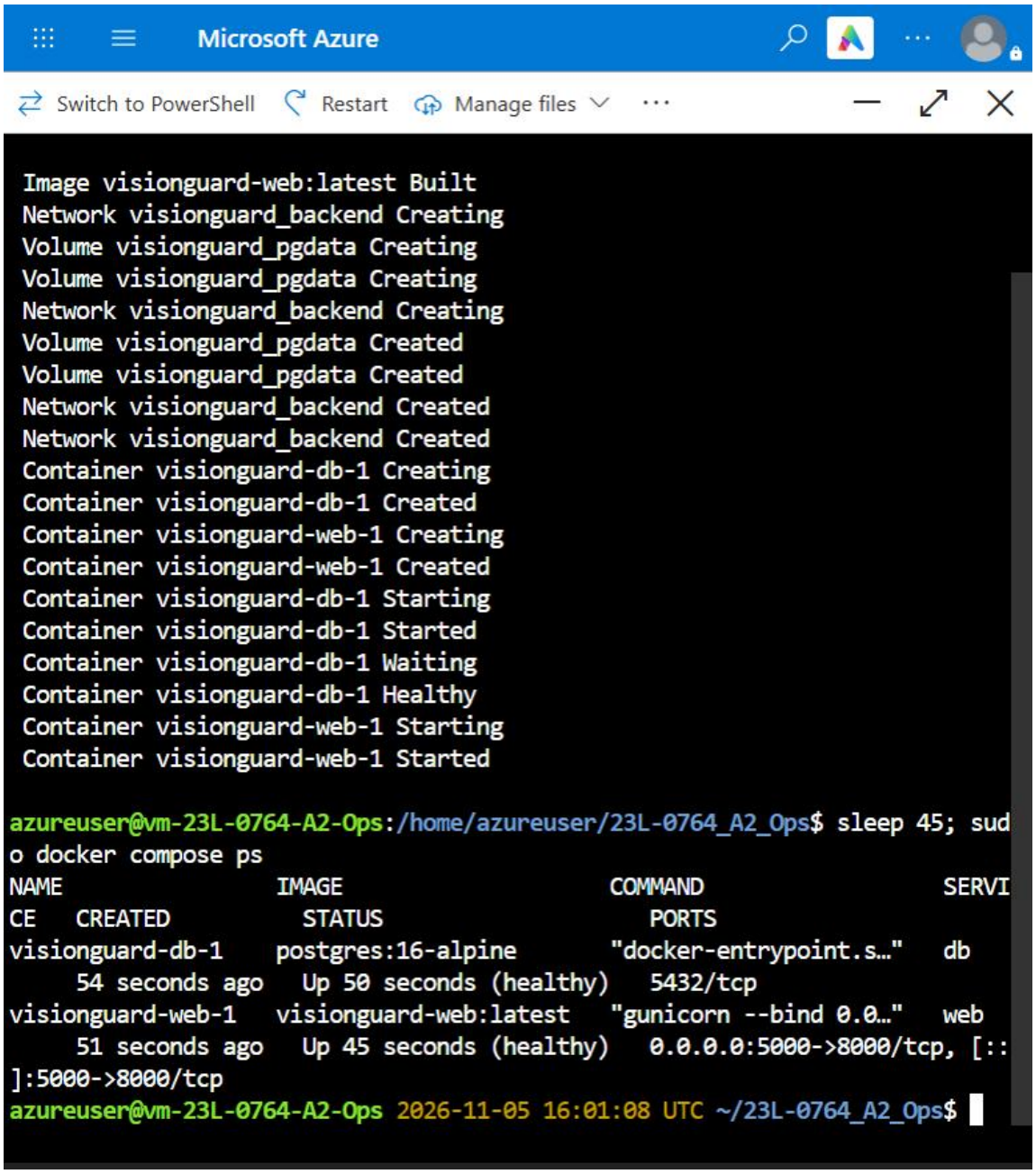


Figure 6.5: `docker compose ps`: both services healthy.

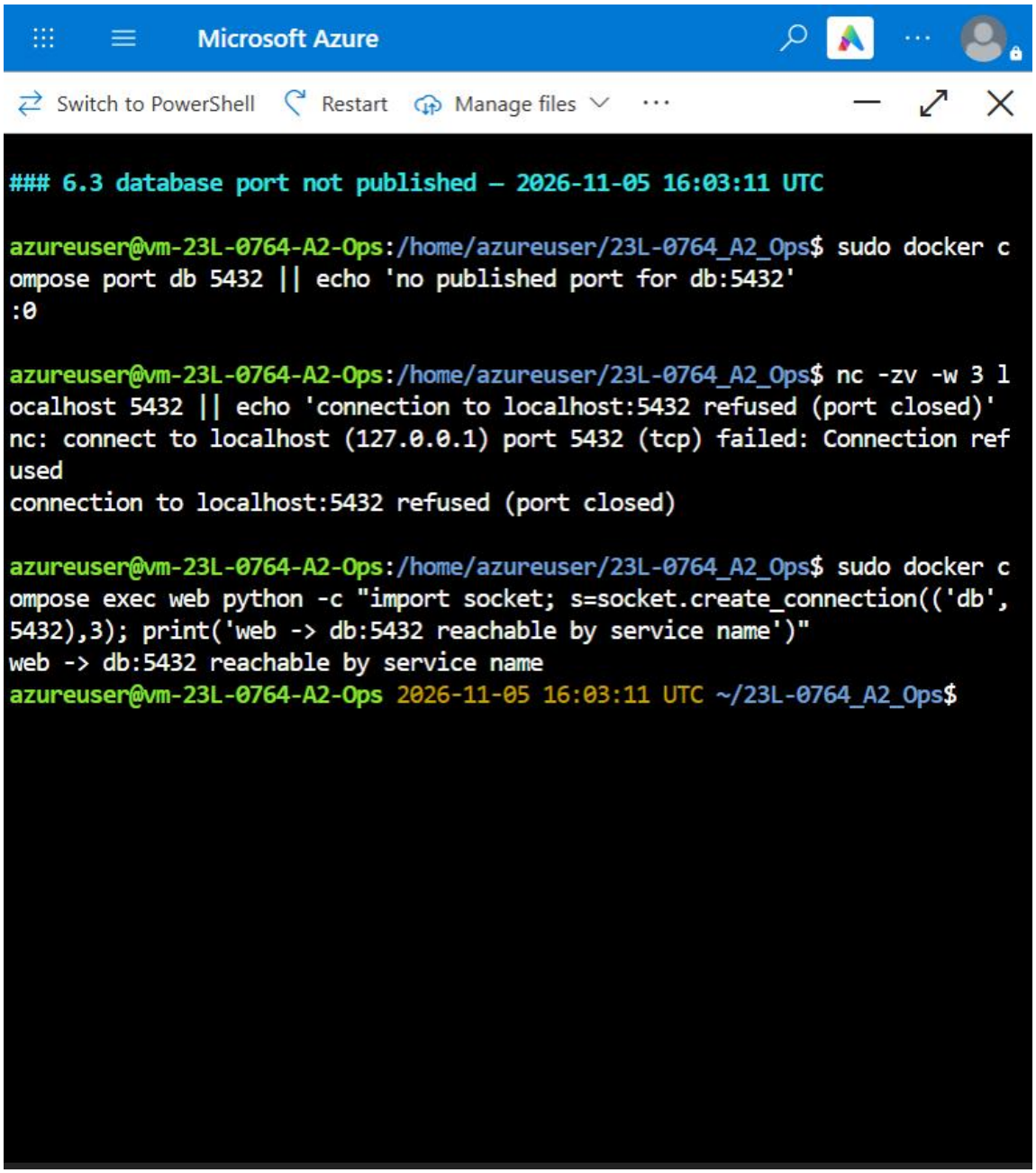


Figure 6.6: The database port is closed from outside (`docker compose port db 5432` is empty; a connection to localhost:5432 is refused).

6.4 Image analysis

Image	Size
`visionguard-web:naive` (`docker/Dockerfile.naive`)	6.06 GB (1.87 GB compressed content)
`visionguard-web:latest` (optimised)	2.16 GB (422 MB)
`visionguard-web:remote` (`WITH_TF=false`)	385 MB (90.8 MB)

Table 6.1: Image sizes from `docker images`.

The naive image uses the full `python:3.11` base and the GPU `tensorflow` wheel with its CUDA libraries, keeps the pip cache and runs as root. `docker history` shows that the biggest layer by far is the `pip install` of TensorFlow and the serving libraries (1.59 GB), followed by the base image. My own code is only about 150 KB (`app` 82 KB, `src` 66 KB), and the user and metadata layers are a few KB each. For the optimised image I switched to the slim base and `tensorflow-cpu`, turned off the pip cache and added the ignore file.

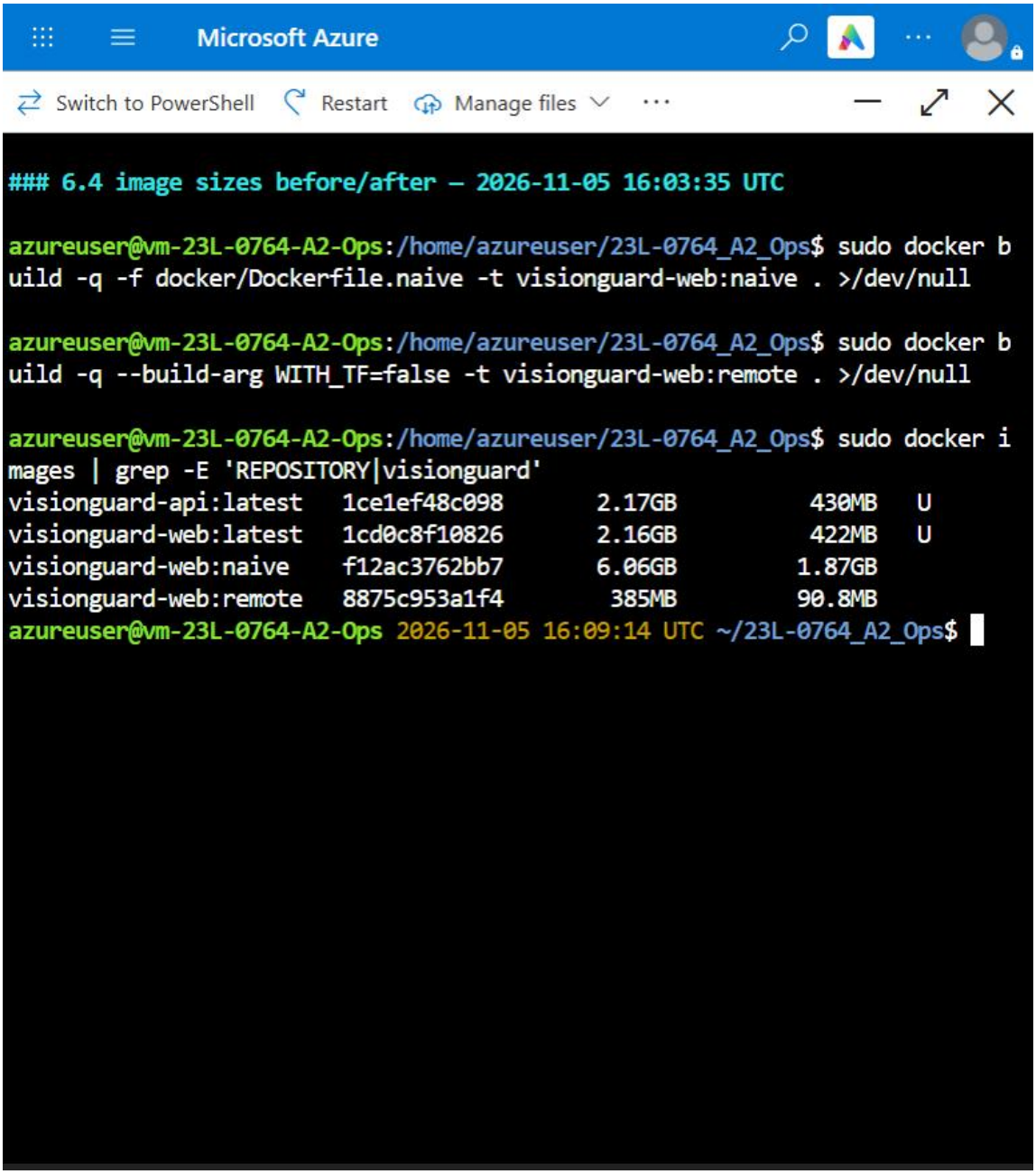


Figure 6.7: `docker images`: before and after.

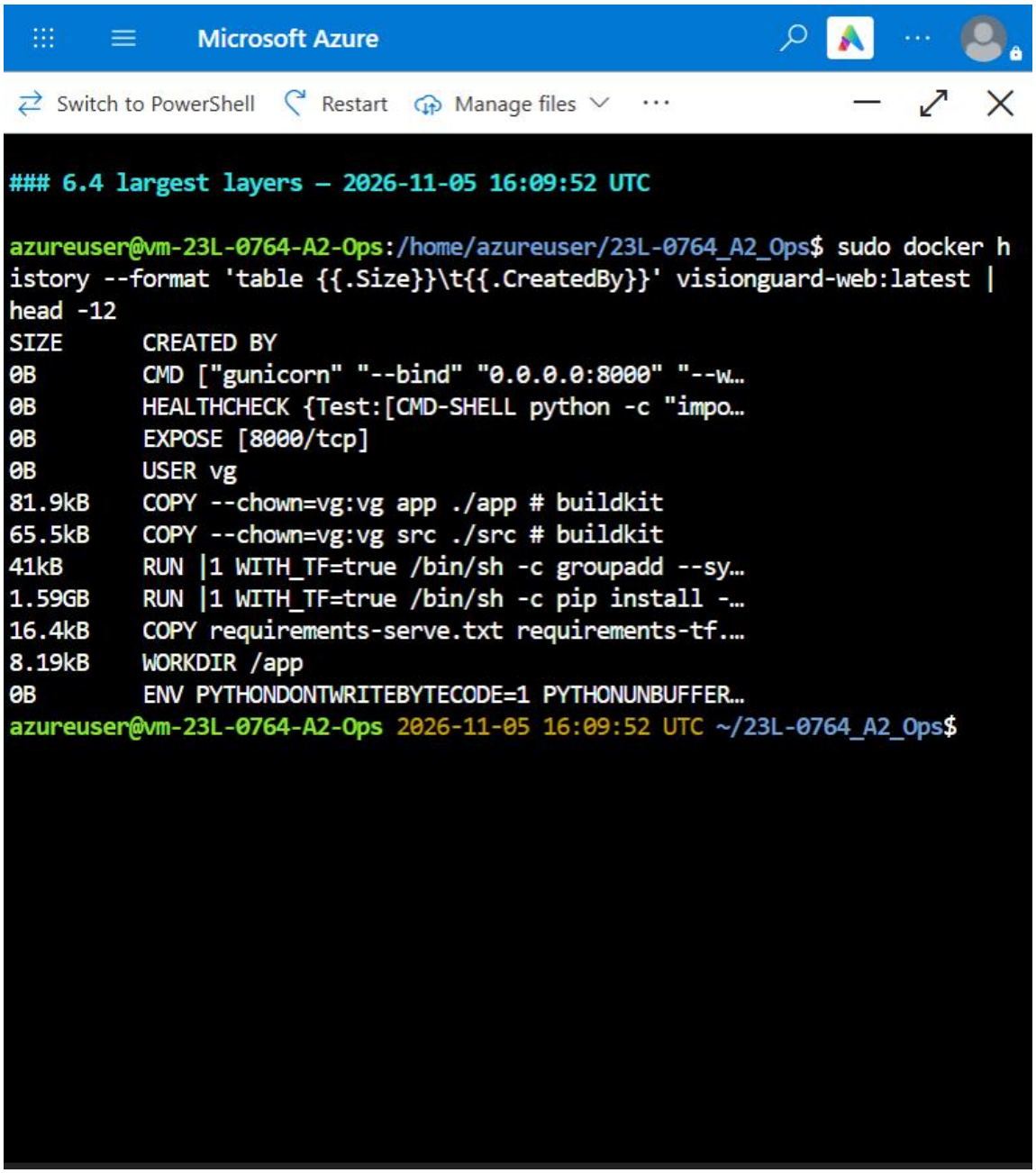


Figure 6.8: `docker history`: the TensorFlow layer dominates.

Together these changes took the web image from 6.06 GB down to 2.16 GB, a 64% reduction. Almost all of that comes from the CUDA libraries that the GPU wheel pulls in. Building without TensorFlow for `INFERENCE_MODE=remote`, where the model runs in the cloud API, brings it down to 385 MB, 94% smaller than the naive image.

6.5 Operations and troubleshooting

For day-to-day operations I used `docker compose stop web`, `start`, `restart db`, `docker compose logs -tail 30 web`, `docker compose exec web sh` and `docker compose exec db psql -U visionguard`.

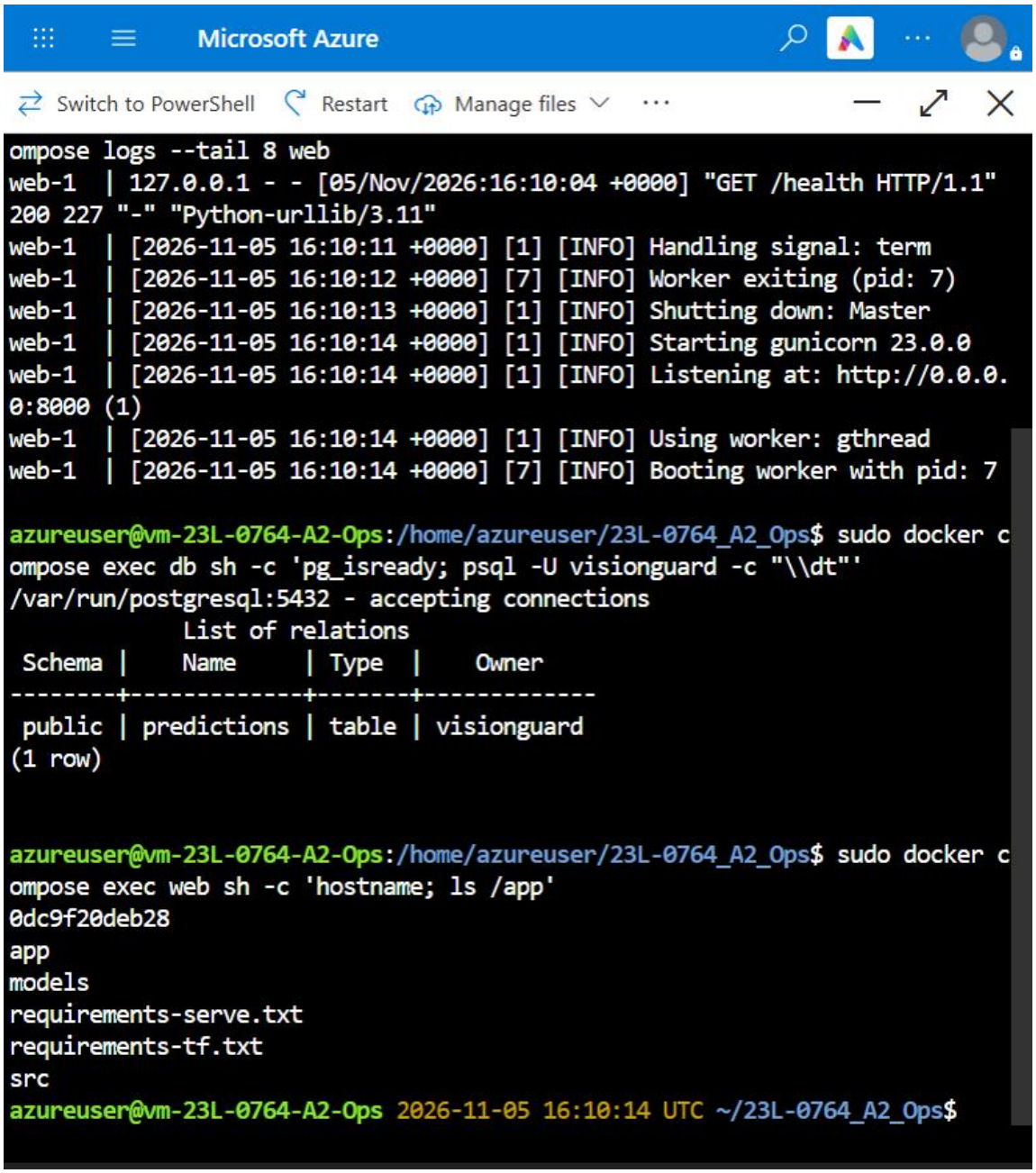


Figure 6.9: Start/stop/restart, logs and shells in both services.

To practise troubleshooting I broke the stack on purpose. I changed `POSTGRES_PASSWORD` in `.env` and recreated only the web container, and `docker compose ps` showed it restarting. Using only the logs, `docker compose logs web` showed my retry loop (`database not ready (OperationalError), retry n/10`) followed by `FATAL: password authentication failed for user "visionguard"`. So the network and the database were working, since the database answered, and the credentials were wrong. One detail surprised me: the PostgreSQL image only applies `POSTGRES_PASSWORD` when the volume is initialised for the first time, so the database still had the old password and only the client side had changed. To fix it I restored the value in `.env` and ran `docker compose up -d -force-recreate web`. The service was healthy again and the history was still there.

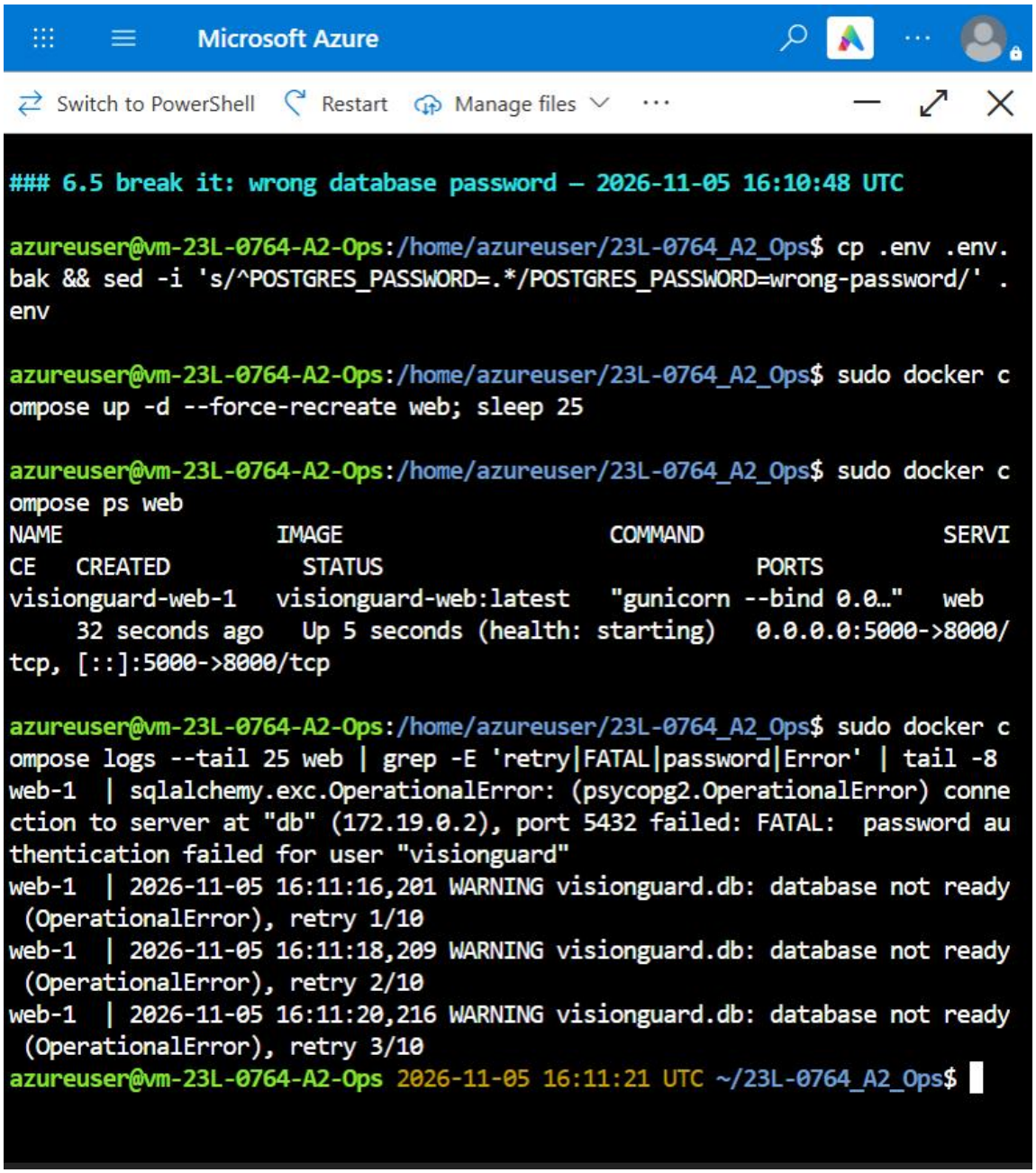


Figure 6.10: Log excerpt of the failure.

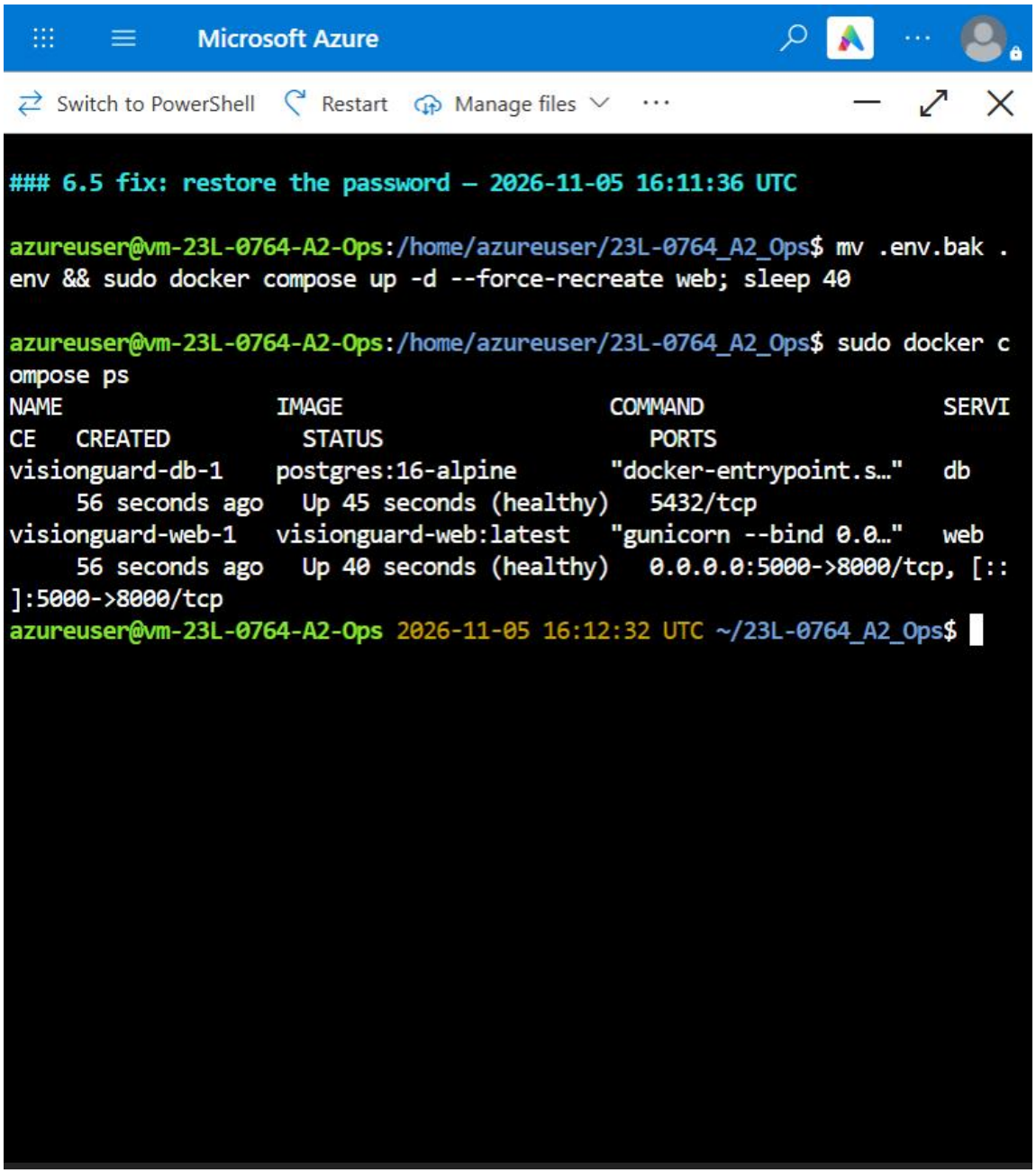


Figure 6.11: After the fix: both services healthy.

6.6 Tool lab: GitHub Actions (CI)

For the toy part I used a separate orphan branch of the repository, `ci-toy`, which shares no history or files with `main` (its content is `labs/ci_toy/`). A workflow runs on every push with a matrix of Python 3.11 and 3.12, so the same job runs twice in parallel, once per version. I changed an assertion to a wrong value, which gave a red run, and then fixed it, which gave a green one.

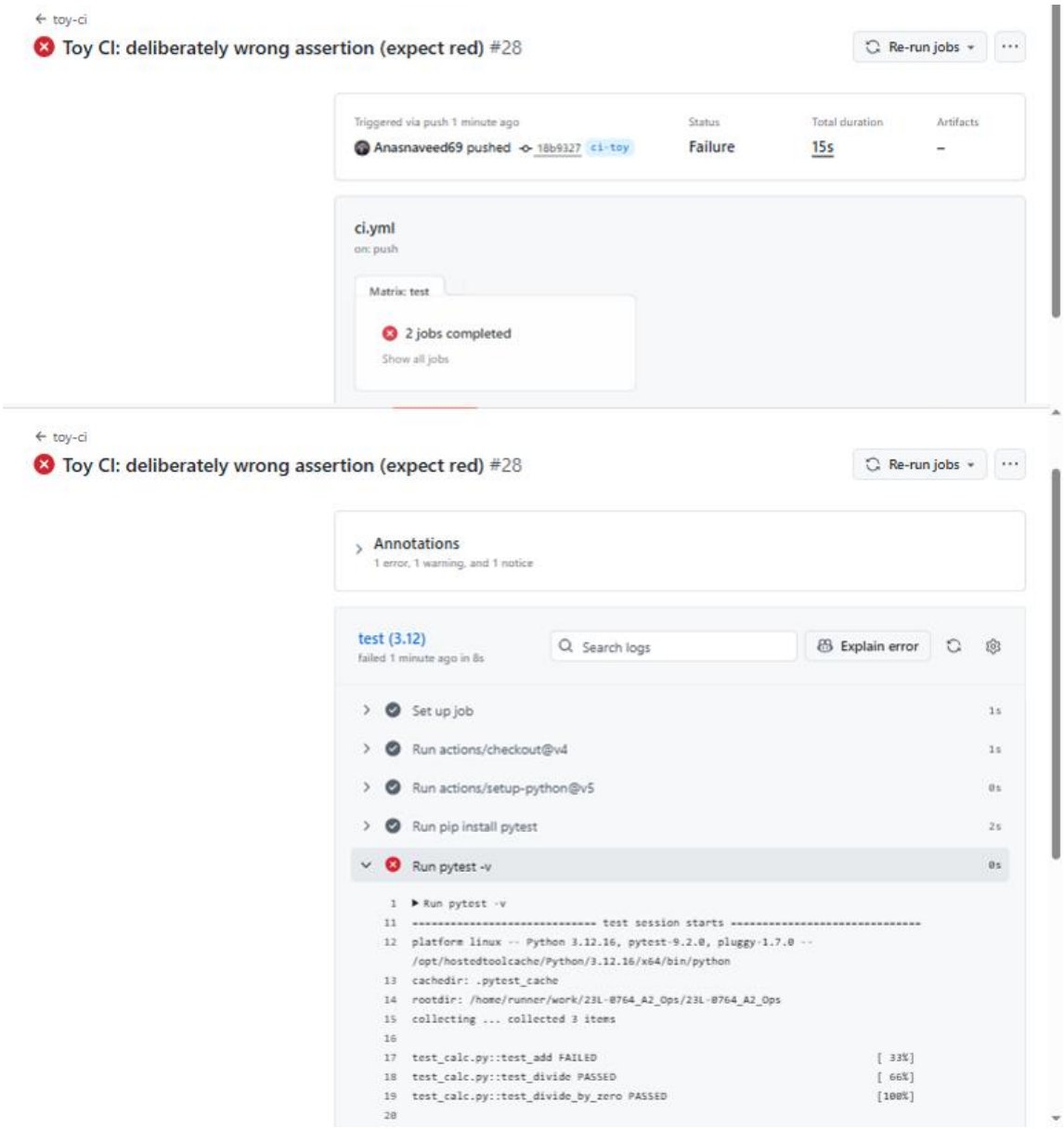


Figure 6.12: Toy workflow: failing test, red run.

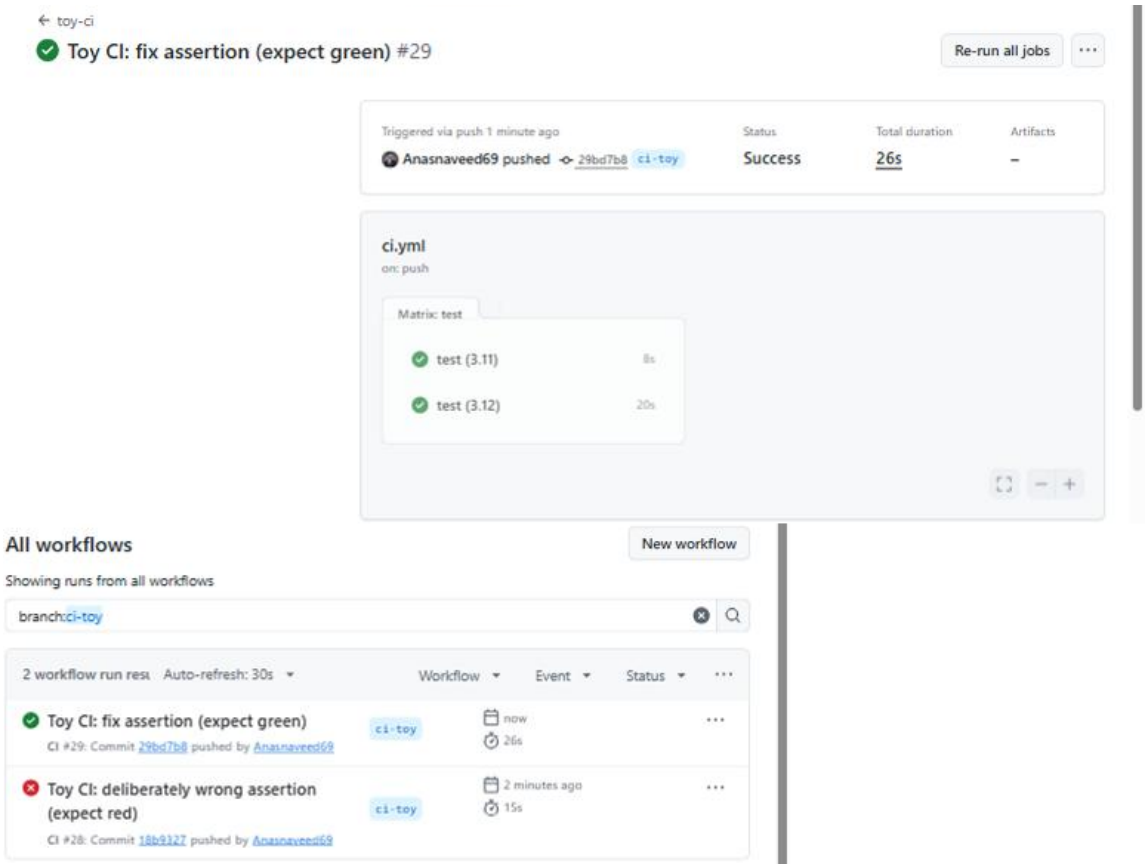


Figure 6.13: Toy workflow: fixed, green run on both Python versions.

For VisionGuard, `.github/workflows/ci.yml` runs three jobs on every push:

1. Lint and tests on Python 3.11 and 3.12: `ruff check .` and then `pytest`. The 27 tests cover upload validation, all routes and the DB trail with a fake predictor, the inference module, and the API's key check with a tiny model.
2. Tests against PostgreSQL, using a `postgres:16` service container, so the trail is tested on the same database engine as in production.
3. A Docker job that builds the web and API images (with layer caching), starts them, waits for `/health`, and checks that the process user is `vg` and that the API returns 401 without a key. It uses the committed champion model, and only generates a tiny dummy model if that file is missing.

The badge in `README.md` shows the status of the latest run.

Secrets such as a DagsHub token or a deploy key belong in the repository's Settings, under Secrets and variables, then Actions. GitHub stores them encrypted, the workflow refers to them as `${{ secrets.NAME }}`, and GitHub injects them at run time and masks them in the logs. If I wrote them into the workflow file, anyone who can read the repository would see them, they would stay in the Git history forever, and every fork would copy them.

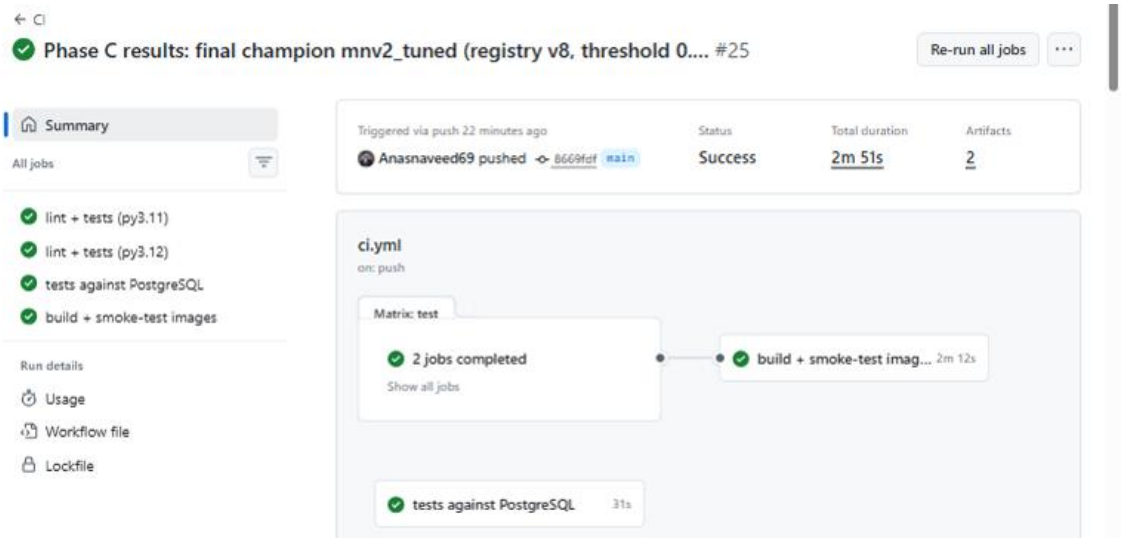


Figure 6.14: VisionGuard CI: all jobs green.

Figure 6.15: CI status badge in the README.

7. API Deployment

As the assignment asks, the model goes to the cloud as an HTTP API, and the Flask web app runs on my machine and calls it. I went with pathway C: a single Ubuntu VM on Azure running a small Compose stack. I used Azure for Students, which gives $100 of credit with a university e-mail and does not need a credit card.

The API is live at `https://20-219-10-43.sslip.io`.

Compose / local element	Azure equivalent used
Host running Docker	VM `vm-23L-0764-A2-Ops` in resource group `rg-23L-0764-A2-Ops`, region Central India, Ubuntu 24.04 x64, size `Standard_B2s_v2` (2 vCPU, 8 GB)
Published ports 80/443	Network Security Group inbound rules and `ufw` on the VM
`api` service	`api/Dockerfile`: gunicorn, Flask API and TensorFlow-CPU, with the model built in
HTTPS	Caddy reverse proxy with an automatic Let's Encrypt certificate for `<ip>.sslip.io`
Secrets (`.env`)	`deploy/.env`, created on the VM only (`chmod 600`) and never committed
Database	stays with the web app on my machine (Task 8 design)

Table 7.1: Mapping the stack to Azure.

Before deploying I looked into a few things. The student credit lasts 12 months, and a 2-vCPU burstable VM costs a few cents an hour, so a week of uptime uses only a small part of it. I stop (deallocate) the VM when I do not need it. The subscription also has an Azure Policy called "Allowed resource deployment regions". Reading it with `az policy assignment list` showed that only Malaysia West, Central India, Austria East, UAE North and Southeast Asia are allowed.

For the size, TensorFlow plus the model needs roughly 1 GB of RAM when it starts. A 1 GB VM would get killed by the out-of-memory killer, so I needed a 2-vCPU size with at least 4 GB, and I added 2 GB of swap on top. I chose an x86-64 image so the same `tensorflow-cpu` wheel as on my laptop and in CI could be used; Arm would need different wheels.

There are two firewalls to get through. The Network Security Group at the cloud level has to allow ports 80 and 443, and so does the VM's own firewall, `ufw`. If either one blocks a port, the traffic does not get through.

The prediction endpoint needs a secret `X-API-Key` header. The server compares it in constant time with `hmac.compare_digest`, so the response time does not give away how much of a guessed key was right. Because of HTTPS the key never travels in clear text. Request bodies are capped at 6 MB by Caddy and at 5 MB by the app. `/health` is public, but it only shows the model name and version.

Step-by-step deployment

I did everything from Azure Cloud Shell, a terminal in the browser that is already logged in to my subscription, using two scripts from the repository so the deployment can be repeated: `deploy/azure_cloudshell.sh` for the cloud resources and `deploy/vm_bootstrap.sh` for the setup inside the VM.

1. Register the resource providers. On a brand-new subscription `Microsoft.Compute`, `Microsoft.Network` and `Microsoft.Storage` are `NotRegistered`, and every VM deployment fails. `az provider register -n Microsoft.Compute -wait` (and the same for Network and Storage) turns them on.

2. Create the VM: `az vm create -g rg-23L-0764-A2-Ops -n vm-23L-0764-A2-Ops -image Ubuntu2404 -size Standard_B2s_v2 -admin-username azureuser -generate-ssh-keys -public-ip-sku Standard -nsg-rule SSH`. This creates the VM, its disk, a virtual network, a public IP and a Network Security Group (NSG) that only allows SSH. `-generate-ssh-keys` creates a key pair, so there is no password login at all.
3. Open the cloud firewall. `az vm open-port -port 80 -priority 1010` and `-port 443 -priority 1020` add NSG inbound rules for HTTP, which the certificate challenge needs, and for HTTPS.

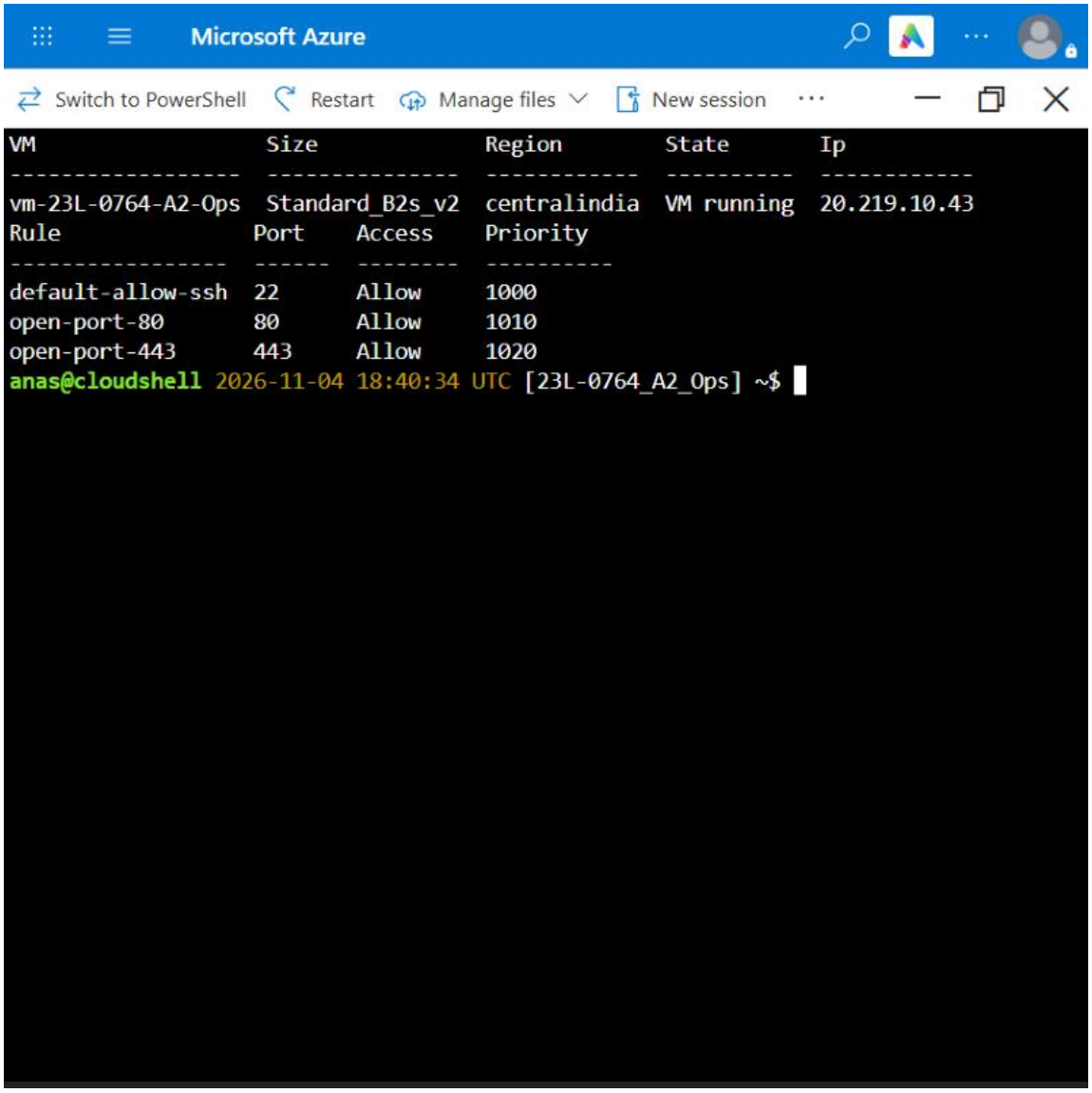


Figure 7.1: The VM (size, region, state, public IP) and the NSG inbound rules 22/80/443, listed from Cloud Shell.

4. Prepare the VM over SSH with `deploy/vm_bootstrap.sh`. It installs Docker Engine and the compose plugin, turns on `ufw` with only SSH, 80 and 443 allowed (the second firewall), creates 2 GB of swap and clones the repository, which includes the exported champion model (Chapter 3). It then writes `deploy/.env` with a random 256-bit `API_KEY` (`secrets.token_urlsafe(32)`) and `API_DOMAIN=<ip-with-dashes>.sslip.io`. The free sslip.io DNS service resolves that name to the IP, so Let's Encrypt can issue a certificate without me buying a domain.

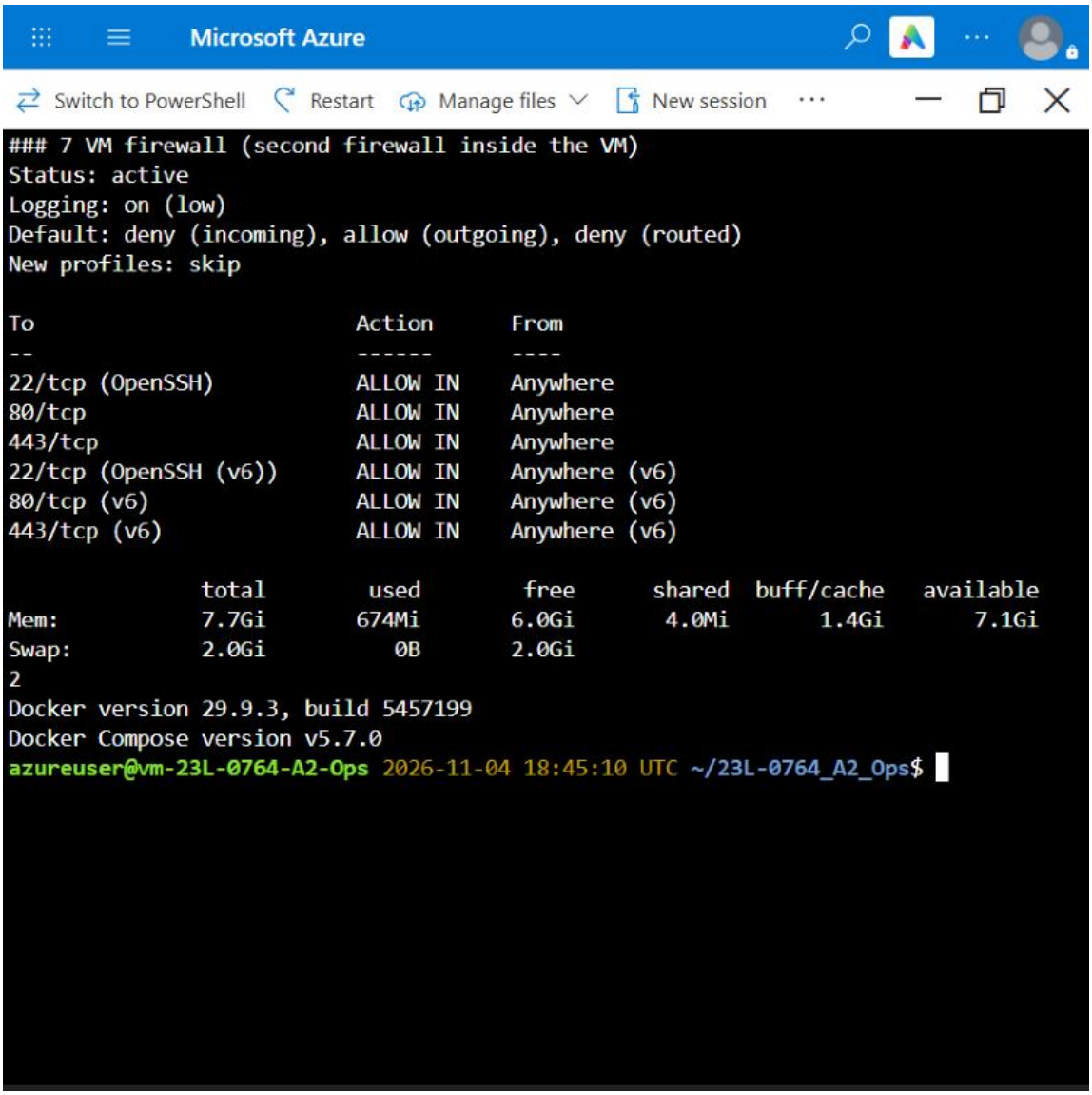


Figure 7.2: The VM's own firewall (`sudo ufw status verbose`), memory including swap, Docker versions.

5. Start the stack with `docker compose -env-file deploy/.env -f deploy/docker-compose.cloud.yml up -d -build`. The API image is built with the model inside. Caddy only starts once the API is healthy, then gets the certificate and forwards `https://API_DOMAIN` to `api:8080`. Caddy is the only service that publishes ports, so the API container can only be reached on the internal network.

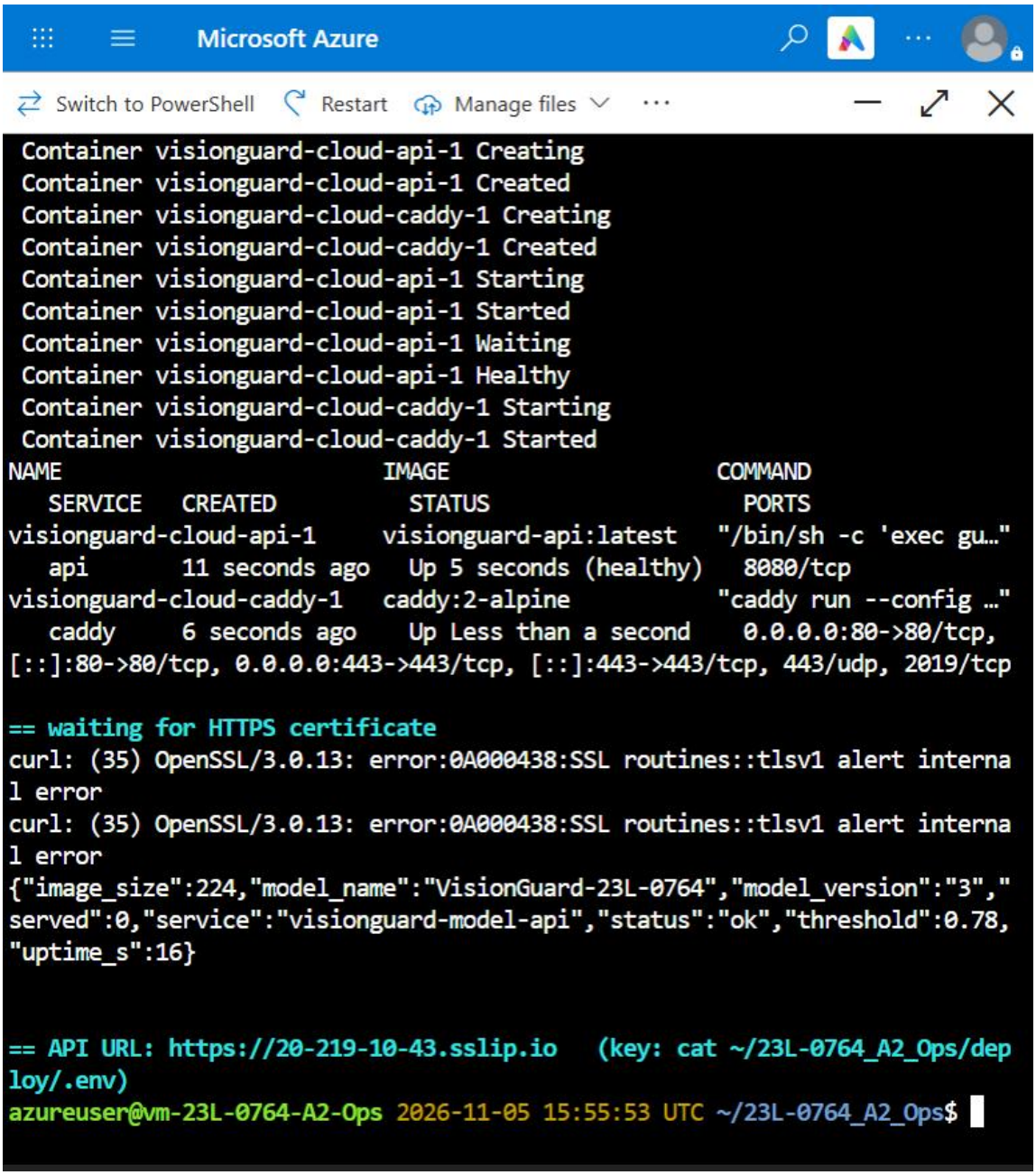


Figure 7.3: Cloud stack running: API healthy, Caddy up, HTTPS health check answered.

6. Check it from another network by opening `/health` there.

Figure 7.4: `https://20-219-10-43.sslip.io/health` opened from outside the VM's network.

Three things went wrong along the way. The first `az vm create` failed in every region with an unreadable Python traceback, *"The content for this response was already consumed"*, which is an Azure CLI bug that hides the real error. When I checked the provider state, the cause turned out to be the unregistered resource providers, and registering them fixed it. Next, `Standard_B2s` failed with `SkuNotAvailable` (*"currently not available in location CentralIndia"*) because of capacity limits, and the same happened in the other allowed regions. I made the script loop over similar small sizes, and `Standard_B2s_v2` (2 vCPU, 8 GB) was available in Central India. Finally, the first API image build on the VM failed with `"/models/champion.keras": not found`, because I started it before the champion had been trained and exported. Once the model was committed, the build went through. After the final training round I pulled the new commit on the VM and rebuilt the API, so it now serves version 8.

Testing with Postman

The collection `deploy/VisionGuard.postman_collection.json` has two variables, `base_url` and `api_key`, and five requests with test scripts that check the status code and the response fields:

1. `GET /health` should return 200 with the model name and version.
2. `POST /v1/predict` with an AI image (form-data field `image`) should return 200 with the label, confidence and latency.
3. The same with a real photo.
4. The same without the `X-API-Key` header should return 401.
5. A non-image file should return 415. I used a `.md` file.

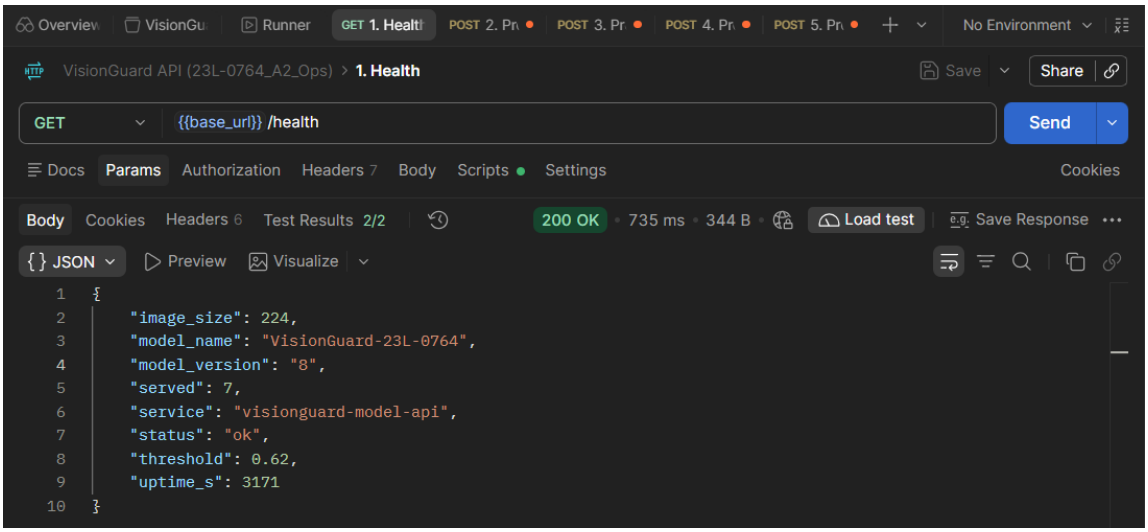


Figure 7.5: Postman: health.

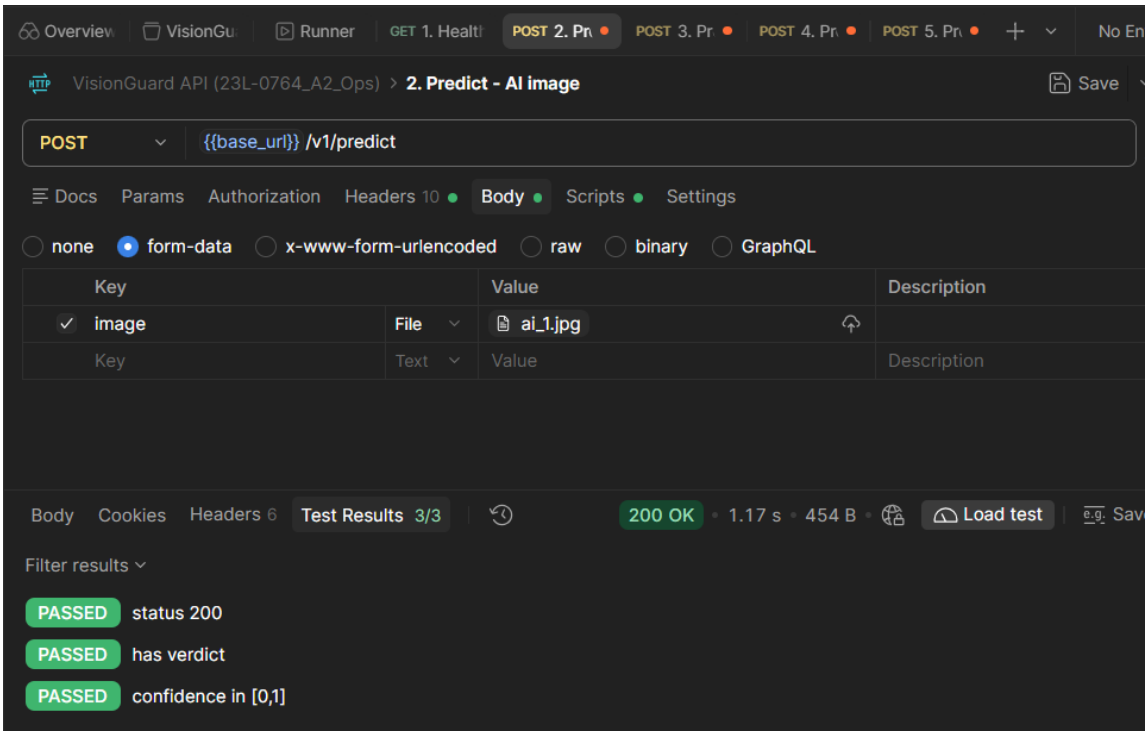


Figure 7.6: Postman: AI image (`ai_1.jpg`) returns 200 and all three test assertions pass.

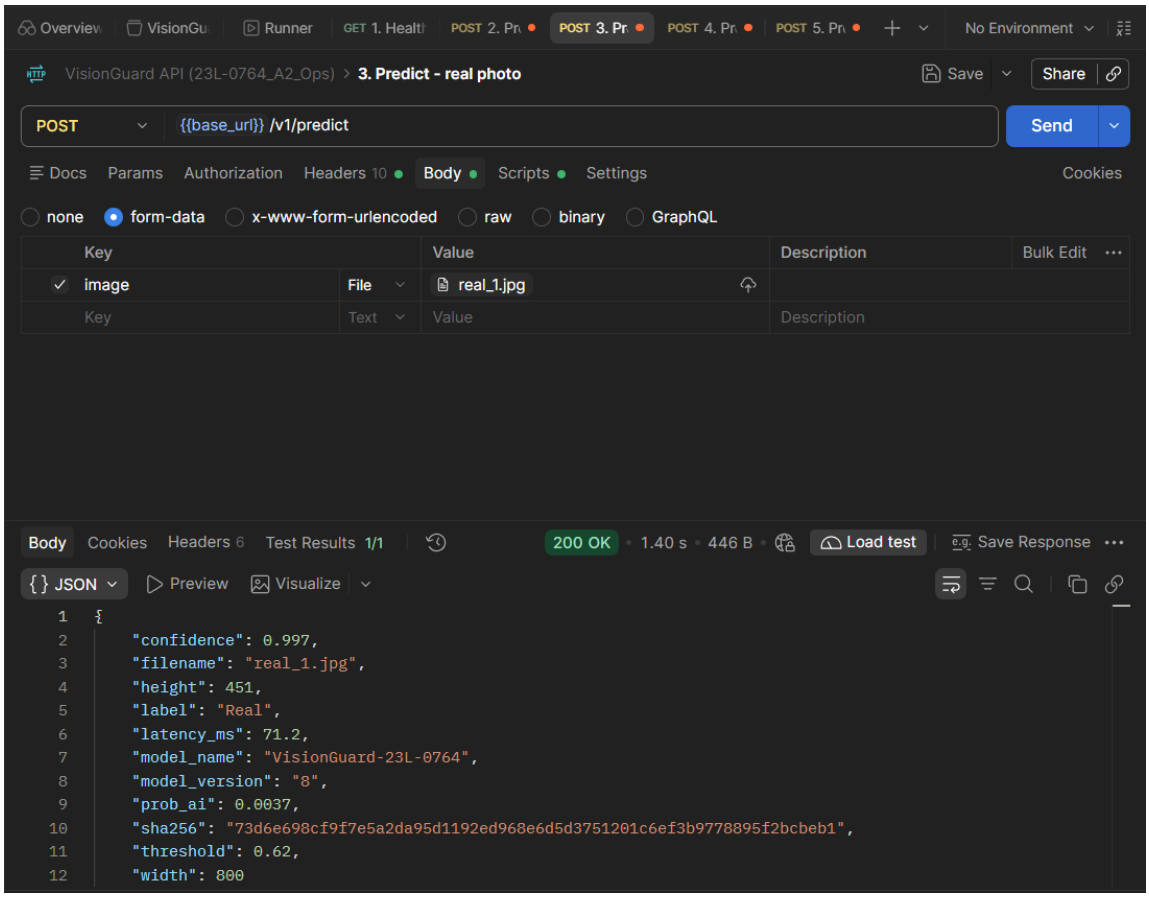


Figure 7.7: Postman: real photo and the verdict JSON.

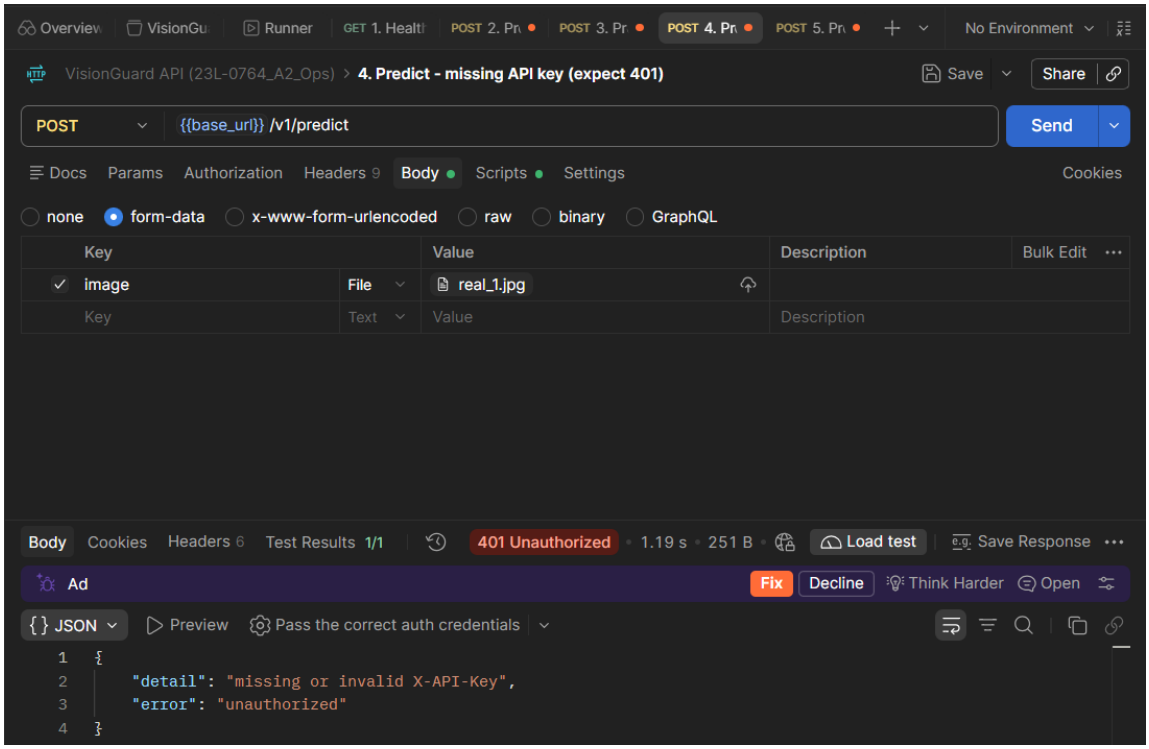


Figure 7.8: Postman: missing API key rejected (401).

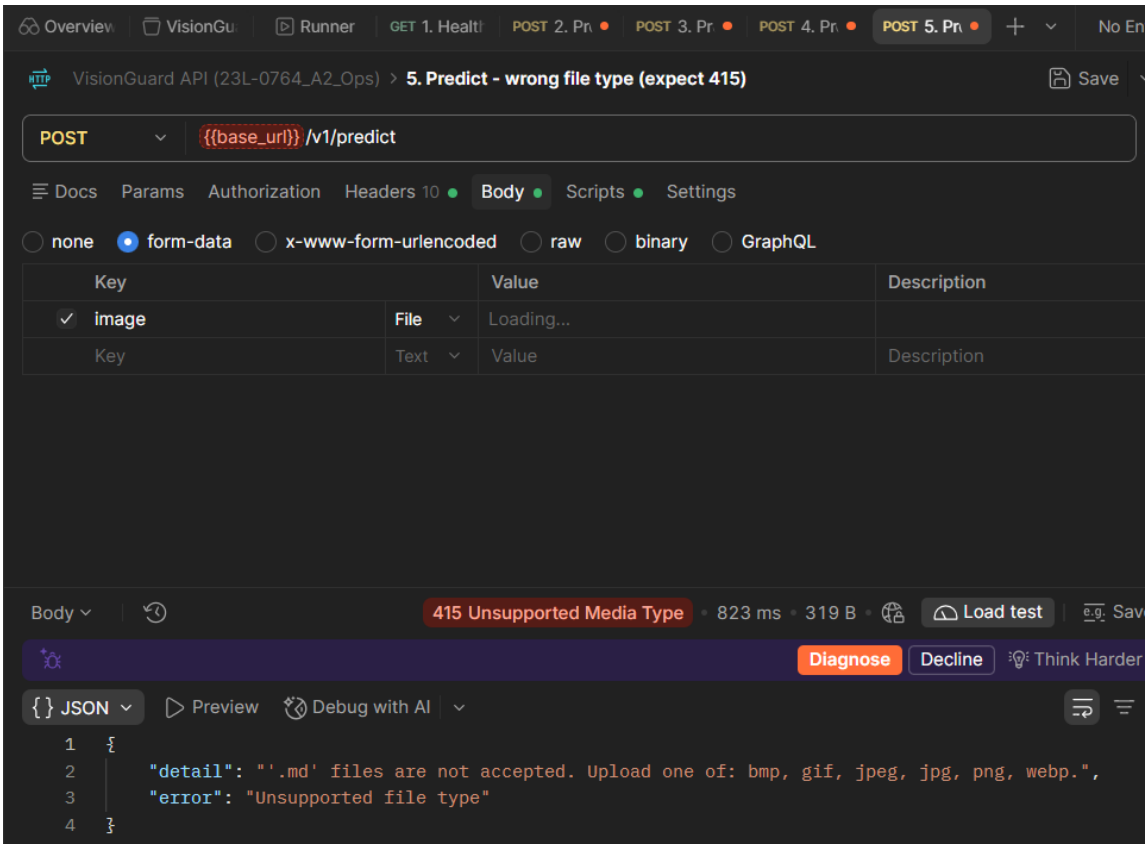


Figure 7.9: Postman: wrong file type (`.md`) rejected (415).

Here is the response for `real_1.jpg` from Postman:


```
{"confidence": 0.997, "filename": "real_1.jpg", "height": 451, "label": "Real",
 "latency_ms": 71.2, "model_name": "VisionGuard-23L-0764", "model_version": "8",
 "prob_ai": 0.0037, "sha256": "73d6e698...f2bcbeb1", "threshold": 0.62, "width": 800}
```


The model itself takes about 70 ms per image on the 2-vCPU VM (`latency_ms` was between 69 and 77 ms for every request in the API log). The round trip from Lahore is much longer and depends on the client: 0.2 to 0.4 s from the Flask app on my laptop, which re-uses its HTTPS connection, and 0.7 to 1.4 s from Postman, which opens a new TLS connection for each request and uploads through its agent. Nearly all of that time is network.

8. Integrating Flask and the Cloud API

The web app has two back-ends behind the same interface (`app/predictors.py`). `LocalPredictor` wraps the in-process `Predictor`. `RemotePredictor` sends the validated image bytes as multipart form data to `MODEL_API_URL/v1/predict` over HTTPS with the `X-API-Key` header, through one re-used `requests.Session` (so connections are pooled) and a 30 s timeout. Switching between them is one environment variable, `INFERENCE_MODE=remote`, and the routes, templates and database code stay the same. In remote mode the web image can even be built without TensorFlow (`WITH_TF=false`), since the model only lives in the cloud.

For each upload, the browser posts to `/predict`, and the web app first validates the file locally, so a wrong file type never leaves my machine. It then calls the API, stores the decision, the model version reported by the API, `inference_mode = remote` and the round-trip latency in the database, and shows the result page. If the API is down or returns an error, the user gets a 503 page and the attempt is still saved with `status = error`.

For this test the web app ran directly on my Windows laptop through `scripts/run_local.ps1`. The script creates a virtual environment, sets `FLASK_DEBUG=0` and keeps the history in a local SQLite file, using the same SQLAlchemy code that writes to PostgreSQL in the Compose stack (Chapter 5).

Browser — http://127.0.0.1:5000/health

GET http://127.0.0.1:5000/health (Flask app on my laptop)


```
{
  "database": "ok",
  "debug": false,
  "inference_mode": "remote",
  "model": {
    "api": "https://20-219-10-43.sslip.io",
    "model_name": "VisionGuard-23L-0764",
    "model_version": "8",
    "threshold": 0.62
  },
  "model_backend": "ok",
  "status": "ok",
  "time": "2026-11-05T16:59:49+00:00"
}
```


Figure 8.1: Web app health in remote mode: the model backend is the cloud API.

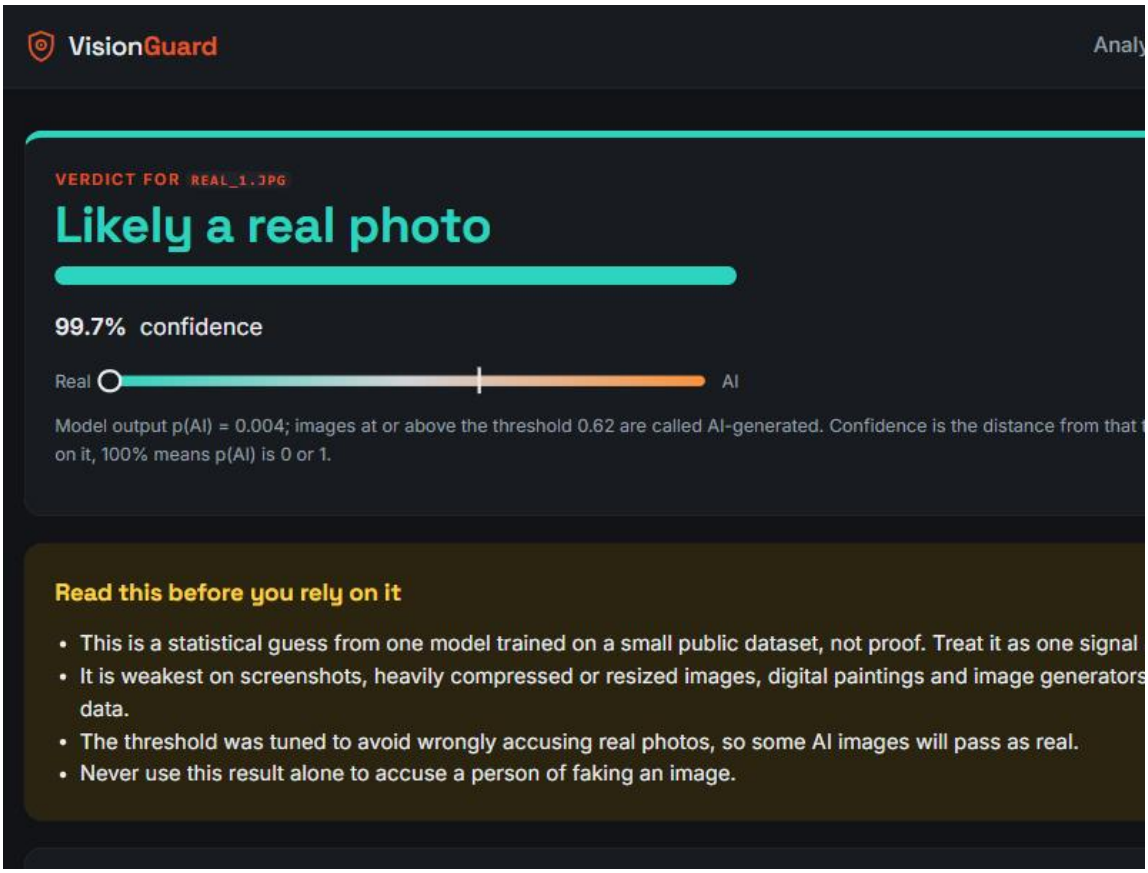


Figure 8.2: Upload of `real_1.jpg` through the UI, answered by the cloud model (record #5, 230 ms).

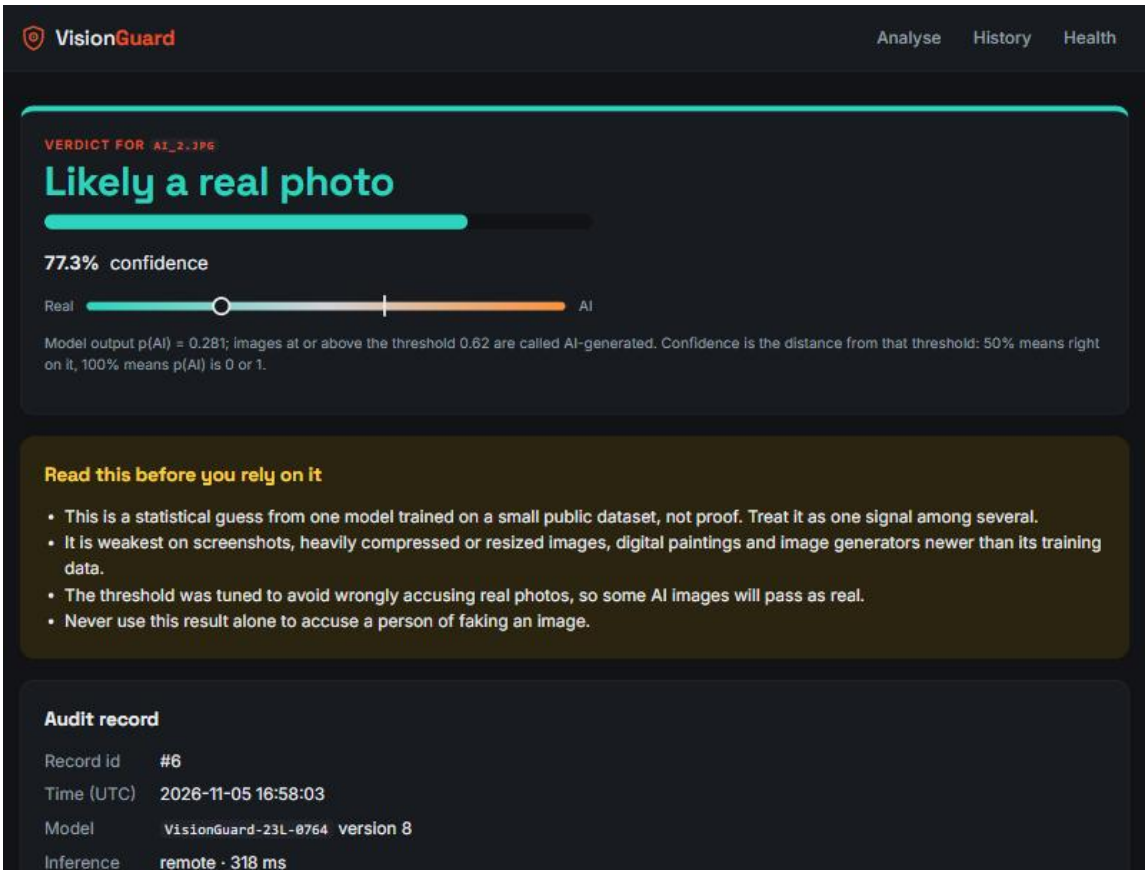


Figure 8.3: Upload of `ai_2.jpg`: an AI image that the model misses ($p(\text{AI}) = 0.28 < 0.62$).

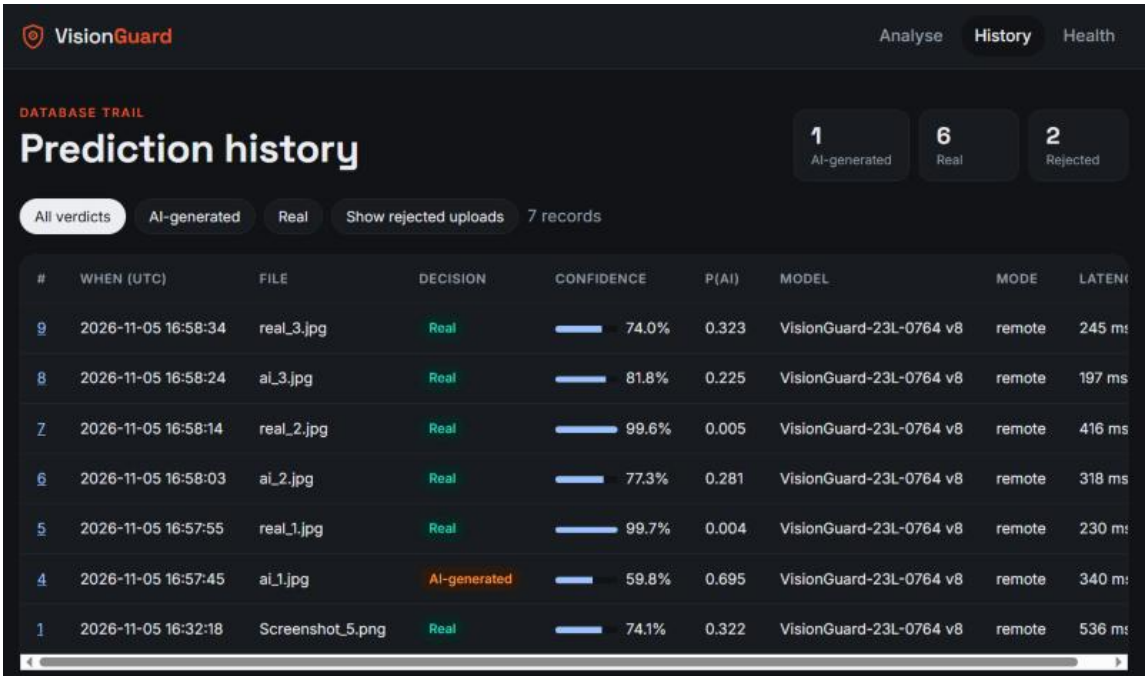


Figure 8.4: History menu: the database log, with `mode = remote` rows.

```
2026-11-05 16:57:45,399 INFO visionguard.api: predict ai_1.jpg -> AI-genera
ted (0.598) in 70 ms from 39.52.100.128
172.18.0.3 - - [05/Nov/2026:16:57:45 +0000] "POST /v1/predict HTTP/1.1" 200
 277 "-" "VisionGuard-web/1.0"
2026-11-05 16:57:55,724 INFO visionguard.api: predict real_1.jpg -> Real (0
.997) in 71 ms from 39.52.100.128
172.18.0.3 - - [05/Nov/2026:16:57:55 +0000] "POST /v1/predict HTTP/1.1" 200
 269 "-" "VisionGuard-web/1.0"
2026-11-05 16:58:03,840 INFO visionguard.api: predict ai_2.jpg -> Real (0.7
73) in 73 ms from 39.52.100.128
172.18.0.3 - - [05/Nov/2026:16:58:03 +0000] "POST /v1/predict HTTP/1.1" 200
 269 "-" "VisionGuard-web/1.0"
2026-11-05 16:58:14,872 INFO visionguard.api: predict real_2.jpg -> Real (0
.996) in 77 ms from 39.52.100.128
172.18.0.3 - - [05/Nov/2026:16:58:14 +0000] "POST /v1/predict HTTP/1.1" 200
 271 "-" "VisionGuard-web/1.0"
2026-11-05 16:58:24,446 INFO visionguard.api: predict ai_3.jpg -> Real (0.8
18) in 69 ms from 39.52.100.128
172.18.0.3 - - [05/Nov/2026:16:58:24 +0000] "POST /v1/predict HTTP/1.1" 200
 268 "-" "VisionGuard-web/1.0"
2026-11-05 16:58:33,952 INFO visionguard.api: predict real_3.jpg -> Real (0
.740) in 71 ms from 39.52.100.128
172.18.0.3 - - [05/Nov/2026:16:58:33 +0000] "POST /v1/predict HTTP/1.1" 200
 270 "-" "VisionGuard-web/1.0"
azureuser@vm-23L-0764-A2-Ops 2026-11-05 17:01:45 UTC ~/23L-0764_A2_Ops$
```

Figure 8.5: The same requests arriving in the API logs on the Azure VM.

To test it with several inputs I uploaded six held-out test images from my laptop through the UI, three AI-generated and three real. The cloud model (version 8) answered all six and each one was logged with `mode = remote`. All three real photos were classified correctly, two of them very clearly ($p(\text{AI})$ of 0.004 and 0.005) and one less so (0.32). One of the three AI images was caught, with $p(\text{AI}) = 0.69$, just above the 0.62 threshold. The other two (0.28 and 0.23) were called real, so the model got 4 out of 6 right.

That is about what I expected from the threshold I chose in Section 2.5. It favours precision, and indeed no real photo was accused, but the price is recall on the AI class, which is 0.66 on the test set. The API logs show about 70 ms of model time per image on the VM, while the round trip seen by the web app, from Lahore to Central India over HTTPS, was between 197 and 416 ms. The earlier rejected uploads (415 and 422) never reached the API at all, because the web app validates files before sending them.

9. Conclusion and Limitations

The whole pipeline can be reproduced from start to end. DVC pins the data version, every run records its commit, data version, seed, parameters, metrics and artifacts in MLflow on DagsHub, the served model is picked by a written rule on validation data and promoted with a registry alias, and the system starts with one `docker compose up`. The model API runs on Azure behind HTTPS and an API key, and the web app stores every decision with its time, model version and confidence.

The model itself has clear limits. The dataset is small and scraped. The AI images come from a few generators, and the "real" images include edited and stock photos, classical paintings and clip-art. The error analysis and Grad-CAM (Section 2.6) show that the model partly learned style, treating painterly images as AI and photographic ones as real, instead of real generation artefacts. On new generators, screenshots or images re-compressed by social media it will do worse than the test numbers suggest. Size, format and compression also differ between the classes (Chapter 1), so even after resizing some of what the model learned may be those shortcuts.

The metrics are noisy as well. A single train/validation/test split of a few hundred images moves noticeably with small changes, and cross-validation or a bigger, more varied dataset would make the comparison in Section 3.5 more reliable. On the deployment side, one VM is a single point of failure, and there is no autoscaling, rate limiting or monitoring of prediction drift yet.

If I continued the project, I would train on a larger dataset with more generators, add JPEG re-compression as a robustness test at evaluation time, log production predictions back to MLflow to watch for drift, and have CI deploy the API image on every tagged release.